

68000 · Z80 · YM2610

NEO GEO SDK

Programmer's Manual and Library Reference

Eagle Software

Release v1.7.0 · MIT

<https://github.com/eaglesoftware777/neogeosdk>

Contents

Part 1 · NEO·GEO SDK — Overview

NEOGEOSDK_v1.7.0.md

- 01 / OVERVIEW — The pitch
- 02 / HARDWARE — The target machine
- PART ONE — For everyone
 - 03 / NEO GEO DEV 101
 - 04 / GRAPHICS — How a frame is drawn
 - 05 / MEMORY — VRAM access via LSPC
 - 06 / TIMING — Interrupts
 - 07 / IPC — 68000 ↔ Z80 communication
 - 08 / COLOUR — Palettes
 - 09 / PHILOSOPHY — Design intent
 - 10 / STRUCTURE — Repository map
 - 11 / BUILD — End-to-end pipeline
 - 12 / TOOLCHAINS
 - 13 / TARGETS — What you can build
- PART TWO — For the programmer
 - 14 / THE GAME CONTRACT
 - 15 / THE 2D ENGINE
 - 16 / GRAPHICS PIPELINE — Artbox
 - 17 / AUDIO — The stack
 - 18 / THE SOUND DRIVER
 - 19 / THE FRAME BUDGET
 - 20 / THE SHIPPED GAMES
 - 21 / SETUP
 - 22 / COMMON SYMPTOMS
 - 23 / DOCUMENTATION
 - 24 / RELEASE — What v1.7.0 adds
 - 25 / END

Part 2 · Introduction

docs/INTRODUCTION.md

1. What this SDK is
2. The target machine
3. What is in the box
4. The two engines
5. Where to go next
6. Licence and intended use

Part 3 · Programmer's Manual

docs/PROGRAMMERS_MANUAL.md

Part I — The build

1. Two makefiles, one build
2. Selecting a game
3. The five stages
4. Build behaviour worth knowing
5. Debugging

Part II — Writing the game

6. The contract
7. Project layout

- 8. Driving the engine
- 9. Characters
- 10. Sprites, slots, and priority
- 11. The FIX layer
- 12. Palettes and the backdrop
- 13. Input
- Part III — Assets
 - 14. Art
 - 15. Sound
- Part IV — Staying inside the frame
 - 16. The budget
 - 17. The rules
- Appendix A — Register quick reference
- Appendix B — Interrupts
- Appendix C — Common symptoms

Part 4 · Toolchain 3.0

`docs/TOOLCHAIN.md`

- Contents
- Code generation
- Language standards
- Linking without an operating system
- Layout
- Use with the SDK
- Verified

Part 5 · Dependencies

`docs/DEPENDENCIES.md`

- Linux / WSL
- Windows
- Bundled tools (no install needed)
- Version summary

Part 6 · Tutorial: Hello World

`docs/HELLO_WORLD.md`

- 0. Before you start
- 1. Build and run it
- 2. What actually ran
- 3. Change something
- 4. Add a sprite
- 5. Turn it into your own game
- 6. Stepping up to the engine
- 7. When it does not work

Part 7 · Adding a New Game

`docs/ADDING_A_GAME.md`

- 1. Create the game directory tree
- 2. Write `games/mygame/game.mk`
- 3. Implement the required entry points
- 4. Build the game
- 5. Per-game ROM folder
- 6. Per-game hash XML
- 7. Test in MAME
- 8. FIX layer text

- 9. Using the 2D engine
- 10. Linker script
- 11. Sound assets (optional)
- 12. Distribution package

Part 8 · The Shipped Games

`docs/GAMES.md`

- demo — the engine reel (id 777)
- demo_plus — the C++ engine (id 778)
- skylance — Sky Lance (id 779)
- helloworld (id 772)
- tutorial (id 555)
- neogeogame (id 775)
- Choosing a starting point

Part 9 · The Demo Chapter Map

`docs/DEMO_CHAPTERS.md`

- The chapters
- Reading the reel as documentation
- Page colours
- FIX row map

Part 10 · Makefile Integration

`docs/MAKEFILE_INTEGRATION.md`

- Current object lists
- Current linker state
- Minimal game-engine-enabled game pattern
- Default module set
- Sound hook integration
- Debug and GDB Trace Targets

Part 11 · Debugging with GDB

`docs/GDB_GUIDE.md`

- Overview
- Installing GDB
- Debug build
- `make gdb`
- `make gdb-remote`
- Debugging both CPUs
- `make gdb-trace`
- Useful GDB commands for 68000 NeoGeo targets
- Linker map
- Overriding GDB path
- Notes

Part 12 · ROM Bank Switching

`docs/BANKSWITCH.md`

- Background
- Bank switch register
- SDK API
- How to use it (advisory)
- Current demo limits
- References

Part 13 · SDK Library Reference

SDK_API_GUIDE.md

- Sprite Control Block Helpers
- Palette and VRAM Helpers
- FIX Layer and HUD Helpers
- Timing and Housekeeping Helpers
- Input and Status Helpers
- Sound API
- 2D Game Engine Layer
- Typical Call Patterns
- Notes

Part 14 · 2D Engine Layer Guide

docs/GAME_ENGINE_LAYER.md

- Module overview
- VRAM and CRAM write contract
- Headers
- Startup
- Per-frame call
- Optional frame hooks
- Characters
- Sprite binding
- Action scripts
- Level and Camera
- NPCs
- Physics
- FIX Cache
- Game events
- Timers, progress, status, properties
- Border constraints
- Runtime limits
- Important build note
- Compatibility aliases

Part 15 · The FIX SDK Module

docs/NG_FIX_SDK.md

- Files
- Core ideas
- Basic text
- Raw tile output
- Custom font mapping
- Numbers, boxes, bars
- Compatibility

Part 16 · Scene and Video Ownership

docs/NEOUNIVERSAL_VIDEO_MODEL.md

- Purpose
- Rules
- Result

Part 17 · 2D Engine Reference — C

docs/API_2D_ENGINE_C.md

- Contents
- Core

- ng_defs
- ng_engine
- ng_game_events
- ng_game_interrupt
- ng_game_time
- ng_progress
- ng_properties
- ng_runtime
- ng_scene
- ng_status
- ng_timers

Math

- ng_fixed

Video

- ng_art_asset
- ng_bg
- ng_depthfx
- ng_fix
- ng_palette_assets
- ng_palette_fx
- ng_render_queue
- ng_sprite_group
- ng_sprite_pool
- ng_sprite_window
- ng_vram

Characters

- ng_actions
- ng_border_constraints
- ng_chars
- ng_joystick
- ng_npcs
- ng_physics

World

- ng_camera
- ng_level

Effects

- ng_debug
- ng_demo_advanced
- ng_feedback
- ng_particles

Part 18 • 2D Engine Reference — C++

[docs/API_2D_ENGINE_CPP.md](#)

- What this build adds
- Choosing between the builds
- Classes and methods
- Free functions

Part 19 • Deluxe 2D Module Guides

[docs/DELUXE_MODULES.md](#)

- Camera System
- Depth FX
- Palette FX
- Particle System

Render Queue
Sprite Groups
Performance Rules

Part 20 • Sound Driver Reference

docs/SOUND_DRIVER.md

1. Architecture
2. The 68000-side API
3. The raw command protocol
4. Content pipelines
5. Practices that keep audio stable
6. What the demo shows

Part 21 • Sound Studio Guide

docs/SOUND_STUDIO_GUIDE.md

Tabs
Typical Workflow

Part 22 • Graphics Pipeline Reference

docs/ARTBOX_PIPELINE.md

The colour target
Default quantiser
Resampling
Palette banks
Import size
Synthesised backdrops
Tile stride
Palette RAM budget
Palette anchors
HD conditioning
Alternative pipelines
Asset ordering and ids
Generated outputs
Verifying a build
Artbox Studio
Inputs
Rule Fields
Categories
Generated Files
Build
Runtime Use
Artbox Studio UI

Part 23 • Tools and Utilities

docs/TOOLS.md

1. Graphics — artbox/
 - 1.1 Conversion cores
 - 1.2 Pipeline stages
 - 1.3 Utilities
 - 1.4 Shell wrappers
 - 1.5 GIMP plug-ins — tools/gimp-plugins/
2. Sound — sound/tools/
 - 2.1 Compilers
 - 2.2 Sample encoding

- 2.3 Driver build and comparison
- 2.4 Sound Studio
- 3. Verification — tools/
- 4. Launchers and environment
 - 4.1 Repository root
 - 4.2 dist/
 - 4.3 install/
 - 4.4 win/
- 5. Desktop applications
 - 5.1 Artbox Studio — artbox/artbox_studio.py
 - 5.2 Sound Studio — sound/sound_studio.py
- 6. Debug artefacts

PART 1

NEO·GEO SDK — Overview

The pitch, the target hardware, and the design philosophy, in the same voice as the release presentation.

NEOGEOSDK_v1.7.0.md

EAGLE SOFTWARE · MIT · A HARDWARE-CENTERED SDK

Direct 68000-side control over VRAM, palettes, sprites, DMA and FIX tiles, and a custom YM2610 sound stack driven by a Z80 sound driver. **68000 · Z80 · YM2610** — SNK Arcade (MVS) and Home (AES) **TECHNICAL OVERVIEW · 2026**

01 / OVERVIEW — The pitch

Real Neo Geo development, made practical.

For self-made games, demos and homebrew that want to stay close to original Neo Geo development practice — while still using modern build tooling, scripting, and emulator-based test loops.

The goal is not to abstract the hardware away. The goal is to make working with it possible on Linux, WSL, and Windows.

Target	SNK Neo Geo — cartridge-era arcade hardware (MVS) and the home AES
Release	v1.7.0 · MIT licensed
Host platforms	Linux · WSL · Windows
Build languages	C99 · C++14 · 68000 asm · Z80 asm · Python
Audio layers	FM · SSG · ADPCM-A · ADPCM-B
Shipped games	6, all building to real ROMs
Engine builds	2 — plain C and C++14, same ABI

02 / HARDWARE — The target machine

Three chips. One bus. No engine.

	Chip	Role
Main CPU	M68000 @ 12 MHz	Game logic, VRAM, palettes, sprites, DMA, FIX tiles — directly
Sound CPU	Z80A @ 4 MHz	Runs the dedicated driver; talks to the YM2610 over command ports
Sound chip	YM2610	FM synthesis, SSG square waves, ADPCM-A and ADPCM-B sample playback

The development model

Graphics, palettes, FIX tiles, ROM layout and sound commands are part of the day-to-day programming model rather than hidden behind a large engine layer. The SDK preserves that — it gives you headers, build glue, asset pipelines and a 2D engine you can opt into, not abstractions you cannot see past.

PART ONE — For everyone

03 / NEO GEO DEV 101

Before the SDK matters, the hardware has to make sense. Six sections on what every Neo Geo programmer needs to know: the FIX layer, sprites, palettes, VRAM access, interrupts, and how the two CPUs talk.

04 / GRAPHICS — How a frame is drawn

Three layers, fixed priority. A frame is stacked, not composited.

FIX layer — 40 × 32 grid of 8 × 8 tiles in a fixed location, **28 rows visible**. Up to 15 colours per cell, 16 palettes, 4096 tiles. This is the HUD / text plane, and it is always on top. (128 KB ROM · S1)

Sprites — up to **380 on screen, 96 per scanline**. Each sprite is a vertical strip of up to 32 tiles of 16 × 16 pixels, with per-sprite vertical and horizontal *reduction* and a chain bit that welds a strip to its left neighbour. (up to 128 MB ROM · C ROMs)

Backdrop — a single solid colour behind everything else. One value from 65,536. That is the entire background layer. (1 colour)

Why this matters

There is no tile-mapped background plane. Every moving and static element — player, enemies, terrain, parallax — is built from sprites. Sprite count and the horizontal-per-line limit drive your scene budget.

Two rules that catch everyone

- **SCB2 can only shrink, never stretch.** Author art at its largest intended size.
- **HIGHER slot number is drawn IN FRONT.** Backgrounds belong at slots 1–32, characters at 96–223, foreground effects at 288+.

05 / MEMORY — VRAM access via LSPC

You don't poke VRAM directly. Talk to LSPC; it talks to VRAM.

Address	R/W	Register
\$3C0000	W	VRAM address (16-bit word index)
\$3C0000	R	Read data (address register unchanged)
\$3C0002	W	VRAM write data — writing pushes to VRAM
\$3C0004	R/W	Auto-increment — added to the address after every write
\$3C0006	W	Mode register — timer / IRQ / animation speed
\$3C0008	W	Timer high
\$3C000A	W	Timer low
\$3C000C	W	Interrupt clear (bit 1 timer, bit 2 VBL)

VRAM size 64 KB + 4 KB. All access is word-aligned; long-word and byte access are not allowed.

Timing rule ~1.4 μ s. Wait at least 16 CPU clocks after touching the address register before reading. Cross-bank writes cannot use auto-increment.

SDK shortcut The headers in sdk/ wrap these registers in C macros, so you write to named symbols rather than raw addresses.

06 / TIMING — Interrupts

Two interrupts. That's the whole frame loop.

IRQ 1 • V-BLANK — fires once per frame, vector \$64. This is your main game tick: read inputs, run logic, push the frame's sprite list into VRAM. The window is ~3 ms.

```
MOVE.W #4, $3C000C ; ack VBL
MOVE.B D0, $300001 ; kick watchdog
```

IRQ 2 • TIMER — fires when the LSPC timer counter hits 0, vector \$68. Decrements every 167 ns (one pixel). Used for raster effects: split scrolls, mid-frame palette swaps, status bars.

```
MOVE.W #2, $3C000C ; ack timer
```

Tip For "every N pixels", set mode-register bits 4–7 to %1001 and Timer = N–1. For multiple arbitrary scanlines, use mode %1101 and reload the timer inside the handler.

07 / IPC — 68000 ↔ Z80 communication

One byte at a time.

Direction	Port	What happens
68000 → Z80	\$320000 (write byte)	The game writes a 1-byte sound code. The Z80 reads it on its own port and dispatches to the right music / SFX / FM / SSG handler.
Z80 → 68000	\$320000 (read byte)	The Z80 can post a status byte back; the game reads from the same address.

SDK abstraction You don't write that byte by hand. You call `playMusic()` / `playSFX()` with a named id from `sdk/sound_ids.h`. The SDK encodes it; the Z80 driver decodes it.

One caveat A prefix byte and its parameter are two separate NMIs on the Z80. Space multi-byte commands with a `waitVbl()` or the second one can be dropped.

08 / COLOUR — Palettes

Colour is paletted, not direct. 4,096 on screen, of 65,536.

15 colours per character — 4 bits of palette index, colour 0 is transparent. 256 palettes total: 16 reserved for FIX, 239 for sprites, 1 for the backdrop.

Palette RAM \$400000–\$401FFF. Two banks, toggled via \$3A000F / \$3A001F for snappy palette swaps. The shadow bit at \$3A0011 dims the entire display — pause screens, hit flashes.

****The backdrop register is the last word of palette RAM, \$401FFE.**** \$402000 is a mirror that silently does nothing.

The colour word is not RGB444. Bit 15 is the dark bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top four bits of R, G and B — **5 bits per channel, 32 levels, in steps of 8**. The artbox quantiser snaps to exactly that lattice.

09 / PHILOSOPHY — Design intent

The machine is close to the metal by design. The SDK doesn't try to change that.

Preserve	Modernise	Resist
Original Neo Geo development practice — direct hardware register access	Build tooling, asset scripting, and emulator-based test loops on current operating systems	Heavy abstraction layers that hide what the hardware is actually doing

10 / STRUCTURE — Repository map

```

neogeosdk/
├─ sdk/                68000 headers, runtime, and both 2D engines
│  └─ 2d_engine/       the C engine (ng_*.c / ng_*.h)
│  └─ 2d_engine_plus/  the C++ engine (ng_*.cpp / ng_*.hpp), same ABI
│  └─ ng_fix/ ng_audio/ ng_video/ ng_scene/ ng_show/ bsp/
│  └─ neogeo.h macro.h hardware registers and inline helpers
│  └─ sound_ids.h      named music / SFX / FM / SSG ids
├─ games/              one directory per game, each self-contained
│  └─ demo/             the 26-chapter engine reel (id 777)
│  └─ demo_plus/        C++-engine smoke test (id 778)
│  └─ skylance/         Sky Lance, a complete shooter (id 779)
│  └─ helloworld/       minimal FIX text + one sample (id 772)
│  └─ tutorial/         minimal engine loop (id 555)
│  └─ neogeogame/       empty starting point (id 775)
├─ artbox/             PNG → sprite / FIX / C-ROM pipeline + Artbox Studio
├─ sound/              YM2610 stack: Z80 driver, MML, FM, SSG, samples
├─ docs/               the documentation set
├─ tools/              validators, the API generator, the manual builder
├─ dist/ roms/ hash_eagle/ build output and MAME hash sets
├─ Makefile            Linux + WSL build
└─ MakefileWin32.mak   native Windows build

```

Top-level entry games/<name>/main.c and user.c — the 68000-side game flow and the BIOS-hook startup handlers. main.c is the file you edit; user.c you almost never touch.

Asset pipelines artbox/ and sound/ — two parallel pipelines: PNG → sprite C-ROMs and FIX, plus WAV and MML → ADPCM and driver tables.

Output target roms/<game>/ — generated ROM sets kept in-tree so MAME test runs are instant.

11 / BUILD — End-to-end pipeline

Five stages. One `make all`.

Stage	Target	Produces
01	art	Sprite C-ROMs, screens.c, sprite_meta.h, palettes
02	sfix	The S1 FIX ROM, 128 KB
03	sound	ADPCM samples, V-ROM, and the M1 Z80 driver ROM
04	p1	The 68000 program ROM, 512 KB
05	*(in p1)*	hash_eagle/<game>/neogeo.xml — the MAME hash set

```
make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg all
make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg test
```

Every game carries its own `game.cfg`, and the build cross-checks `GAME=` against it — which is what stops you silently building one game's art into another game's ROM.

In day-to-day work you run `p1` almost exclusively. It takes seconds.

12 / TOOLCHAINS

Component	Tool
68000 C / C++	m68k-unknown-elf-gcc 16.2.0 (toolchain 3.0: binutils 2.47, gdb 17.2, newlib 4.6.0, libstdc++), freestanding, -march=68000
68000 link	m68k-unknown-elf-ld with a per-game <code>neogeo.ld</code>
ROM shaping	objcopy → srec_cat crop / byte-swap / pad
Z80 sound driver	wla-z80 + wlalink
Asset conversion	Python 3.10+, Pillow, NumPy
Desktop tools	PyQt6 (Artbox Studio, Sound Studio)
Test loop	MAME with a generated hash set
Debug	GDB, MAME debugger, MAME Lua scripting

C++ builds use `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics`. The games use no standard library, no `malloc`, no floating point and no order-dependent global constructors; the toolchain ships `newlib` and `libstdc++` for code that wants them. The same toolchain is provided for Linux (static) and Windows (no DLL dependencies) and the two produce identical ROMs; see `docs/TOOLCHAIN.md`.

13 / TARGETS — What you can build

```

make all          art + sfix + sound + pl
make pl          68000 program ROM only
make aes / mvs   force the AES or MVS platform variant
make art         rebuild sprite C-ROMs from artbox
make art-crt     same, through the CRT-preprocessing pipeline
make sfix / srom rebuild the S1 FIX ROM
make sound       samples + vrom + fmpatches + fm + mml + ssgconfig + ssg + mlrom
make mlrom       the shipping ASM Z80 driver build
make mlrom-c     the experimental C driver build
make compare-driver build both and diff the ROM images
make dist        package dist/roms/<game>.zip
make test        run in MAME (MVS, sp-s2.sp1 BIOS)
make test-aes    run in MAME (AES, unibios22)
make debug       run MAME with the debugger attached
make debug-build P1 with debug symbols and a linker map
make gdb / gdb-trace / gdb-remote
make bios-list / games-list / menu / help
make clean / sound-clean / art-clean / clean-all

```

PART TWO — For the programmer

14 / THE GAME CONTRACT

A Neo Geo cartridge boots into the BIOS, not into `main()`. The BIOS calls back for power-on, eye-catcher, game and title; `user.c` provides those hooks. What you write is two functions:

```

void game_boot(void);    /* once, after the BIOS hands over */
void game_frame(void);   /* once per frame, forever */

```

`game_frame()` returns every frame, so anything that must persist is static or file-scope. A plain local is gone before the next call — the most common first bug on this platform.

15 / THE 2D ENGINE

The SDK ships the same engine twice: plain C in `sdk/2d_engine/`, C++14 in `sdk/2d_engine_plus/`. Same design, same public extern "C" ABI, so a C game can link the C++ build and vice versa. One engine per ROM, selected with `USE_2D_PLUS=1`.

Module	Header	What it does
Frame tick	<code>ng_game_interrupt.h</code>	The whole engine tick, with five hook points
Characters	<code>ng_chars.h</code>	64-object pool: movement, animation, hitboxes, damage
Actions	<code>ng_actions.h</code>	Table-driven action scripts with sound and FX hooks
NPCs	<code>ng_npcs.h</code>	Patrol bounds, home position, think callbacks
Physics	<code>ng_physics.h</code>	Gravity, drag, solid rectangles, grounded tests
Level	<code>ng_level.h</code>	World bounds, scroll, camera helpers
Camera	<code>ng_camera.h</code>	Follow, dead zone, look-ahead, shake, cinematic pan
Render queue	<code>ng_render_queue.h</code>	128-slot VBlank-safe deferred VRAM/palette writes

Module	Header	What it does
Sprite groups	ng_sprite_group.h	Dirty-flag sticky-bit chains — write only what changed
Sprite pool	ng_sprite_pool.h	The hardware slot map
FIX	ng_fix.h	Text layer with a dirty-cell cache
Backgrounds	ng_bg.h	Two scrolling sprite layers with parallax factors
Palette FX	ng_palette_fx.h	Fade, flash, pulse, colour cycle — all queue-safe
Particles	ng_particles.h	32-slot fixed pool, typed spawns, priority eviction
Feedback	ng_feedback.h	Hitstop + shake + flash + sound hook in one call
Depth FX	ng_depthfx.h	Perspective projection, Z → shrink, fog, starfields
Joystick	ng_joystick.h	Edge detection, repeat, motion specials
Fixed point	ng_fixed.h	16.16 arithmetic, sin/cos and shrink tables
Debug	ng_debug.h	FIX-layer performance overlay

Full references: docs/API_2D_ENGINE_C.md, docs/API_2D_ENGINE_CPP.md.

16 / GRAPHICS PIPELINE — Artbox

Drop PNGs into games/<game>/artbox/in/<category>/, run `make art`. The pipeline quantises to the Neo Geo colour lattice, derives palettes, packs tiles into C-ROMs, and generates `screens.c` (one `showScreenN()` per asset) and `sprite_meta.h` (geometry per asset).

The v1.7.0 quantiser

Four ideas, and together they took mean colour error from dE 9.10 to 8.33 and p95 from 18.19 to 15.66 across the reference assets, with measured dither speckle down a third.

Match in Lab, not RGB. Palette selection, k-means refinement, and the final per-pixel remap all run in CIE-Lab through a precomputed 32^3 lattice LUT. Matching in one space and measuring in another is what makes a quantiser that scores well look wrong.

Refit the palette onto the hardware grid, then re-optimize. k-means picks colours in continuous space; snapping them to the 32-level lattice moves every centroid. Lloyd iterations with the snap *inside* the update step, keeping a move only when total error drops, and re-seeding duplicate or unowned slots from the worst-served pixels.

Blue-noise dither, and only where it helps. A 32×32 void-and-cluster mask orders the mix between the two nearest palette entries along a serpentine scan. A dead band skips the mix entirely within 18 % of either endpoint — those are the pixels where dithering adds speckle without adding colour.

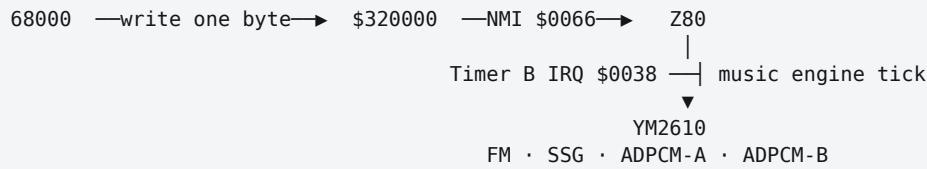
Push contrast and saturation before clustering, with a soft knee. Fifteen colours pull everything toward the middle of the gamut, because the average of a cluster is always less saturated than its members. Compensate before clustering, and compress the extremes with `tanh` rather than clipping — clipping folds every boosted highlight onto pure white, throwing colour detail away before k-means ever sees it.

Ordering

Categories are processed as backgrounds, characters, effects, eyecatcher, npc, screens, titles, then any other directory alphabetically. **An asset id is its 1-based position in that sequence —**

inserting a file in the middle renumbers everything after it.

17 / AUDIO — The stack



Area	Channels	Use
ADPCM-A	6, one-shot	Short sound effects, voice samples
ADPCM-B	1 stream	Backing tracks, ambience, long cues
SSG	3 square waves	MML melody, plus a standalone sequencer
FM	Compiled sequencer, 16-patch bank	Music, stings, formant effects

Content pipelines

Source	Compiler	Target
sound/mml/*.mml	mml_compile.py	mml
sound/fm/*.fm	fm_compile.py	fm
sound/fm/patches.fm	fm_patch_compile.py	fmpatches
sound/ssg/*	ssg_compile.py	ssg
sound/samples/in_wav_a/*.wav	adpcm_enc.py	samples
sound/samples/in_wav_b/*.wav	adpcm_enc.py	samples
voice samples	vrom.py	vrom
all of the above	wla-z80 + wlalink	mlrom

Full reference: docs/SOUND_DRIVER.md.

18 / THE SOUND DRIVER

soundInit() once at boot, then named calls with ids from sdk/sound_ids.h:

```

playMusic(SOUND_MUSIC_E);    playSFX(SOUND_SFX_7);
playSFXB(track);            playFMTrack(SOUND_FM_A);
playSSGTrack(SOUND_SSG_B);   soundSetSSGPreset(3);
soundFadeOut();              soundSceneReset();
speakText("READY!");
  
```

Beyond playback the driver exposes chip-level control — ADPCM-B pan (reg \$11), FM LFO (reg \$22), SSG noise period (reg \$06) — and CSM, Composite Sine Mode, which auto-keys FM channel 2 from Timer A for formant and vowel-slide effects.

Practices that keep audio stable: initialise once; use `soundSceneReset()` on scene change rather than stacking stops; space multi-byte commands with `waitVbl()`; fade before you stop; and budget ADPCM-A effects the way you budget sprites — there are six channels and a seventh trigger steals one.

19 / THE FRAME BUDGET

Resource	Limit
CPU	~12 MHz 68000, ~200,000 cycles per frame
VBlank	~3 ms — all VRAM writes must fit
Sprites	380 total, 96 per scanline
Palettes	16 entries per bank, 256 banks
FIX	40 × 32 cells, 28 rows visible, 16 palettes

****No malloc during gameplay.**** Every engine subsystem uses a fixed pool: characters 64, NPCs 32, particles 32, palette FX 8, render queue 128.

****No float or double.**** There is no FPU; every float costs hundreds of cycles in software emulation. Use 16.16 fixed-point.

No division in the frame loop. DIVS/DIVU cost 76–158 cycles each. Use `ng_sin_tab[]`, `ng_cos_tab[]`, `ng_shrink_tab[]`, and right shifts.

Write only what changed. Sprite groups carry dirty flags; the FIX layer has a dirty-cell cache.

Queue every VRAM write. Post to the render queue from game logic; let `ng_render_queue_flush()` drain it in VBlank.

20 / THE SHIPPED GAMES

Game	Id	Engine	What it is
demo	777	C	The 26-chapter engine reel
demo_plus	778	C++	The same engine through the C++ API
skylance	779	C	Sky Lance — a complete vertical shooter
helloworld	772	—	FIX text and one sample; the tutorial target
tutorial	555	C	The minimal engine loop
neogeogame	775	C	An empty starting point

Sky Lance

Three pilots — ROOK (BLUE LANCE), KIRA (GREEN WING), BLAZE (RED STRIKER) — differentiated by fire rate, speed and single vs. twin bolt rather than by damage numbers. Seven stages, each running 5–7 squadrons before its boss: CRIMSON KEEP, IRON TIDE, SOL CORE, NIGHT RAZOR, ROTOR NEST, EARTH HAMMER, SPIRE GOD.

Two hardware lessons are visible in its source: the playfield is the 256 px column the backdrop covers, not the full 320 px screen, because a page is 16 × 16 tiles; and the backdrops had to be authored to tile seamlessly, because the sprite chip can shrink but never stretch.

The demo reel

26 chapters, each isolating one subsystem, with the chapter number printed top-right so a problem can be reported by number. **A** advances, **C** restarts.

BOOT · TITLE · FIX LAYER · FIX FX · SPRITE SCREENS · CHARACTERS · CHAR SELECT · PHYSICS · CAMERA LAB · PALETTE FX · PARTICLES · PARTICLE LOAD · FEEDBACK · DEPTH FX · DEPTH PARALLAX · NPCS · MINI-GAME · JOYSTICK · SCROLL LEVEL · CHAR 2D · TARGET RANGE · DEPTH RIDE · SOUND · SKY LANCE · STAR RAID LANCE · CREDITS

21 / SETUP

```
# 1. install the toolchain
install/install-linux.sh          # or install-ubuntu.sh / install-wsl.bat
                                  # or install-windows.bat

# 2. check it
m68k-unknown-elf-gcc --version
wla-z80 --version
python3 --version                # 3.10+, with Pillow and NumPy

# 3. build and run
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test
```

Set SDKHOME to the checkout. See docs/DEPENDENCIES.md and the installation sections of README.md — including the WSL2 + Ubuntu 24.04 + PulseAudio walkthrough for working audio under WSL.

22 / COMMON SYMPTOMS

Symptom	Cause
Sprite invisible behind the background	Slot priority — HIGHER is in front
Sprite too large, cannot shrink further	SCB2 only reduces; re-author the art
Backdrop colour ignored	Wrote \$402000 instead of \$401FFE
Text draws as solid blocks	The BIOS space glyph is opaque; use <code>ng_fix_blank_cell()</code>
FIX palette wraps onto text colours	Only 4 palette bits — fold duplicates below 16 banks
Pool fills and spawns stop	Objects marked destroyed but never <code>ng_chars_remove()</code> d
Loop skips the last objects	<code>ng_chars_count()</code> used as an array bound
Second byte of a sound command lost	Prefix and parameter not spaced by <code>waitVbl()</code>
Sprites dropped along one row	The 96-per-scanline limit
Green self-test screen on boot	The BIOS self-test, not your ROM — try <code>BIOS=unibios40</code>
Tearing or corrupt tiles	VRAM written outside VBlank instead of queued

23 / DOCUMENTATION

Document	For
docs/INTRODUCTION.md	What the SDK is, what the machine is, the reading order
docs/HELLO_WORLD.md	Your first ROM, in ten minutes
docs/PROGRAMMERS_MANUAL.md	Build, layout, engine tick, assets, frame budget
docs/API_2D_ENGINE_C.md	Every call in the C 2D engine
docs/API_2D_ENGINE_CPP.md	The C++14 build of the same engine
SDK_API_GUIDE.md	The bare-metal 68000 helpers
docs/SOUND_DRIVER.md	The Z80 driver, the YM2610, the audio pipelines
docs/ARTBOX_PIPELINE.md	PNG to C-ROM, the quantiser, Artbox Studio
docs/GAMES.md	What each shipped game demonstrates
docs/DEMO_CHAPTERS.md	The 25 demo chapters
docs/TOOLS.md	Every helper script — art, sound, verification, launchers, GIMP plug-ins
docs/ADDING_A_GAME.md	Starting your own project
docs/GAME_ENGINE_LAYER.md	How the engine tick fits together
docs/performance_rules.md	Staying inside the frame
docs/GDB_GUIDE.md	Debugging on real timing
docs/neogeosdk_v1.7.0_manual.pdf	The printable manual and API reference

24 / RELEASE — What v1.7.0 adds

v1.7.0 consolidates the whole neo_universal_2d line of work — 226 commits over the mainline. It is the largest release the SDK has had.

The big pieces

Area	What arrived
Tool chain 3.0	GCC 16.2.0, binutils 2.47, gdb 17.2, newlib 4.6.0, libstdc++; static Linux binaries and DLL-free Windows executables that build identical ROMs
2D engine	35 modules, built from nothing in thirteen staged passes: characters, actions, physics, NPCs, camera, level, particles, feedback, depth FX, palette FX, sprite groups, render queue, FIX layer, input, timers, events, fixed-point math
C++14 engine	The same engine with member methods and singletons, identical external "C" ABI, selected with USE_2D_PLUS=1
Supporting layers	sdk/ng_fix (a standalone FIX SDK), plus ng_audio, ng_scene, ng_show, ng_video, bsp

Area	What arrived
Multi-game builds	Per-game id, ROM folder, artbox, sound tree, linker script and MAME hash set, with a game.cfg cross-check that stops one game's art building into another's ROM
Windows	Native MakefileWin32.mak builds, the shipped ROMs built from the Windows toolchain, CRLF churn stopped
Installers	One-shot setup for Linux, Ubuntu, native Windows and WSL
Art pipeline	Lab clustering on the hardware lattice, blue-noise dithering, budgeted per-tile palette banks carried through both engines, linear-light resampling, import at display size, halo removal, non-destructive screen fitting, CRT and HD alternative routes, and verification that decodes the built C-ROMs
Audio	A driver that keeps musical time (124 Hz tick, live BPM), nine ADPCM-B beds with hardware repeat, eight FM tracks, nine SSG tracks, a recorded voice bank, pan / LFO / noise / tempo / CSM control, a working fade engine, hardware-exact ADPCM encoders, and the 68000-side acknowledgement rule
Games	Sky Lance, the 26-chapter reel, demo_plus, neogeogame, and a working template set
Verification	Host renderer tests for both engines, art and sound tool tests, emulator captures of VRAM, palette RAM, controls and recorded audio
Desktop tools	Artbox Studio and Sound Studio, both PyQt6
Documentation	The set listed in section 23, including generated API references and this document

Engine correctness — found by observation

A long arc of hardware-behaviour fixes, each of which changed how the SDK behaves rather than only how it reads.

- **Sprite teardown rewritten** — a full 64-word SCB1 wipe with blank tile 0xFFFF (not tile 0, which may carry art), sprites parked off-screen, a two-tier teardown with a previous-strips tail clear, and per-window VRAM footprint tracking. Ghost sprites persisting across scene transitions were the visible symptom.
- **Sprite slot priority reversed** — HIGHER is in front. The earlier claim was the opposite, and backgrounds parked at "behind" slots were drawing over characters.
- **The backdrop register corrected** to \$401FFE. \$402000 is a mirror that silently does nothing.
- **The FIX layer's visible row origin corrected** — 28 visible rows, visible row y is map row y + 2.
- ****REG_PALBANK0 / REG_PALBANK1 address swap fixed.****
- **Per-axis camera dead zone fixed.**
- ****START_GAME syncs to VBlank before clearing sprites**, removing the black-box flash on entry.**
- ****Weak fallbacks for ng_screen_table[] / ng_screen_count**** so a game shipping no screen art can link at all.
- **The Z80 polling race fixed** — soundCommand called isZ80Ready() twice; the trailing call triggered an NMI whose handler cleared the sound port, so the 68000 polling loop read 0, re-triggered, and deadlocked permanently. A frozen white screen with no sound.
- **The acknowledgement rule** — the reply port drops to 0 while the driver reads a byte and rises to 1 once it is queued, and it idles at 1 because the BIOS requires it. soundCommand() now waits for

the drop before the rise; sent on the stale 1, a two-byte command lost its parameter and the music bed that followed never started.

- **SCB3 is the sprite's height on screen in characters**, whatever the shrink register says, and the two shrink axes are read differently — X from the top nibble, Y from the whole byte.

One thing that was tried and removed

An SSG envelope and three-formant phoneme speech engine was built and evaluated on hardware, then removed: a pure YM2610 SSG path cannot synthesise intelligible speech regardless of driver tricks. Recorded ADPCM-A samples are the shipping answer, and `speakText()` is built on them.

Full detail, subsystem by subsystem: `CHANGELOG.md`.

25 / END

Neo Geo SDK v1.7.0 • Eagle Software • MIT

Repository: <https://github.com/eaglesoftware777/neogeosdk>

The project is intended only for lawful homebrew development, technical research, education, preservation-oriented study, or authorised consulting work. It must not be used for unauthorised copying, distribution, reproduction, sale, conversion, or operation of copyrighted commercial software.

PART 2

Introduction

What the SDK is, the machine it targets, and how to read the rest.

`docs/INTRODUCTION.md`

Eagle Software · v1.7.0 · MIT

This is the document to read first. It explains what the SDK is, what the machine underneath it looks like, what you get in the box, and the order in which the rest of the documentation is meant to be read.

1. What this SDK is

The Neo Geo SDK is a hardware-centered development kit for SNK Neo Geo cartridge hardware — the MVS arcade board and the AES home console. It gives you a 68000 cross-build, a Z80 sound stack, two asset pipelines, a 2D game engine in both C and C++, six working example games, and an emulator test loop that runs from make.

It deliberately does **not** hide the hardware.

On a Neo Geo, graphics, palettes, FIX tiles, ROM layout, and sound commands are part of day-to-day programming rather than something an engine abstracts away. That is the machine's actual programming model, and it is the reason Neo Geo software looks and performs the way it does. The SDK preserves that model and modernises everything around it: the build, the asset conversion, the debugging, and the test cycle.

The SDK preserves	The SDK modernises	The SDK resists
Direct register access, real VRAM/palette writes, hardware sprite budgets	Build tooling on Linux, WSL and Windows; Python asset pipelines; MAME-driven test loops; GDB	Heavy abstraction layers that hide what the hardware is doing

2. The target machine

Three chips, one bus, no operating system.

Part	Chip	Role
Main CPU	Motorola 68000 @ 12 MHz	Game logic, VRAM, palettes, sprites, DMA, FIX tiles
Sound CPU	Zilog Z80A @ 4 MHz	Runs the sound driver; talks to the YM2610 over command ports
Sound chip	Yamaha YM2610	FM synthesis, SSG square waves, ADPCM-A and ADPCM-B sample playback

How a frame is drawn

A Neo Geo frame is **stacked, not composited**. Three fixed layers, fixed priority:

1. **FIX layer** — a 40 × 32 cell grid of 8 × 8 tiles, of which **28 rows are visible**, in a fixed screen position. 15 colours per cell, 16 palettes, 4096 tiles. This is the HUD and text plane, and it is always on top.
2. **Sprites** — up to **381 sprites** on screen, **96 per scanline**. Each sprite is a vertical strip of up to 32 tiles of 16 × 16 pixels, with per-sprite horizontal and vertical *reduction* and a chain ("sticky") bit that welds a strip to its left neighbour.
3. **Backdrop** — one solid colour behind everything, from the 65,536-colour pool. That is the entire background layer.

There is no tile-mapped background plane. Every moving and static element — player, enemies, terrain, parallax — is built out of sprites. Sprite count and the per-line limit are what drive your scene

budget.

Two facts catch every newcomer:

- **Sprites can only shrink, never stretch.** The SCB2 scale field is a *reduction* factor. Art must be authored at its largest intended size.
- **Lower sprite slot number is drawn IN FRONT.** Putting a background at slot 1 puts it in front of your hero. Backgrounds belong at high slots.

Colour

Colour is paletted, not direct. 15 colours per character (4 bits of index; index 0 is transparent), 256 palettes of 16 entries in palette RAM at \$400000–\$401FFF, giving 4,096 colours on screen chosen from 65,536.

A Neo Geo colour word is not plain RGB444. Bit 15 is the "dark" bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top 4 bits of R, G and B. The result is **5 bits per channel, 32 levels, in steps of 8** — which is why the artbox quantiser snaps to that lattice rather than to a 4-bit grid.

The **backdrop register is the last word of palette RAM**, at \$401FFE. \$402000 is a mirror that silently does nothing.

Timing

Two interrupts, and that is the whole frame loop.

IRQ	Source	Vect or	Use
1	Vertical blank	\$64	The game tick: read input, run logic, push the frame's sprite list into VRAM
2	LSPC raster timer	\$68	Raster effects: split scrolls, mid-frame palette swaps, status bars

VBlank is roughly 3 ms wide. Everything that writes VRAM from gameplay should be queued and flushed inside it — which is what the engine's render queue does for you.

VRAM

You do not poke VRAM directly. You talk to the LSPC, and it talks to VRAM.

Address	R/W	Register
\$3C0000	W	VRAM address (16-bit word index)
\$3C0000	R	Read data (address register unchanged)
\$3C0002	W	VRAM write data — writing pushes to VRAM
\$3C0004	R/W	Auto-increment, added to the address after every write
\$3C0006	W	Mode register — timer / IRQ / auto-animation speed
\$3C0008	W	Timer high
\$3C000A	W	Timer low
\$3C000C	W	Interrupt clear (bit 1 timer, bit 2 VBL)

All access is word-aligned; long-word and byte access are not allowed. Wait at least 1.4 μs (16 CPU clocks) after touching the address register before reading. The SDK wraps all of this in named C helpers.

The two CPUs talking

One byte at a time. The 68000 writes a sound code to \$320000; the Z80 reads it on its own port and dispatches to the right music, SFX, FM or SSG handler. The Z80 can post a status byte back through the same address.

You do not write that byte by hand — you call `playMusic()` / `playSFX()` with a named id from `sdk/sound_ids.h`, the SDK encodes it, and the Z80 driver decodes it.

3. What is in the box

```
neogeosdk/
├── sdk/                68000 headers, runtime, and both 2D engines
│   ├── 2d_engine/      the C engine (ng_*.c / ng_*.h)
│   ├── 2d_engine_plus/ the C++ engine (ng_*.cpp / ng_*.hpp), same ABI
│   ├── ng_fix/ ng_audio/ ng_video/ ng_scene/ ng_show/ bsp/
│   ├── neogeo.h macro.h hardware registers and inline helpers
│   └── sound_ids.h      named music / SFX / FM / SSG ids
├── games/              one directory per game, each self-contained
│   ├── demo/           the 26-chapter engine reel (id 777)
│   ├── demo_plus/      C++-engine smoke test (id 778)
│   ├── skylance/       Sky Lance, a complete shooter (id 779)
│   ├── helloworld/     minimal FIX-text + one sample (id 772)
│   ├── tutorial/        minimal engine-loop skeleton (id 555)
│   └── neogegame/       empty starting point for your game(id 775)
├── artbox/             PNG → sprite/FIX/C-ROM pipeline + Artbox Studio
├── sound/              YM2610 stack: Z80 driver, MML, FM, SSG, samples
├── docs/               this documentation set
├── tools/              validators and the manual generator
├── dist/ roms/ hash_eagle/ build output and MAME hash sets
├── Makefile            Linux / WSL build
└── MakefileWin32.mak   native Windows build
```

4. The two engines

The SDK ships the same 2D engine twice: once in plain C (`sdk/2d_engine/`), once in C++14 (`sdk/2d_engine_plus/`). They are not different engines — they are the same design with the same public extern "C" ABI, so a C game can link against the C++ build and vice versa.

Build	Choose it when
<code>sdk/2d_engine</code>	You are writing C. Smallest code-size footprint, simplest call chain. This is the default.
<code>sdk/2d_engine_plus</code>	You are writing C++. You want member methods (<code>cam.follow(...)</code>), singletons (<code>CharManager::instance()</code>), and the scene-stack helpers.

Pick one per ROM with the `USE_2D_PLUS` make flag:

```
make p1          # link the C engine (default)
make p1 USE_2D_PLUS=1 # link the C++ engine
```

The C++ build compiles under `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics -ffreestanding -march=68000`. No standard library, no `malloc`, no floating point, no order-dependent global constructors.

Full call-by-call references:

- `API_2D_ENGINE_C.md`
- `API_2D_ENGINE_CPP.md`

5. Where to go next

If you want to...	Read
Build your first ROM in ten minutes	HELLO_WORLD.md
Understand the whole build, layer by layer	PROGRAMMERS_MANUAL.md
Look up an engine call	API_2D_ENGINE_C.md / API_2D_ENGINE_CPP.md
Look up a bare-metal SDK call	../SDK_API_GUIDE.md
Convert artwork	ARTBOX_PIPELINE.md
Make sound	SOUND_DRIVER.md, SOUND_STUDIO_GUIDE.md
Start a new game	ADDING_A_GAME.md
See what each shipped game demonstrates	GAMES.md, DEMO_CHAPTERS.md
Stay inside the frame budget	performance_rules.md
Look up a helper script	TOOLS.md
Debug on real hardware timing	GDB_GUIDE.md

6. Licence and intended use

MIT. The SDK is intended for lawful homebrew development, technical research, education, preservation-oriented study, and authorised consulting work. It must not be used for unauthorised copying, distribution, reproduction, sale, conversion, or operation of copyrighted commercial software.

PART 3

Programmer's Manual

The build system, the project layout, the game contract, the engine tick, the asset pipelines, and the frame budget.

`docs/PROGRAMMERS_MANUAL.md`

Eagle Software · Neo Geo SDK v1.7.0

The working manual for building Neo Geo software with this SDK: the build system, the project layout, the game contract, the engine tick, the asset pipelines, and the rules that keep a frame inside 16.7 ms.

Read INTRODUCTION.md first if you have not — it covers the hardware this manual assumes you know. For a first ROM, follow HELLO_WORLD.md. For call-by-call detail, use API_2D_ENGINE_C.md, API_2D_ENGINE_CPP.md and ../SDK_API_GUIDE.md.

Part I — The build

1. Two makefiles, one build

Host	Command
Linux, WSL	make ...
Windows (native)	make -f MakefileWin32.mak ...

They implement the same targets and produce byte-identical ROMs. Everything in this manual applies to both; only the invocation differs.

2. Selecting a game

Every game directory carries a game.cfg naming itself, and the repo root carries one naming whichever game you are working on. The build cross-checks GAME= against the cfg and stops if they disagree — which is what stops you from silently building the demo's art into another game's ROM.

```
make GAME=skylance GAME_CFG_FILE=games/skylance/game.cfg all
```

Setting CURRENT_GAME in the root game.cfg lets you drop both arguments for the game you work on most.

3. The five stages

make all runs them in order. Each is also a target on its own.

Stage	Target	Produces
1	art	Sprite C-ROMs (<id>-c1.c1, <id>-c2.c2), screens.c, sprite_meta.h, palettes
2	sfix	The S1 FIX ROM (<id>-s1.s1), 128 KB
3	sound	ADPCM samples, V-ROM, and the M1 Z80 driver ROM
4	p1	The 68000 program ROM (<id>-p1.p1), 512 KB
5	*(in p1)*	hash_eagle/<game>/neogeo.xml — the MAME hash set

In day-to-day work you will run p1 almost exclusively; it takes seconds on a warm tree. Re-run art when you change a PNG, sound when you change audio, and all before you ship.

Full target list

```

make all                art + sfix + sound + p1
make p1                 68000 program ROM only
make aes / mvs          force the AES or MVS platform variant
make art                rebuild sprite C-ROMs from artbox
make art-crt            same, through the CRT-preprocessing pipeline
make sfix / srom        rebuild the S1 FIX ROM
make sound              samples + vrom + fmpatches + fm + mml + ssgconfig + ssg + m1rom
make vrom               rebuild the V-ROM from ADPCM assets
make m1rom              the shipping ASM Z80 driver build
make m1rom-c            the experimental C driver build
make compare-driver     build both and diff the ROM images
make fm / fmpatches / mml / ssg / ssgconfig / samples
make dist               package dist/roms/<game>.zip
make test               run in MAME (MVS, sp-s2.spl BIOS)
make test-aes           run in MAME (AES, unibios22)
make test BIOS=unibios40
make unit-tests         host-side sprite renderer tests (no emulator)
make check              unit tests plus a complete ROM set
make debug              run MAME with the debugger attached
make debug-build        P1 with debug symbols and a linker map
make debug-artifacts    size, symbols, readelf, map and disassembly dumps
make gdb / gdb-trace / gdb-remote
make bios-list          list the BIOS images MAME can see
make games-list         list the games in the tree
make menu / help        the interactive launcher
make clean / sound-clean / art-clean / clean-all

```

4. Build behaviour worth knowing

- ROMs are written per game: roms/demo/777-p1.p1, roms/skylance/779-p1.p1.
- hash_eagle/<game>/neogeo.xml is regenerated on every p1 with correct CRC/SHA1 and loadflag="load16_word_swap" on the P-ROM. MAME will not load the cart without it.
- P1 generation crops to the full 512 KB window (0x080000) *before* the byte swap and padding. This is the required format for both MAME and hardware — a ROM built without the crop boots to garbage.
- make samples uses the bundled Python WAV converter by default. Set SOX=/path/to/sox only if you specifically want the SoX path.
- make dist puts the ROM files at the archive root, no subfolder.
- make m1rom-asm is the authoritative M1 path; the C driver is for comparison.

5. Debugging

```

make debug-build        # P1 with symbols + linker map
make debug-artifacts    # size / nm / readelf / map / disassembly into dump/
make gdb                # GDB on out/game
make gdb-remote GDB_REMOTE=localhost:23946
make gdb-trace          # scripted batch trace into dump/gdb_trace.txt
make debug              # MAME with the debugger attached

```

Symbol addresses come from m68k-unknown-elf-nm -n out/game, which is how you find a variable to watch from a MAME Lua script. MAME's -autoboot_script plus install_write_tap is the practical way to trace a value across frames — keep the tap handle in a global or it is collected and silently stops firing.

Full workflow: GDB_GUIDE.md.

Part II — Writing the game

6. The contract

A Neo Geo cartridge boots into the BIOS, not into `main()`. The BIOS calls back into the cart for power-on, eye-catcher, game and title; `user.c` provides those hooks and you rarely touch it.

What you write is:

```
void game_boot(void);    /* once, after the BIOS hands over */
void game_frame(void);   /* once per frame, forever */
```

`game_frame()` returns every frame, so anything that must persist across frames is `static` or file-scope. A plain `local` is gone before the next call — the most common first bug on this platform.

7. Project layout

```
games/mygame/
├─ game.mk          GAME_NAME, GAME_ID, GAME_SCENES, GAME_EXTRA_INCLUDES
├─ game.cfg         CURRENT_GAME=mygame
├─ main.c           game_boot() / game_frame(), and the generated screen table
├─ user.c           BIOS hooks – interrupt vectors, USER dispatch
├─ eyecatcher.c     generated by the artbox
├─ neogeo.ld        linker script
├─ neogeo_mvs.c     MVS platform stub
├─ neogeo_aes.c     AES platform stub
├─ scenes/          your code, listed in GAME_SCENES
├─ artbox/in/       source PNGs
└─ sound/           MML, FM, SSG and WAV sources
```

`game.mk`:

```
GAME_NAME = My Game
GAME_ID   = 780
GAME_SCENES = title stage boss
GAME_EXTRA_INCLUDES = -Igames/other/artbox # optional
```

`GAME_ID` becomes the ROM filename prefix and must be unique in the tree. In use: 555 tutorial, 772 helloworld, 775 neogeogame, 777 demo, 778 demo_plus, 779 skylance.

`GAME_SCENES` is an explicit list, not a wildcard, so helper files that are `#included` elsewhere do not get compiled twice.

Details: `ADDING_A_GAME.md`.

8. Driving the engine

The minimum:

```
#include "sdk/2d_engine/ng_engine.h"

void game_boot(void) { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

`ng_game_engine_frame()` runs a fixed order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — then flushes the render queue inside `VBlank`. You attach your game at five points:

```
ng_game_engine_set_hooks(before_logic,
                        collision_logic,
                        after_events,
                        before_draw,
                        after_draw);
```

Use the hooks rather than reimplementing the tick. Ordering inside the tick exists for reasons that are not obvious from outside: physics resolves after character movement but before the camera reads positions, and the render queue flush must be the last thing that happens.

9. Characters

ng_chars is a fixed pool of 64. Add, bind art, set a body, and the engine moves, animates, culls and draws them.

```
NGCharacter *c = chars_add(KIND_PLAYER, x, y);
ng_char_bind_asset(c, &asset_view);
ng_char_set_body(c, 0, 0, 24, 40);
ng_char_set_speed_fp(c, vx_fp, vy_fp);
ng_char_set_priority(c, NG_RENDER_BAND_PLAYER, 0);
```

Two traps that have cost real debugging time:

****ng_chars_count()** is not an array bound. ****** It returns the number of *active* objects. Once a slot frees, the active count is lower than the highest occupied index, and a loop bounded by it silently skips the tail. Iterate NG_MAX_CHARS and skip inactive slots:

```
for (i = 0; i < NG_MAX_CHARS; i++) {
    NGCharacter *c = chars_at(i);
    if (!c || !c->active) continue;
    ...
}
```

****Remove with ng_chars_remove().**** Setting a "destroy pending" life state is not acted on by the engine — the object stays active, the pool fills, and new spawns start failing. This presents as "the player can only fire six shots, then never again".

For collision over two pools, gather index lists first and then test list against list. A naive double loop over NG_MAX_CHARS is 64 × 64 rectangle tests per frame.

10. Sprites, slots, and priority

HIGHER slot number is drawn IN FRONT. This is the observed hardware direction. Earlier revisions of the engine headers claimed the opposite, and the result was backgrounds parked at "behind" slots drawing over characters.

Slots	Use
1-32	Background layers, parallax — the true back
33-95	Title and one-off generated screens
96-223	Characters and NPCs, assigned by ng_chars
224-255	Effects
256-287	Particles
288-379	Foreground / temporary effects

NG_SPR_BG0_FIRST and NG_SPR_BG1_FIRST predate the correction; the names read as if they were about layer order and they are not. Prefer explicit slot numbers in new code.

Other hardware facts that shape your code:

- **SCB2 only shrinks.** There is no stretch. Author art at its largest intended size and scale down. A "30 %" sprite is 0x4C-ish in the shrink field, not a multiplier you can exceed.
- ****SCB3's Y field is 496 - y, nine bits.**** So y and y + 512 are the same position, and negative Y wraps correctly — you do not need to clamp a sprite moving off the top.
- **381 sprites, 96 per scanline.** The per-line limit is what actually bites: a wide row of enemies at the same Y will drop sprites long before the total budget is reached.
- **A page is 16 × 16 tiles.** A scrolling background must be exactly one page in the scroll axis and seamless at the join, because you cannot stretch it to fit.

11. The FIX layer

40 × 32 cells of 8 × 8 pixels, of which **28 rows are visible**. Visible row y is map row y + 2; the ng_fix_* helpers apply that for you.

The FIX map word is (palette << 12) | tile — **four bits of palette**, so banks 0–15 only. A game with more than 16 FIX palettes has to fold duplicates; sky_fix_palettes_init() in Sky Lance is a worked example of doing that at boot.

The BIOS SFIX font's space glyph is opaque. Printing " " punches a solid colour-index-2 plate through whatever is behind it. To leave a cell genuinely empty use ng_fix_blank_cell(); tile 0xFF is the transparent one.

The layer is always on top of every sprite. That is why it is the right place for a HUD and the wrong place for anything that should pass behind a character.

12. Palettes and the backdrop

Palette RAM is \$400000–\$401FFF. A colour word is not RGB444: bit 15 is the dark bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top four bits of R, G and B — **5 bits per channel, 32 levels, in steps of 8**.

****The backdrop register is the last word of palette RAM, \$401FFE.**** \$402000 is a mirror that silently does nothing; writing there is a frequent cause of "my backdrop colour is ignored".

ng_palette_fx runs fades, flashes, pulses and colour cycles on a bank, all routed through the render queue so an effect started in game logic still lands inside VBlank.

13. Input

```
ng_joystick_update();                /* once per frame, first */
if (ng_joy_pressed() & BUTTON_A) { ... } /* edge */
if (ng_joystick_state()->raw & JOY_LEFT) { ... }
```

poll_joystick() gives you the raw current state; acting on it directly fires once per frame for as long as the button is held. ng_joy_repeat() gives menu-style auto-repeat, ng_joy_held_frames() gives charge timing, and ng_joy_special_qcf() / ng_joy_special_dp() recognise motion inputs.

Part III — Assets

14. Art

Drop PNGs into `games/<game>/artbox/in/<category>/` and run `make art`. The pipeline quantises to the Neo Geo colour lattice, builds palettes, packs tiles into C-ROMs, and generates `screens.c` (one `showScreenN()` per asset) and `sprite_meta.h` (geometry per asset).

Categories are processed in a fixed order — backgrounds, characters, effects, eyecatcher, npc, screens, titles — and any other directory is appended alphabetically. **Asset ids are the 1-based position in that order**, so inserting a file in the middle renumbers everything after it. The shipped games pad filenames to control ordering; do the same.

characters and npcs share one derived 15-colour master palette per group. Other categories get per-asset palettes. That is why a sprite that looks wrong after being moved between categories is usually a palette problem, not a conversion problem.

Full detail, including the quantiser and the Artbox Studio GUI: `ARTBOX_PIPELINE.md`.

15. Sound

`soundInit()` once at boot, then `playMusic()`, `playSFX()`, `playSFXB()`, `playFMTrack()`, `playSSGTrack()` with ids from `sdk/sound_ids.h`.

Every call posts one byte to a second CPU. Nothing is synchronous, and multi-byte commands (a prefix plus a parameter) must be spaced by a `waitVbl()` or the second byte can be dropped.

Full detail: `SOUND_DRIVER.md`. Composing tools: `SOUND_STUDIO_GUIDE.md`.

Part IV — Staying inside the frame

16. The budget

Resource	Limit
CPU	~12 MHz 68000, ~200,000 cycles per frame
VBlank	~16 scanlines, ~1.1 ms — all VRAM writes must fit
Sprites	380 total, 96 per scanline
Palettes	16 entries per bank, 256 banks
FIX	40 × 32 cells, 28 rows visible, 16 palettes

17. The rules

****No malloc during gameplay.**** Every engine subsystem uses a fixed pool. Dynamic allocation fragments RAM and produces non-deterministic pauses.

Pool	Size	Constant
Characters	64	<code>NG_MAX_CHARS</code>
NPCs	32	<code>NG_MAX_NPCS</code>

Pool	Size	Constant
Particles	32	NG_PART_MAX_PARTICLES
Palette FX	8	NG_PALFX_MAX_SLOTS
Render queue	128	NG_RQ_MAX_CMDS

****No float or double.**** There is no FPU; every float costs hundreds of cycles in software emulation. Use 16.16 fixed-point from `ng_fixed.h`.

No division in the frame loop. DIVS/DIVU cost 76–158 cycles each. Use `ng_sin_tab[]`, `ng_cos_tab[]`, `ng_shrink_tab[]`, right shifts, and fixed-point multiply.

Write only what changed. Sprite groups carry dirty flags; the FIX layer has a dirty-cell cache. Rewriting an unchanged sprite chain every frame is the fastest way to blow the VBlank budget.

Queue every VRAM write. Post to the render queue from game logic and let `ng_render_queue_flush()` drain it in VBlank. Writing VRAM outside VBlank produces tearing and corrupt tiles.

Budget sprites per line, not per screen. 96 is the number that matters.

More: `performance_rules.md`.

Appendix A — Register quick reference

Address	R/W	Register
\$3C0000	W	VRAM address (word index)
\$3C0000	R	VRAM read data
\$3C0002	W	VRAM write data
\$3C0004	R/W	Auto-increment
\$3C0006	W	Mode — timer / IRQ / auto-animation
\$3C0008 / \$3C000A	W	Timer high / low
\$3C000C	W	IRQ acknowledge (bit 1 timer, bit 2 VBL)
\$320000	R/W	Sound command port to / from the Z80
\$400000–\$401FFF	R/W	Palette RAM
\$401FFE	W	Backdrop colour
\$3A000F / \$3A001F	W	Palette bank select
\$3A0011	W	Shadow / display dim
\$300001	W	Watchdog kick

Word-aligned access only. Wait $\geq 1.4 \mu\text{s}$ after touching the VRAM address register before reading.

Appendix B — Interrupts

IRQ	Source	Vector	Acknowledge
1	V-Blank	\$64	MOVE.W #4, \$3C000C

IRQ	Source	Vector	Acknowledge
2	LSPC timer	\$68	MOVE.W #2, \$3C000C

Kick the watchdog (MOVE.B D0, \$300001) once per frame in the VBlank handler or the board resets.

Appendix C — Common symptoms

Symptom	Cause
Sprite invisible behind the background	Slot priority — higher is in front
Sprite bigger than intended, cannot shrink further	SCB2 only reduces; re-author the art
Backdrop colour ignored	Wrote \$402000 instead of \$401FFE
Text draws solid blocks	BIOS space glyph is opaque; use <code>ng_fix_blank_cell()</code>
FIX palette wraps onto text colours	Only 4 palette bits — fold duplicates below 16 banks
Pool fills and spawns stop	Objects marked destroyed but never <code>ng_chars_remove()</code> d
Loop skips the last objects	<code>ng_chars_count()</code> used as an array bound
Second byte of a sound command lost	Prefix and parameter not spaced by <code>waitVbl()</code>
Sprites dropped along one row	96-per-scanline limit
Green self-test screen on boot	BIOS self-test, not your ROM — try BIOS=unibios40
Tearing or corrupt tiles	VRAM written outside VBlank instead of queued

PART 4

Toolchain 3.0

Code generation, language standards, freestanding linking, and the verified compiler/debugger layout.

`docs/TOOLCHAIN.md`

The `x-tools-v3` bundle is the compiler, binutils, C and C++ libraries and debugger the SDK builds with. It targets the Motorola 68000 in the Neo Geo and runs on any x86-64 Linux: every host program in it is statically linked and needs no shared library, no interpreter and no package from the machine it runs on.

Contents

Component	Version	Notes
GCC	16.2.0	C and C++ front ends
GNU binutils	2.47	as, ld, objcopy, objdump, nm, size, readelf, strip, gprof, lto-dump
GNU gdb	17.2	cross debugger with TUI, XML target descriptions, internal readline
newlib	4.6.0	C library, single-threaded, nano malloc, C99 and long long formatted I/O, floating-point formatted I/O
libstdc++	16.2.0	full C++ standard library, quiet terminate (no demangler or stdio pulled in)
libgcc	16.2.0	software integer division, multiplication and floating point for the 68000

Target triplet: `m68k-unknown-elf`.

Bundle	Host	Archive	Installed
<code>x-tools-v3</code>	x86-64 Linux, static binaries	<code>x-tools-v3.tar.xz</code> , 58 MB	280 MB
<code>x-tools-v3-win</code>	64-bit Windows, .exe files importing only system DLLs	<code>x-tools-v3-win.zip</code> , 106 MB	250 MB

Both bundles are built from the same sources with the same configuration and produce byte-identical ROMs from the same input.

Code generation

- Default CPU `-mcpu=68000`, so a build that names no CPU still produces code the Neo Geo can run. `-m68020` and later are accepted for other boards; the libraries link against those too, being 68000 code.
- Single library set built for the 68000 with `-Os`. No multilib.
- Link-time optimisation: `-flto` is available for compile and link. The linker has no plugin support (a consequence of static host binaries), so archives of LTO objects should be built with `-ffat-lto-objects`; objects named on the link line are optimised across translation units.
- Graphite loop optimisations are available.
- No TLS, no shared libraries, no thread model, no SSP, no quadmath.

Language standards

Language	Default	Supported through	Verified here
C	gnu23	C23 (<code>_BitInt</code> excepted: not available on this target)	-std=gnu99, -std=c23
C++	gnu+17	C++26	-std=c++14, -std=c++23, -std=c++26 in freestanding mode with -fno-exceptions -fno-rtti

The C++ headers (`<array>`, `<span>`, `<bit>`, `<algorithm>`, `<type_traits>`, `<ranges>`, `<utility>` and the rest) are usable with `-ffreestanding` and no runtime support beyond what the SDK already provides.

Linking without an operating system

The SDK links with `-nostdlib` and supplies its own start-up code, so it uses nothing below except `libgcc`. For programs that want the C library: `-lc` provides `libc.a` (with `libg.a` and `libm.a` beside it); `newlib`'s own system-call stubs are not built in, so a program that reaches the OS layer (`write`, `sbrk`, `exit`...) defines those functions itself, links `-lnosys`, or passes `--specs=nosys.specs`. `crt0.o` and the `bcc`, `idp` and `idpgdb` board scripts from `libgloss` are installed for reference.

Layout

```
x-tools-v3/
  m68k-unknown-elf/
    bin/                          m68k-unknown-elf-gcc, -g++, -ld, -gdb, ...
    lib/gcc/m68k-unknown-elf/16.2.0/
      libgcc.a, crtbegin.o, crtend.o, headers
    m68k-unknown-elf/include/     C library headers, c++/16.2.0/
    m68k-unknown-elf/lib/         libc.a, libg.a, libm.a, libstdc++.a,
                                  libsupc++.a, libnosys.a, ldscripts/
    share/gdb/                   gdb data directory (relocatable)
    share/licenses/              licences of every component
```

The bundle is relocatable: unpack it anywhere and point `SDKHOME` (or `XT00LS_ROOT`) at its parent.

Use with the SDK

Makefile looks for the toolchain in this order and takes the first found:

1. `$(SDKHOME)/x-tools-v3`
2. `$(SDKHOME)/x-tools-v2`
3. `$(SDKHOME)/x-tools`

or set `XT00LS_ROOT=<directory containing m68k-unknown-elf/>` on the make command line.

MakefileWin32.mak looks for `x-tools-v3-win` first, then `x-tools-v2-win`, then `M68K_ELF_ROOT`. Its link step now hands the linker forward-slash object paths, because the linker script places sections by object file name (`out/ng_*.o`) and backslash spellings never matched those patterns; with that, a Windows build of any game is byte-identical to the Linux build.

Verified

- All six games in the tree build with no warnings: demo, demo_plus (the C++ engine, -std=c++14), skylance, neogeogame, helloworld, tutorial. The demo's C build is 369,785 bytes of text against 377,973 from the previous toolchain at the same -O0.
- The demo ROM set built with it passes the full 26-chapter capture tour with VRAM and palette RAM checked against the manifest, the controller run, and the live audio check.
- -flto links a multi-file program with cross-unit inlining.
- gdb loads the demo's ELF, resolves symbols and disassembles 68000 code.
- The Windows bundle builds all six games through MakefileWin32.mak with no warnings; every program ROM matches the Linux build byte for byte, and every .exe imports only KERNEL32, msvcrt, ADVAPI32, USER32, WS2_32 and bcrypt.

PART 5

Dependencies

Core Python build packages, optional SoX, and the desktop GUI packages, kept separate.

`docs/DEPENDENCIES.md`

v1.7.0 — extra dependencies - The HD artbox alt scripts (artbox/img2neo_hd.py, artbox/fixtiles_hd.py) require the same packages as the main pipeline (numpy, pypng, Pillow). No new dependencies, but they're slower because of the bilateral filter + CLAHE numpy passes. - The C++ engine (USE_2D_PLUS=1) is built by the same m68k-unknown-elf-g++ shipped with x-tools-v3. No extra toolchain.

This document lists every tool and library required to build the NeoGeoSDK on Linux, WSL, and Windows.

Linux / WSL

System packages

Install with apt-get (Ubuntu / Debian):

```
sudo apt-get update
sudo apt-get install \
    git \
    mame \
    srecord \
    cmake \
    build-essential \
    python3 \
    python3-pip \
    python3-numpy \
    python3-pil \
    sqlite3
```

sox is optional. The build falls back to the bundled Python converter when it is not present:

```
sudo apt-get install sox # optional
```

Python packages

```
python3 -m pip install --user pypng
```

Core modules used by the build scripts:

Module	Used by	Install via
PIL	artbox/img2neo.py, artbox/romtiles.py	python3-pil / pillow
numpy	artbox/img2neo.py	python3-numpy
pypng	artbox/romdbimgimport.py	pip install pypng
sqlite3	artbox/neorom.db tooling	stdlib (always available)
wave	sound sample tools	stdlib
struct	sound sample tools	stdlib
argparse	all CLI tools	stdlib
pathlib	all CLI tools	stdlib
json	artbox/asset_rules.py, manifest tools	stdlib

WLA-DX (Z80 assembler)

Build from source:

```
git clone https://github.com/vhelin/wla-dx
cd wla-dx
cmake -S . -B build
cmake --build build -j
sudo cp build/binaries/wla-z80 build/binaries/wlalink /usr/local/bin/
```

Verify:

```
wla-z80 --version
wlalink --version
```

m68k-unknown-elf toolchain (Linux)

Download the pre-built x-tools-v3.tar.xz from the release page and place it next to the repository:

```
mkdir -p $HOME/neogeo
cd $HOME/neogeo
tar -xf x-tools-v3.tar.xz
```

What is in it - GCC 16.2, binutils 2.47, gdb 17.2, newlib 4.6, libstdc++, all as static x86-64 Linux binaries - is specified in TOOLCHAIN.md.

Expected layout:

```
$HOME/neogeo/
neogeosdk/
x-tools-v3/
  m68k-unknown-elf/
    bin/
      m68k-unknown-elf-gcc
      m68k-unknown-elf-ld
      m68k-unknown-elf-objcopy
      m68k-unknown-elf-objdump
      m68k-unknown-elf-gdb
      m68k-unknown-elf-nm
      m68k-unknown-elf-readelf
      m68k-unknown-elf-addr2line
      m68k-unknown-elf-size
```

Verify:

```
export SDKHOME=$HOME/neogeo
$SDKHOME/x-tools-v3/m68k-unknown-elf/bin/m68k-unknown-elf-gcc --version
```

The Linux makefile fallback order is:

1. `\$\$SDKHOME/x-tools-v3`
2. `\$\$SDKHOME/x-tools-v2`
3. `\$\$SDKHOME/x-tools` (legacy)

MAME

```
sudo apt-get install mame
```

Or build from source: <https://www.mamedev.org/>

Windows

m68k toolchain (Windows)

Unpack x-tools-v3-win.zip from the release page next to the repository. It holds the same toolchain as the Linux bundle (see TOOLCHAIN.md) built for 64-bit Windows, with no DLL dependencies. The Win32 makefile (MakefileWin32.mak) auto-detects it first:

```
<sdk root>\x-tools-v3-win\m68k-unknown-elf\bin\m68k-unknown-elf-gcc.exe
```

Then the previous bundle:

```
<sdk root>\x-tools-v2-win\m68k-unknown-elf\bin\m68k-unknown-elf-gcc.exe
<sdk root>\x-tools-v2-win\m68k-elf\bin\m68k-elf-gcc.exe
```

If none exists, it falls back to a SysGCC-style M68K_ELF_ROOT:

```
C:\SysGCC\m68k-elf\bin\m68k-elf-gcc.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-ld.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-objcopy.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-objdump.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-gdb.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-nm.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-readelf.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-addr2line.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-size.exe
```

If your toolchain root differs, pass it on the command line:

```
make -f MakefileWin32.mak M68K_ELF_ROOT=D:\toolchains\m68k-elf all
```

Python

Install Python 3 from <https://www.python.org/> or the Microsoft Store. Make sure the py launcher is available:

```
py --version
```

Install required packages:

```
py -0p
py -m pip --version
py -m pip install --upgrade pip
py -m pip install numpy pillow pypng
```

The same module table applies as on Linux (see above).

Important: use `py -m pip` instead of `pip` or `pip3`. The Windows build scripts call `py`, so using the same launcher avoids interpreter/package mismatch errors.

Quick verification:

```
cd artbox
py -c "import sys; print(sys.executable); import PIL, numpy, png, img2neo; print('OK!')"
```

GNU Make

Use the `make.exe` that ships with your toolchain bundle, or install from:

- <https://gnuwin32.sourceforge.net/packages/make.htm>
- <https://www.msys2.org/>

Verify:

```
make --version
```

WLA-DX (Windows)

Build from source with CMake and a MinGW or MSVC toolchain, or download a pre-built binary from:

- <https://github.com/vhelin/wla-dx/releases>

Place `wla-z80.exe` and `wlalink.exe` in a directory on PATH.

srec_cat (Windows)

The Windows build includes a pre-built `win\srec_cat.exe` in the repository. No separate installation is needed.

xxd (Windows)

A pre-built `win\xxd.exe` is also included.

MAME (Windows)

Download from <https://www.mamedev.org/> and place `mame.exe` on PATH.

sox (Windows)

Optional. If not present, the bundled Python WAV converter is used automatically. If you want SoX:

- <https://sourceforge.net/projects/sox/files/sox/>

Place `sox.exe` on PATH and pass it to make:

```
make -f MakefileWin32.mak SOX=sox samples
```

Bundled tools (no install needed)

File	Purpose
<code>win/srec_cat.exe</code>	ROM byte manipulation (Windows)
<code>win/xxd.exe</code>	ROM hex dump (Windows)
<code>sound/tools/wav_to_raw_pcm.py</code>	Python WAV converter fallback
<code>sound/tools/*.py</code>	ADPCM encoding, MML compile, FM tools
<code>artbox/*.py</code>	Graphics pipeline tools

Version summary

Dependency	Minimum version tested
GCC (m68k cross)	12.x

Dependency	Minimum version tested
WLA-DX	10.x
Python	3.9
Pillow	9.x
numpy	1.21
MAME	0.250
srec_cat	1.64
GNU Make	4.x

PART 6

Tutorial: Hello World

From a clean checkout to a running ROM, then into a game of your own.

`docs/HELLO_WORLD.md`

Eagle Software · Neo Geo SDK v1.7.0

This tutorial takes you from a clean checkout to a running ROM that prints text on screen and plays a sample when the player presses **A**. It uses the games/helloworld project that ships with the SDK, then shows you how to copy it into a game of your own.

Budget: about ten minutes, most of it the first art build.

0. Before you start

You need the toolchain installed and SDKHOME set. See DEPENDENCIES.md and the installation sections of ../README.md, or run the installer for your platform from install/.

Quick check:

```
m68k-unknown-elf-gcc --version      # 68000 cross-compiler
wla-z80 --version                   # Z80 assembler for the sound driver
python3 --version                   # 3.10+, with Pillow and NumPy
make --version
```

1. Build and run it

```
# Linux / WSL
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test

# Windows
make -f MakefileWin32.mak GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make -f MakefileWin32.mak GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test
```

all runs five stages in order — **art**, **sfix**, **sound**, **p1**, and the hash-set generation MAME needs. test launches MAME on the result.

Every game in the tree carries its own game.cfg naming itself, and the repo-root game.cfg names whatever game you are working on. The GAME_CFG_FILE= argument is what lets you build a different game without editing the root file. If the two disagree, the build stops and tells you.

You should see Hello World! and Press A for sound on a black screen. Press **A**: sample 0 plays.

2. What actually ran

A Neo Geo cartridge boots into the BIOS, not into your main(). The BIOS calls back into the cartridge for four things — power-on, eye-catcher, game, and title — and every game in this SDK provides those hooks in user.c. You almost never edit that file.

What you *do* write is two functions in main.c:

```
void game_boot(void);    /* called once, after the BIOS hands over */
void game_frame(void);   /* called once per frame, forever */
```

That is the whole contract for a minimal game.

[games/helloworld/main.c](#)

```

#include <stdint.h>
#include "sdk/2d_engine/ng_fix.h"
#include "sdk/neogeo.h"
#include "sdk/macro.h"    /* button and joystick bit masks */

void game_boot(void) {
    ng_fix_init();          /* set up the FIX text layer */
    ng_fix_clear();         /* wipe whatever the BIOS left */
    ng_fix_puts(3, 15, "Hello World!", 0); /* x, y in 8x8 cells; 0 = palette */
    ng_fix_puts(3, 17, "Press A for sound", 0);
    soundInit();           /* boot the Z80 + YM2610 */
}

static uint16_t prev_joy = 0; /* file scope: survives frames */

void game_frame(void) {
    uint16_t joy = poll_joystick(); /* JOY_* / BUTTON_* bitfield */

    if (!(prev_joy & BUTTON_A) && (joy & BUTTON_A)) {
        playSFX(0); /* rising edge only */
    }
    prev_joy = joy;
}

```

Three things in that file are worth internalising, because they apply to every Neo Geo program you will write:

The FIX layer is cell-addressed, not pixel-addressed. `ng_fix_puts(3, 15, ...)` means column 3, row 15 of a 40×32 grid of 8×8 tiles. Only 28 rows are visible, and the SDK's FIX helpers already account for the two-row offset between map rows and screen rows.

Input is polled, not queued. `poll_joystick()` returns the *current* state. If you act on the raw value you will fire once per frame for as long as the button is held, which is almost never what you want — hence the `prev_joy` edge test. The 2D engine's `ng_joystick` module does this bookkeeping for you (`ng_joy_pressed()`), and that is what a real game should use.

Sound is a one-byte command to another CPU. `playSFX(0)` writes a sound code to $\$320000$. The Z80 picks it up on its own port and dispatches it. Nothing about that call is synchronous; do not expect a return value or a completion signal.

Why `prev_joy` is at file scope

A static inside `game_frame()` would work too. What must *not* happen is a plain local — `game_frame()` returns every frame, so a stack local is gone before the next call. This is the single most common first bug.

3. Change something

Edit the text, then rebuild only the 68000 side:

```

make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg p1
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test

```

`p1` skips art and sound and just recompiles and relinks the program ROM. On a warm tree it takes a couple of seconds, and it is the target you will use ninety percent of the time.

Some things to try:

Change	What you learn
<code>ng_fix_puts(3, 15, "Hello World!", 1)</code>	Palette 1 instead of 0. FIX palettes are 4 bits — banks 0-15 only.

Change	What you learn
Add <code>setBACKDROP(0x8001);</code> in <code>game_boot()</code>	The backdrop register is the whole background layer.
Print joy with <code>ng_fix_put_u16(3, 19, joy, 0, 0)</code>	Watch the input bitfield live.
<code>playSFX(1)</code> on <code>BUTTON_B</code>	Sample ids come from <code>sound/samples/in_wav_a/</code> .

4. Add a sprite

`ng_fix_*` draws text. Anything that moves is a sprite, and sprites come from the artbox.

1. Drop a PNG into `games/helloworld/artbox/in/characters/`.
2. Re-run the art stage: `sh make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg art`
3. The pipeline writes `games/helloworld/artbox/sprite_meta.h` with one entry per asset, plus the C-ROM data the hardware reads tiles from.
4. Include `sprite_meta.h` and bind the asset to a character with `ng_char_bind_asset()`, or draw it directly with the `sprite-group` API.

The asset id is its 1-based index in the generated manifest, and the order is decided by directory (backgrounds, characters, effects, eyecatcher, npc, screens, titles, then any other directory alphabetically) and then by filename. Adding a file in the middle of a category renumbers everything after it, so the shipped games pad names to control ordering. `ARTBOX_PIPELINE.md` covers this properly.

5. Turn it into your own game

`games/neogeogame` (id 775) is the same skeleton with nothing in it, ready to be renamed. To start clean:

```
cp -r games/helloworld games/mygame
```

Then edit **four** things:

File	Change
<code>games/mygame/game.mk</code>	<code>GAME_NAME</code> , and a <code>GAME_ID</code> no other game uses
<code>games/mygame/game.cfg</code>	<code>CURRENT_GAME=mygame</code>
<code>games/mygame/main.c</code>	your <code>game_boot()</code> / <code>game_frame()</code>
<code>games/mygame/artbox/in/</code>	your PNGs

`GAME_ID` becomes the ROM filename prefix (777-p1.p1, 779-c1.c1, ...), so it has to be unique across the tree or two games will overwrite each other's output. Ids in use: 555 tutorial, 772 helloworld, 775 neogeogame, 777 demo, 778 demo_plus, 779 skylance.

If your game needs more than `main.c`, add scene files under `games/mygame/scenes/` and list them in `GAME_SCENES` in `game.mk` — the build compiles exactly that list, so helper files that are `#included` elsewhere do not get compiled twice.

Full instructions: `ADDING_A_GAME.md`.

6. Stepping up to the engine

helloworld talks to the hardware directly. games/tutorial (id 555) is the same size but drives the 2D engine instead:

```
#include "sdk/2d_engine/ng_engine.h"

void game_boot(void) { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

ng_game_engine_frame() runs the whole engine tick in order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — and flushes the render queue during VBlank. You hook into it at five points with ng_game_engine_set_hooks(): before logic, collision, after events, before draw, after draw.

From there:

- GAME_ENGINE_LAYER.md — how the engine tick fits together
- API_2D_ENGINE_C.md — every C call, by module
- API_2D_ENGINE_CPP.md — the C++ build
- GAMES.md — what each shipped game demonstrates, and games/skylance as a complete worked example

7. When it does not work

Symptom	Cause
ERROR: GAME mismatch	GAME= and CURRENT_GAME in the cfg disagree. Pass the right GAME_CFG_FILE=.
Green screen with test text on boot	The BIOS self-test, not your ROM. It is normal on some BIOS revisions; make test BIOS=unibios40 skips it.
Blank white or black screen	Usually a backdrop or palette that was never loaded. Check setBACKDROP() and that your palettes are uploaded before you draw.
Text draws as solid blocks	The BIOS SFIX font's space glyph is <i>opaque</i> colour index 2. To leave a cell truly empty use ng_fix_blank_cell(), not " ".
Sprite is invisible	Lower slot = in front. A background parked at a low slot covers everything. Also check the sprite was not left with SCB3 = 0x8000.
Sprite is the wrong size	SCB2 only shrinks. Author art at full size and scale down.
Link error about ng_screen_table	Only if you removed the weak fallback in ng_bg.c; a game with no generated screens links against it.

The demo ROM prints a chapter number in the top-right corner of every scene precisely so problems can be reported by number — see DEMO_CHAPTERS.md.

PART 7

Adding a New Game

Every step to add a new game directory to the multi-game tree.

`docs/ADDING_A_GAME.md`

This guide walks through every step needed to add a new game called mygame with ROM ID prefix 888. Substitute your own names throughout.

1. Create the game directory tree

```
games/
├── mygame/
│   ├── game.mk
│   ├── main.c
│   ├── user.c
│   ├── eyecatcher.c
│   ├── neogeo_mvs.c           (copy from games/demo/ and customise)
│   ├── neogeo_aes.c          (copy from games/demo/ and customise)
│   ├── artbox/
│   │   └── infix/            (FIX-layer tile imports, may be empty)
│   ├── scenes/
│   │   ├── mygame.c / .h     (game-specific scene files)
│   └── sound/                (optional – per-game sound assets)
│       ├── fm/
│       ├── mml/
│       ├── ssg/
│       └── samples/
```

The easiest starting point is to copy games/demo/ and rename files.

2. Write games/mygame/game.mk

```
GAME_NAME = My Game
GAME_ID   = 888
GAME_SCENES = mygame
```

GAME_SCENES is a space-separated list of .c filenames (without the .c) inside games/mygame/scenes/. Every file listed here is compiled and linked.

3. Implement the required entry points

main.c — called at boot after BIOS init:

```
#include "sdk/neogeo.h"
void game_main(void) { /* your attract loop / title / gameplay */ }
```

user.c — BIOS USER hook (called every VBlank):

```
#include "sdk/neogeo.h"
void user_frame(void) { /* optional per-VBlank logic */ }
```

eyecatcher.c — MVS insert-coin screen:

```
#include "sdk/neogeo.h"
void eyecatcher(void) { /* eyecatcher attract loop */ }
```

See games/demo/ for working examples of each.

4. Build the game

```
# Linux
make GAME=mygame all

# Windows
make -f MakefileWin32.mak GAME=mygame all
```

GAME_ID is read from games/mygame/game.mk automatically.

Individual steps are available if you only changed one subsystem:

Target	What it rebuilds
p1	68k code → P1 ROM
art	Sprites → C1/C2 ROMs
sfix	Fix-layer font → S1 ROM
sound	Z80 driver + samples → M1/V1 ROMs
all	art + sfix + sound + p1

5. Per-game ROM folder

All six ROMs land in roms/mygame/:

```
roms/
├── mygame/
│   ├── 888-p1.p1
│   ├── 888-m1.m1
│   ├── 888-s1.s1
│   ├── 888-v1.v1
│   ├── 888-c1.c1
│   └── 888-c2.c2
```

MAME is invoked with `-cart1 mygame` so it looks in roms/mygame/ automatically.

6. Per-game hash XML

gen_hash.py generates hash_eagle/mygame/neogeo.xml with an entry for mygame and ROM prefix 888. The HASHPATH passed to MAME is:

```
hash_eagle/mygame ; hash_eagle ; hash
```

MAME finds hash_eagle/mygame/neogeo.xml first (highest priority), so different games never stomp each other's checksums.

Run it manually if needed:

```
GAME=mygame GAME_ID=888 python3 hash_eagle/gen_hash.py
```

7. Test in MAME

```
# Linux
make GAME=mygame test

# Windows
make -f MakefileWin32.mak GAME=mygame test
```

Override the BIOS if needed:

```
make GAME=mygame test BIOS=unibios40
```

8. FIX layer text

Call `ngfix_init()` once at startup and `ngfix_clear()` before drawing any FIX text.

```
#include "sdk/ng_fix/ng_fix.h"

ngfix_init();
ngfix_clear();
ngfix_puts(2, 2, "MY GAME", 0);
```

9. Using the 2D engine

To use `ng_game_engine_init` / `ng_game_engine_frame` from `ng_game_interrupt.h`:

```
#include "sdk/2d_engine/ng_game_interrupt.h"

void game_main(void)
{
    ng_engine_init_hardware(0);      /* clear VRAM, set transparent tile */
    ng_game_engine_init();           /* init all subsystems */

    ng_game_engine_set_hooks(
        my_before_logic,
        my_collision_logic,
        my_after_events,
        my_before_draw,
        my_after_draw
    );

    while (1) {
        waitVbl();
        ng_game_engine_frame();      /* runs logic, draw, palette FX, flush */
    }
}
```

`ng_game_engine_init()` now wires up every subsystem automatically: render queue, particles, palette FX, feedback, and depth FX.

10. Linker script

Copy `games/demo/neogeo.ld` to `games/mygame/neogeo.ld`. Adjust section sizes if your game has more/fewer scenes or a larger BSS footprint.

The catch-all patterns at the end of `game_engine_bss` handle `.data` and `.bss` sections from engine objects regardless of path separator, which is required for Windows GNU ld compatibility.

11. Sound assets (optional)

If your game has its own music/samples, place them in `games/mygame/sound/`:

```
games/mygame/sound/
├── fm/patches.fm           (FM synth patches)
├── fm/*.mml                (FM music tracks)
├── mml/*.mml               (general MML)
├── ssg/config.ssg          (SSG channel config)
├── ssg/*.mml               (SSG tracks)
├── samples/
│   ├── in_wav_a/           (ADPCM-A source WAVs, 8kHz mono 16-bit)
│   └── in_wav_b/           (ADPCM-B source WAVs, 8kHz mono 16-bit)
```

Then build with:

```
make GAME=mygame sound
```

If a directory is missing the build silently skips it, so you only need the subdirectories for the sound features you actually use.

12. Distribution package

```
make GAME=mygame dist
```

This produces:

```
dist/
├── roms/mygame.zip          (all six ROMs)
├── hash_eagle/mygame/neogeo.xml
├── run_mygame.sh / .bat
└── run_mygame_debug.sh / .bat
```

Distribute `roms/mygame.zip` + `hash_eagle/mygame/neogeo.xml` + the launcher scripts. Players place `neogeo.zip` (BIOS) in the `roms/` folder and run the launcher.

PART 8

The Shipped Games

What each of the six reference games is, and which subsystem each one is the working example for.

`docs/GAMES.md`

Eagle Software · Neo Geo SDK v1.7.0

Six complete games live in `games/`. They are not samples in the "toy fragment" sense — every one of them builds to a real ROM that runs in MAME and on hardware, and between them they exercise every public subsystem in the SDK. They are the reference implementations: when a document says "this is how you do X", one of these games is doing X.

Game	Id	Engine	What it is
demo	777	C	The 26-chapter engine reel — every subsystem, in order
demo_plus	778	C++	The same engine through the C++ API, as a smoke test
skylance	779	C	Sky Lance — a complete vertical shooter
helloworld	772	—	FIX text and one sample. The tutorial target.
tutorial	555	C	The minimal engine loop, nothing else
neogeogame	775	C	An empty starting point to copy

Every game carries its own `game.cfg`, so all six build the same way:

```
make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg all
make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg test
```

The repo-root `game.cfg` names whichever game you are actively working on; passing `GAME_CFG_FILE=` lets you build any other without editing it.

demo — the engine reel (id 777)

A single linear flow of **26 chapters**, each demonstrating one subsystem, with the chapter number printed in the top-right corner of every scene so a problem can be reported by number. **A** advances, **C** restarts the current chapter.

Full chapter-by-chapter breakdown: `DEMO_CHAPTERS.md`.

The demo is the largest consumer of the SDK and therefore the best place to read working usage of: the FIX layer and INFIX photo pages, sprite screens, character movesets, gravity and solids, camera follow / dead-zone / shake, palette effects, the particle pool under stress, hitstop and shake feedback, depth projection, two-layer parallax, NPC patrol AI, live joystick handling with two-button specials, a scrolling world, sprite-scaling pseudo-3D, and a playable shooter.

`games/demo/scenes/demo_unified.c` is the reel itself; the other files under `scenes/` are older standalone scenes that the reel draws on.

demo_plus — the C++ engine (id 778)

The same engine, linked from `sdk/2d_engine_plus/` instead of `sdk/2d_engine/`:

```
make GAME=demo_plus GAME_CFG_FILE=games/demo_plus/game.cfg USE_2D_PLUS=1 all
```

Three sub-scenes — `TITLE`, `PARTICLES`, `MARQUEE` — driving the render queue, palette effects, and the particle pool through the C++ API. It shares the demo's artbox rather than duplicating the data, via the `GAME_EXTRA_INCLUDES` hook in its `game.mk`:

```
GAME_EXTRA_INCLUDES = -Igames/demo -Igames/demo/artbox
```

Its purpose is to prove the two engine builds stay ABI-compatible. If a change to the C engine is not mirrored in the C++ one, this is the ROM that fails to build.

skylance — Sky Lance (id 779)

A complete vertical arcade shooter: three pilots, seven stages, one named boss per stage, attract reel, pilot select, scoring, lives, energy, and a game-over/continue flow.

Pilots

Call sign	Name	Fire rate	Speed	Bolt
BLUE LANCE	ROOK	8	3	single — the baseline
GREEN WING	KIRA	11	3	twin — wider spread, slower rate
RED STRIKER	BLAZE	6	4	single — fast and rapid, has to get close

The three are differentiated the way the arcade originals do it: by feel, not by damage numbers.

Stages

#	Backdrop	Boss
1	Mountain	CRIMSON KEEP
2	Coast	IRON TIDE
3	Mountain	SOL CORE
4	Coast	NIGHT RAZOR
5	Mountain	ROTOR NEST
6	Coast	EARTH HAMMER
7	Mountain	SPIRE GOD

Each stage runs 5–7 squadrons of enemies from its own roster, then the boss.

Layout

File	What it holds
scenes/sky_draw.c	Asset binding, the scrolling backdrop, the FIX HUD, the frame pump
scenes/sky_stage.c	Enemy roster, stage table, wave director, boss AI
scenes/sky_sortie.c	Attract reel, pilot select, the flight loop, collisions, scoring
user.c	BIOS handshake, FIX palettes, MVS title
main.c	Generated showScreenN() art entry points — rewritten by make_art

Two hardware lessons worth reading the source for

The playfield is 256 px wide, not 320. The arena is the column the backdrop covers, centred on the screen, with 32 px of backdrop either side. That is a consequence of the backdrop being exactly one 16 × 16-tile page wide, not a style choice.

The backdrop had to be authored to tile. A page is 16 × 16 tiles and the sprite chip can only shrink, never stretch, so a vertically scrolling backdrop has to be exactly one page tall and seamless

at the join. Both backdrops were resized to 256 × 256 and cross-faded across the wrap.

Sky Lance also ships without its own sample set, so its audio path is vrom m1rom rather than the full sound target — see games/skylance/sound/README.md.

A slice of Sky Lance also appears as chapter 24 of the demo reel, so you can see the same code driven two ways.

helloworld (id 772)

The smallest useful ROM: ng_fix_init(), some text, soundInit(), and an edge-triggered playSFX(0) on button A. No engine, no scenes, no sprites.

This is the target of HELLO_WORLD.md, and the right thing to copy when you want to start from bare metal.

tutorial (id 555)

The same size as helloworld, but driving the engine rather than the hardware:

```
void game_boot(void) { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

That is the whole program. It exists to show the shortest path to a running engine tick, and as the skeleton to grow an engine-based game from.

neogeogame (id 775)

An empty game — the BIOS hooks, the linker script, an artbox directory, and nothing else. Rename it, set GAME_ID and GAME_NAME in its game.mk, and start writing. See ADDING_A_GAME.md.

Choosing a starting point

You want to...	Copy
Talk to the hardware directly	helloworld
Get an engine tick running immediately	tutorial
Start from a blank slate with the layout already right	neogeogame
See how a finished game is structured	skylance
Look up how one subsystem is actually driven	demo
Write the game in C++	demo_plus

PART 9

The Demo Chapter Map

The 25-chapter engine reel, chapter by chapter, with the controls and what each one isolates.

`docs/DEMO_CHAPTERS.md`

****Eagle Software · Neo Geo SDK v1.7.0 · games/demo, id 777****

The demo ROM is a single linear flow of 26 chapters. Each one isolates a subsystem, prints its own title and a one-line caption, and shows the chapter number in the top-right corner of the screen — so a problem can be reported as "chapter 14" rather than described.

```
make GAME=demo GAME_CFG_FILE=games/demo/game.cfg all
make GAME=demo GAME_CFG_FILE=games/demo/game.cfg test
```

Control	Effect
A	Advance to the next chapter
C	Restart the current chapter from the top

Chapters that need **C** for their own gameplay disable the restart binding for their duration.

The reel lives in `games/demo/scenes/demo_unified.c`, in `demo_unified_run()`. Every chapter is a function returning `uint8_t`, run through `run_chapter()`, which re-invokes it on a restart request — so no chapter needs its own reset path.

The chapters

#	Title	Subtitle	What it demonstrates
01	BOOT	ENGINE READY	Hardware brought up, engine initialised, chapter index explained
02	TITLE	ATTRACT REEL	Auto-centred attract sequence over generated title art
03	FIX LAYER	TEXT · PALETTES	The 40 × 28 visible text plane, three palettes, the dirty-cell cache
04	FIX FX	INFIX PHOTOS · 4 PHASE ANIMATION	Photographic INFIX pages, animated in four phases with palette cycling
05	SPRITE SCREENS	GENERATED FRAMES	The artbox screen reel — full-page sprite art drawn from <code>screens.c</code>
06	CHARACTERS	FULL WARRIOR MOVESET	A complete moveset cycle: stand, walk, strike, special A, special B
07	CHAR SELECT	PORTRAIT DEMO · JOYSTICK OVERRIDE	Portrait grid with live joystick selection
08	PHYSICS	GRAVITY · SOLIDS · GROUNDED	Gravity, a solid, and a grounded test — the eagle lands on the drawn floor line
09	CAMERA LAB	AUTO TOUR: HARD · SMOOTH · SHAKE · PAN · VERT · DEADZONE	Every camera mode in sequence, with parallax behind
10	PALETTE FX	SPRITE PALETTE STAGES	Fade, flash, pulse and colour-cycle staged on one sprite palette
11	PARTICLES	HERO SPECIAL MOVE + FX	Particles in narrative use — a special move with synced particle beats
12	PARTICLE LOAD	POOL STRESS + PRIORITY EVICTION	The 32-slot pool deliberately flooded, with the live count on screen

#	Title	Subtitle	What it demonstrates
13	FEEDBACK	SHAKE · PARTICLE IMPACTS	Hitstop + shake + palette flash + sound in one <code>ng_impact_event()</code> call, at four intensities
14	DEPTH FX	ONE WARRIOR · CLEAN Z SCALE	<code>ng_depthfx_project()</code> — perspective projection and Z → shrink
15	DEPTH PARALLAX	TWO SCROLLING BG LAYERS	Two independently scrolled sprite background layers at different rates
16	NPCS	PATROL + THINK CALLBACK	Three compact walking NPCs with distinct pacing, patrol bounds and facing
17	MINI-GAME	FIGHT THE CLONE · B STRIKE	Arrows + B. A multi-glyph sword arc with per-frame palette shimmer
18	JOYSTICK	LIVE INPUT · TWO-BUTTON SPECIALS	Live input readout; B+C light special, B+D heavy finisher, B basic strike, against a crate hitbox target
19	SCROLL LEVEL	WORLD MAP · H/V STAGES	A 768-px world with the camera following a jump arc
20	CHAR 2D	EAGLE FLIGHT ARC	Character flight over vertical parallax
21	TARGET RANGE	MOVING TARGETS · SPRITE DEPTH	Crates approaching down depth lanes, breaking apart where they are hit
22	DEPTH RIDE	OBJECTS APPROACH AS YOU WALK	Sprite-scaling pseudo-3D: NPCs approach from the horizon, L/R to move
23	SOUND	YM2610 TOUR · ADPCM-B / FM / SSG / SPEECH	Every audio subsystem in turn, with labels naming the active driver call
24	SKY LANCE	VERTICAL SHOOTER MINI	Waves, boss and scoring over a scrolling sky; separate sprite bands keep the FIX HUD readable
25	STAR RAID LANCE	FORMATION ASSAULT	Combined formation shooter, B fire, movement and C restart before the closing chapter
26	CREDITS	EAGLE SOFTWARE 2026	Module roll with a palette fade-out

Characters bind both the generated tile stride and per-tile palette map. Sky Lance and Star Raid Lance share three CC0 interceptor images imported at 32 pixels, with transparent edges and opaque hull highlights. Their source and license are recorded in `games/demo/artbox/INTERCEPTOR_CREDITS.txt`. Interactive MAME targets explicitly select normal speed and throttling; headless capture tools deliberately run unthrottled for testing. The repeating sky uses `fit=native` so its 144-pixel source repeat stays tile-aligned, without transparent resize padding at scroll wraps. Character colours and other screen-fit rules are unchanged. Chapter music beds use ADPCM-B hardware repeat instead of a scene-frame countdown that cut endings. Palettes are loaded separately; there are no hidden screen draws for loading character colours. The current verification tools capture every chapter and can exercise A-next/C-restart through real emulated controller inputs:

```
python3 tools/demo_capture.py --output /tmp/demo-tour --seconds 1100
python3 tools/demo_capture_report.py /tmp/demo-tour
python3 tools/demo_capture.py --output /tmp/demo-controls --seconds 1100 --controls
python3 tools/demo_capture.py --output /tmp/demo-shooter --chapter 25 --interval 0.016
python3 tools/sound_capture.py --output /tmp/demo-audio
python3 tools/sound_capture_report.py /tmp/demo-audio
python3 tools/demo_audio_capture.py --output /tmp/demo-live --press-every 9
python3 tools/demo_audio_report.py /tmp/demo-live --strict
```

Build the matching demo P1 immediately before capture: the script resolves chapter addresses from out/game. Do not build another game during a capture. --chapter advances earlier chapters with A, then records the selected one; use it separately from --controls. The audio probe boots the real ROM, isolates the 68000 in a watchdog-serviced RAM loop, and sends a fixed command sequence to the Z80. It records a WAV plus YM2610 writes to check tempo, LFO persistence and audible pitch modulation, sample rate, six A voices, B repeat, fades and muting. It says nothing about whether the demo itself gets its commands through: the live capture does, by booting the ROM set, leaving the 68000 in charge, recording the mix and failing on any chapter that goes quiet. The scene probe waits for the first chapter after coin/start before interpreting chapter indices, excluding BIOS RAM-test values.

Reading the reel as documentation

Each chapter is deliberately self-contained, and chap_header() performs a full hardware and engine reset at the top of every one:

```
ng_clear_screen_full();           /* VRAM + SCB wipe, on the clear colour */
demo_sprite_window_cache_reset();/* the tracking cache is NOT VRAM state */
setBACKDROP(DEMO_BG);
ng_level_set_scroll(0, 0);
ng_particles_init();
ng_feedback_init();
ng_palette_fx_init();
```

That reset is itself worth copying. Two of its lines exist because of bugs that were hard to find:

The sprite-window tracking cache is separate from VRAM. It is global across the whole ROM run, so slot numbers claimed by an early chapter stay claimed and starve a later, slot-heavy chapter until they are evicted. Resetting it per chapter gives each one the full budget.

****clearSprs() is deliberately *not* called here.**** ng_clear_screen_full() already does a full ng_sprite_hide_all() with the blank tile 0xFFFF. A second clearSprs() would overwrite that safe fill with tile 0 — which may carry actual art.

Page colours

The demo uses a white page with dark ink, defined once in demo.h:

```
#define DEMO_BG_CLEAR    BLACK    /* only during the wipe */
#define DEMO_BG          WHITE
#define DEMO_INK_BODY    BLACK
#define DEMO_INK_SUB     BLUE
#define DEMO_INK_ACCENT  MIDGREEN
#define DEMO_INK_PROMPT  RED
```

The clear runs on black and the page is restored to white afterwards, because a wipe straight to white flashes.

FIX row map

The demo's chapters share a row layout so captions never collide:

Row	Content	Written by
0	Chapter tag, title, index	<code>chap_header()</code> , once
1	Chapter subtitle	<code>chap_header()</code> , once
2-3	Per-chapter caption lines	the chapter
4-25	Chapter body	the chapter
26	Separator rule + control hint	<code>chap_header()</code> / <code>chap_hint()</code>
27	Bottom caption bar	<code>chap_header()</code>

Only 28 FIX rows are visible; the visible row y is map row $y + 2$, and the SDK's FIX helpers already apply that offset.

PART 10

Makefile Integration

How the per-game and per-engine build variables wire into the two makefiles.

`docs/MAKEFILE_INTEGRATION.md`

v1.7.0 additions - `USE_2D_PLUS=1` switches the engine source set to `sdk/2d_engine_plus/ng_*.cpp` (C++14, no exceptions, no RTTI, no threadsafe statics). Default (`USE_2D_PLUS=0`) builds the `sdk/2d_engine/ng_*.c` modules. - `GAME_EXTRA_INCLUDES` is a per-game variable that `game.mk` can set to add additional `-I...` paths to both `CFLAGS` and `CXXFLAGS` without modifying the top-level Makefile. Used by `games/demo_plus` to pull in `games/demo`'s artbox tables via `#include "../demo/main.c".`make # games/demo_plus/game.mk GAME_NAME = NeoGeo SDK Demo Plus GAME_ID = 778 GAME_SCENES = demo_plus_main GAME_EXTRA_INCLUDES = -Igames/demo -Igames/demo/artbox`` - New game `games/demo_plus` (ID 778) – builds clean with `make GAME=demo_plus p1 USE_2D_PLUS=1``.

The 2D game engine layer is built from the `sdk/2d_engine/ng_*.c` modules.

Current state on main:

- Makefile compiles the engine modules into `out/ng_*.o`
- `MakefileWin32.mak` compiles the same engine modules into `out\\ng_*.o`
- both link those objects into the main 68000 game binary
- both linker scripts place engine state in `game_engine_bss`
- P1 generation crops the linked program to the full 512 KB P-ROM window before byte swap and padding
- debug targets can emit symbol, map, readelf, disassembly, and GDB batch-trace files

This note documents the current wiring so you can extend it safely.

Current object lists

Linux:

```
NG_ENGINE_NAMES=ng_defs ng_properties ng_game_time ng_timers ng_progress ng_status ng_game_events ng_level ng_fix ng_sprite_group ng_actions ng_chars ng_npcs ng_physics ng_border_constraints ng_game_interrupt
NG_ENGINE_OBJ0=$(addprefix out/, $(addsuffix 0.o, $(NG_ENGINE_NAMES)))
```

Windows:

```
NG_ENGINE_OBJ0=out\ng_defs0.o out\ng_properties0.o out\ng_game_time0.o out\ng_timers0.o out\ng_progress0.o out\ng_status0.o out\ng_game_events0.o out\ng_level0.o out\ng_fix0.o out\ng_sprite_group0.o out\ng_actions0.o out\ng_chars0.o out\ng_npcs0.o out\ng_physics0.o out\ng_border_constraints0.o out\ng_game_interrupt0.o
```

Current linker state

Already prepared:

- `sdk/neogeo.ld`
- `sdk/neogeo_win.ld`

Both scripts place the engine modules into:

- `neogeo_user` for code and rodata
- `game_engine_bss` for BSS, COMMON, and data sections

The P1 path keeps the linked game in the `0x000000..0x080000` program ROM window. This is important once the 68000 program grows beyond 128 KB; cropping at `0x020000` would truncate code and can reset the game when execution reaches the missing region.

Minimal game-engine-enabled game pattern

In your 68000 source:

```
#include "sdk/2d_engine/ng_game_engine.h"

void game_boot(void)
{
    game_engine_init();
}

void game_frame(void)
{
    waitVbl();
    game_engine_frame();
}
```

Compatibility aliases remain available:

- `#include "sdk/2d_engine/ng_runtime.h"` still works
- `game_runtime_init()` still calls `game_engine_init()`
- `game_interrupt()` still calls `game_engine_frame()`

Use the game-engine names for new code.

Default module set

The current default build already includes:

- `ng_defs.c`
- `ng_properties.c`
- `ng_game_time.c`
- `ng_timers.c`
- `ng_progress.c`
- `ng_status.c`
- `ng_game_events.c`
- `ng_level.c`
- `ng_fix.c`
- `ng_sprite_group.c`
- `ng_actions.c`
- `ng_chars.c`
- `ng_npcs.c`
- `ng_physics.c`
- `ng_border_constraints.c`
- `ng_game_interrupt.c`

That gives you the full engine layer by default.

When adding a new engine module, update both `NG_ENGINE_NAMES` in `Makefile` and the explicit `NG_ENGINE_OBJ0` list in `MakefileWin32.mak`, then ensure the source is included by `sdk/2d_engine/ng_game_engine.h` if it is part of the public aggregate API.

Sound hook integration

The action system is designed to call back into the existing SDK sound API:

```
static void my_sfx(uint16_t id) { playSFX((uint8_t)id); }
static void my_music(uint16_t id) { playMusic((uint8_t)id); }

actions_set_sound_hooks(my_sfx, my_music);
```

You can also point music hooks at higher-level helpers such as:

- soundPlayTitleMusic()
- soundPlayGameLoop()

when your action scripts are scene-oriented rather than note-oriented.

Debug and GDB Trace Targets

Linux:

```
make debug-build
make debug-artifacts
make gdb-trace
make gdb
make gdb-server
make gdb-remote GDB_REMOTE=127.0.0.1:23946
```

Windows:

```
make -f MakefileWin32.mak debug-build
make -f MakefileWin32.mak debug-artifacts
make -f MakefileWin32.mak gdb-trace
```

DEBUG=1 adds:

- -g3
- -gdwarf-2
- -DNG_DEBUG=1
- linker map output at out/game.map

Generated debug files:

- dump/game.size.txt
- dump/game.sym
- dump/game.readelf
- dump/game.debug.dump
- dump/game.map
- dump/gdb_trace.gdb
- dump/gdb_trace.txt

If the bundled cross-GDB cannot run on the host, pass another compatible GDB:

```
make gdb-trace GDB=/path/to/m68k-gdb
```

PART 11

Debugging with GDB

Symbol addresses, remote debugging, and the trace workflow.

`docs/GDB_GUIDE.md`

v1.7.0 note *Debug builds work the same way for both engine variants. With USE_2D_PLUS=1, the engine source files are .cpp and you can step through C++ methods directly; GDB will demangle the symbols for you. Set a breakpoint in a demo chapter with: `` break demo\_unified.c:chap\_mini\_game break demo\_plus\_main.c:demo\_plus\_main\_run ``*

This guide covers how to use the cross-GDB (m68k-unknown-elf-gdb on Linux, m68k-elf-gdb.exe on Windows) with NeoGeoSDK builds.

Overview

The NeoGeoSDK makefile produces a standard ELF object (out/game) from the 68000 source. When built with DEBUG=1, it carries full DWARF-2 debug information including source line numbers, function names, and variable locations.

The debugging tools can:

- read symbols from out/game without a connected target
- connect to MAME's GDB debugger module and step live 68000 code
- inspect, break, and trace both the 68000 and Z80 with MAME's debugger
- run batch scripts to dump symbol tables, section layout, and function lists

Installing GDB

Linux / WSL

The cross-GDB is included in the x-tools-v3 release asset alongside the compiler (gdb 17.2 with TUI):

```
$SDKHOME/x-tools-v3/m68k-unknown-elf/bin/m68k-unknown-elf-gdb
```

If the bundled GDB cannot start because of missing host libraries (common on newer Ubuntu), install a system GDB that understands the m68k ELF format:

```
sudo apt-get install gdb-multiarch
```

Then override the GDB path in the makefile:

```
make gdb-trace GDB=gdb-multiarch
make gdb GDB=gdb-multiarch
make gdb-remote GDB=gdb-multiarch GDB_REMOTE=127.0.0.1:23946
```

Windows

On Windows, the makefile resolves GDB from the selected compiler prefix. With x-tools-v3-win it is:

```
<sdk root>\x-tools-v3-win\m68k-unknown-elf\bin\m68k-unknown-elf-gdb.exe
```

and with the previous x-tools-v2-win one of:

```
<sdk root>\x-tools-v2-win\m68k-unknown-elf\bin\m68k-unknown-elf-gdb.exe
<sdk root>\x-tools-v2-win\m68k-elf\bin\m68k-elf-gdb.exe
```

SysGCC fallback:

```
C:\SysGCC\m68k-elf\bin\m68k-elf-gdb.exe
```

The Win32 makefile sets GDB automatically from M68K_ELF_ROOT.

Debug build

Always build with `DEBUG=1` before using GDB:

```
make debug-build
```

```
make -f MakefileWin32.mak debug-build
```

This runs two steps:

1. `make DEBUG=1 p1` — compiles with `-g3 -gdwarf-2 -DNG_DEBUG=1` and links with a map file
2. `make DEBUG=1 debug-artifacts` — writes the following files to `dump/`:

File	Contents
<code>dump/game.size.txt</code>	section sizes from <code>size</code>
<code>dump/game.sym</code>	symbol table from <code>nm -n</code>
<code>dump/game.readelf</code>	full ELF metadata from <code>readelf -a</code>
<code>dump/game.debug.dump</code>	full disassembly with source from <code>objdump -DhtS</code>
<code>dump/game.map</code>	linker map (sections and symbol addresses)

make gdb

Opens an interactive GDB session on `out/game` with no target connected. Useful for inspecting symbols and disassembly offline:

```
make gdb
```

Inside GDB:

```
(gdb) info functions
(gdb) info variables
(gdb) disassemble playgame
(gdb) x/32xw 0x100200
(gdb) quit
```

make gdb-remote

Connects to a running target that exposes a GDB stub. NeoGeoSDK uses MAME's documented default port, 23946:

```
make gdb-remote
```

Or with an explicit address:

```
make gdb-remote GDB_REMOTE=127.0.0.1:23946
```

Use two terminals. In terminal 1, build the debug ELF and start MAME's GDB server:

```
make gdb-server
```

On Windows:

```
make -f MakefileWin32.mak gdb-server
```

In terminal 2, connect GDB:

```
make gdb-remote
```

GDB connects to the 68000 main CPU and can use the symbols in out/game:

```
(gdb) break playgame
(gdb) continue
(gdb) info registers
(gdb) stepi
(gdb) next
(gdb) print ng_level_state.scroll_x
```

Override the listening address or port when needed:

```
make gdb-server GDB_HOST=127.0.0.1 GDB_PORT=24000
make gdb-remote GDB_REMOTE=127.0.0.1:24000
```

MAME's GDB module attaches to the first CPU, which is the 68000 on Neo Geo. Use MAME's native debugger for live Z80 stepping and breakpoints.

Debugging both CPUs

Start the native debugger:

```
make debug
```

The relevant Neo Geo device tags are `maincpu` for the 68000 and `audiocpu` for the Z80. In the MAME debugger console:

```
focus maincpu
bp 100000:maincpu
go

focus audiocpu
bp 0038:audiocpu
bp 0066:audiocpu
go
```

`focus` controls which CPU is shown and observed. A device suffix on `bp` sets a breakpoint without relying on the currently selected CPU.

For terminal trace files from both CPUs:

```
make mame-trace
```

This generates and runs `dump/<game>/mame_trace.mds`, writing:

File	CPU
<code>dump/<game>/m68k_trace.txt</code>	Motorola 68000
<code>dump/<game>/z80_trace.txt</code>	Z80 sound CPU

Press F5 or enter go if the debugger remains paused after the script loads.

make gdb-trace

Runs GDB in batch mode and writes a full trace dump to `dump/gdb_trace.txt`:

```
make gdb-trace
```

The generated GDB script (`dump/gdb_trace.gdb`) runs:

```
set pagination off
set confirm off
file out/game
info files
info functions
info variables
maintenance info sections
quit
```

Output goes to `dump/gdb_trace.txt`. Any GDB startup errors go to `dump/gdb_trace.err`.

If GDB is unavailable, the target file contains a plain note instead of crashing the build.

Useful GDB commands for 68000 NeoGeo targets

```
info registers          # d0-d7, a0-a7, pc, sr
info registers d0       # single register
set $d0 = 0x1234        # write register
x/1xw 0x401000          # read VRAM register word
x/8xb 0x100200          # read 8 bytes from RAM
disassemble 0x1000, +64 # disassemble 16 instructions
break *0x1234           # breakpoint at address
watch *0x401800         # watchpoint on VRAM write
```

Linker map

The linker map at `dump/game.map` (produced by `debug-build`) shows the exact address of every function and data object. Use it alongside the symbol table:

```
grep playgame dump/game.map
grep ng_level_state dump/game.sym
```

Overriding GDB path

Both makefiles accept a GDB variable:

```
make gdb GDB=/usr/bin/gdb-multiarch
make gdb-trace GDB=/usr/bin/gdb-multiarch
make gdb-remote GDB=gdb-multiarch GDB_REMOTE=127.0.0.1:23946
```

```
make -f MakefileWin32.mak gdb GDB=D:\tools\m68k-gdb.exe
```

Notes

- The 68000 is big-endian; byte and word reads in GDB reflect that.
- MAME's GDB module does not support all GDB features. Single-step (stepi) and register reads are the primary workflow; use the native MAME debugger for the Z80 and device-specific watchpoints.
- out/game is the ELF target used by GDB. out/game.rom and out/777-p1.p1 are the final ROM images for MAME; GDB does not load those directly.

PART 12

ROM Bank Switching

The P-ROM banking mechanism and the ng_bankswitch stub, for future cartridges that outgrow the 512 KB window.

`docs/BANKSWITCH.md`

v1.7.0 status *The unified 21-chapter demo (games/demo, ID 777) and the new games/demo_plus (ID 778) both fit comfortably in the 512 KB P1 window without banking. Bank switching remains an advisory feature for future games that need more program space.*

This document describes the NeoGeo P-ROM banking mechanism and the `ng_bankswitch` API stub provided in `sdk/2d_engine/ng_bankswitch.h`.

The current NeoGeoSDK demo does not use bank switching. The P1 ROM is kept within the fixed 128 KB window. This document is advisory for future cartridge builds that need more than 512 KB of program space.

Background

The standard NeoGeo memory map exposes the P-ROM (program ROM) at `0x000000` – `0x0FFFFFF` (1 MB). Cartridges that fit within 512 KB use only the lower half of that window and never need banking.

Large cartridges (1 MB, 2 MB, 4 MB) divide the P-ROM into banks of 1 MB each. The first 1 MB (`0x000000` – `0x0FFFFFF`) is always bank 0 and is always visible. A second 1 MB window at `0x200000` – `0x2FFFFFF` is the banked region; writing a bank number to the bank switch register remaps this window to any 1 MB block of the physical ROM.

Address map (with banking active):

<code>0x000000</code> – <code>0x0FFFFFF</code>	fixed window – bank 0 always visible
<code>0x100000</code> – <code>0x1FFFFFF</code>	(not used for P-ROM on standard hardware)
<code>0x200000</code> – <code>0x2FFFFFF</code>	banked window – controlled by register at <code>0x2FFFE</code>

Bank switch register

Address: `0x2FFFE` (word-wide)
 Write: lower byte = bank number (0, 1, 2, ...)
 Effect: maps that 1 MB bank of physical P-ROM into `0x200000` – `0x2FFFFFF`

Example: to access bank 2 of the physical ROM at `0x200000`:

```
*((volatile uint16_t *)0x2FFFE) = 2;
/* now 0x200000 maps to physical ROM offset 0x200000 * 2 = 2 MB */
```

After the bank switch, all reads from `0x200000` – `0x2FFFFFF` come from the new bank until the register is written again.

SDK API

`sdk/2d_engine/ng_bankswitch.h` / `sdk/2d_engine/ng_bankswitch.c` provide two functions:

```
void    ng_bankswitch(uint8_t bank);
uint8_t ng_bankswitch_current(void);
```

`ng_bankswitch(bank)` writes bank to the hardware register and caches the value in a static variable.

`ng_bankswitch_current()` returns the cached bank number without reading the hardware register (the register is write-only on standard hardware).

How to use it (advisory)

1. Linker script

Add a banked section to your linker script. The fixed section stays at 0x000000:

```
SECTIONS {
    .text 0x000000 : { ... } /* fixed window */
    .bank1 0x200000 : AT(0x100000) { ... } /* banked data at physical 1 MB */
}
```

2. Place large data in a banked section

```
__attribute__((section(".bank1")))
static const uint8_t large_table[65536] = { ... };
```

3. Switch bank before accessing banked data

```
ng_bankswitch(1);
value = large_table[index]; /* reads from 0x200000 + offset */
ng_bankswitch(0);          /* restore if callers depend on bank 0 */
```

4. Never bank switch from interrupt context

The VBL and IRQ2 handlers run from the fixed window. If an interrupt fires during a bank switch, it will find the wrong bank in the 0x200000 window. Either keep interrupt code entirely in the fixed window (recommended) or have the interrupt handler save and restore the bank:

```
void NEOGEO_INT vbl_handler(void)
{
    uint8_t saved = ng_bankswitch_current();
    /* ... handler code ... */
    ng_bankswitch(saved);
}
```

5. Alignment

Each bank is a 1 MB block. Physical ROM regions must be padded to 1 MB boundaries before flashing.

Current demo limits

The current demo P1 is 128 KB. The makefile crops and pads to exactly 128 KB:

```
CROP=-crop 0x000000 0x020000
FILL=-fill 0xFF 0x000000 0x020000 -range-padding 4 -o
```

As long as the program fits in 128 KB, no bank switching is needed and `ng_bankswitch` is never called.

References

- SNK hardware documentation and third-party reverse-engineering notes describe the bank register at 0x2FFFFE.
- The BIOS provides a software helper (arcade: \$C00402, home: \$C004A2) that performs the same register write, but direct writes are equivalent and avoid BIOS calling convention overhead.
- WLA-DX linker documentation: <https://github.com/vhelin/wla-dx>

PART 13

SDK Library Reference

The bare-metal 68000 helpers declared in `sdk/neogeo.h` - sprite control blocks, palettes, VRAM, the FIX layer, timing, input, and sound.

`SDK_API_GUIDE.md`

v1.7.0 quick reference - *Sprite slot priority: **HIGHER slot number = drawn IN FRONT.** This is the observed hardware direction; earlier revisions of this guide and of `ng_sprite_pool.h` claimed the opposite, and the result was backgrounds parked at "behind" slots drawing over characters. Backgrounds belong at slots 1-32, characters at 96-223, foreground effects at 288+. - The backdrop register is the ****last word of palette RAM, \$401FFE****. \$402000 is a mirror that silently does nothing. - The FIX layer is 40 x 32 cells with **28 rows visible**; visible row *y* is map row *y* + 2. The FIX map word is `(pal << 12) | tile`, so there are only **16 palette banks**. - The BIOS SFIX font draws its space glyph as an **opaque** colour-index-2 plate. Use `ng_fix_blank_cell()` to leave a cell genuinely empty. - SCB2 can only **shrink**, never stretch. SCB3's Y field is 496 - *y* in nine bits, so *y* and *y* + 512 are identical and negative Y wraps. - C++14 engine: build with `USE_2D_PLUS=1` (engine in `sdk/2d_engine_plus/`). `games/demo_plus` (id 778) links only against the C++ engine and exists to keep the two builds ABI-compatible. - Makefile accepts `GAME_EXTRA_INCLUDES` (set by a game's `game.mk`) so a game can pull in another game's `artbox/header` path without duplicating data. - Every game carries its own `game.cfg`, so all six build the same way: `make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg all`.*

This guide covers the public 68000-side SDK helpers declared in `sdk/neogeo.h`.

For installation, SDKHOME layout, WSL usage, and Makefile targets, see:

- `README.md`
- `docs/GAME_ENGINE_LAYER.md`
- `docs/ARTBOX_PIPELINE.md`
- `docs/INTRODUCTION.md`
- `docs/PROGRAMMERS_MANUAL.md`
- `docs/API_2D_ENGINE_C.md`
- `docs/API_2D_ENGINE_CPP.md`
- `docs/MAKEFILE_INTEGRATION.md`

The SDK is organized in five layers:

1. Sprite control block builders
2. Palette and VRAM helpers
3. FIX layer and HUD helpers
4. Timing and input helpers
5. YM2610 sound helpers

Sprite Control Block Helpers

Use these helpers before calling `vram_sprite()` or the sprite move helpers.

Function	Purpose
<code>setSCB2(x_shrink, y_shrink)</code>	Build the sprite shrink word.
<code>setSCB3(y_pos, sticky_flag, height_factor)</code>	Build the sprite Y/sticky/height word.
<code>setSCB4(x_pos)</code>	Build the sprite X position word.
<code>setSCB1_2(pal_offset, tile_offset, bit3_autoanim, bit2_autoanim, vflip, hflip)</code>	Build the per-tile attribute word.
<code>setFIXDATA(palette_index, tile_number)</code>	Build a FIX layer tile word.

Palette and VRAM Helpers

These functions write directly to Neo Geo VRAM and palette RAM.

Function	Purpose
setBACKDROP(backdrop_color)	Set the backdrop color register.
load_palettes(palette, palette_offset)	Copy 16 palette entries into the palette RAM address passed in palette_offset.
vram_init(start, vram_inc)	Set the VRAM address and auto-increment registers.
setpal(...)	Fill a 16-entry palette buffer in RAM.
vram_SCB1(scb1_tiles, scb1_attrs, tiles_number)	Write SCB1 tile/attribute pairs.
vram_sfix(vram_inc, fix_addr, fix_data)	Write a FIX tile with explicit increment and address.
vram_sfix1(fix_data)	Write a FIX tile using the current VRAM cursor.
vram_SCB234(scb_addr, scb_value)	Write a single SCB2/3/4 value.
vram_sprite(...)	Upload a full sprite entry with SCB1/2/3/4 data.
vram_sprite_mv(x, vram_offset, scb4)	Update a sprite X position in VRAM.
vram_sprite_mv(y, vram_offset, scb3)	Update a sprite Y/height word in VRAM.

FIX Layer and HUD Helpers

Use these helpers for text, counters, debug overlays, and BIOS message coordination.

Function	Purpose
fixtext_out(x, y, text, pal)	Write an ASCII string to the FIX layer.
fixtext_out1(x, y, tiles, pal, count)	Write a tile array to the FIX layer.
fixtext_out2(x, y, a, b, c, mod, tiles, pal, count)	Variant helper used for generated FIX strings.
fix_svalue1(x, y, value, pal, offset)	Display a numeric value using a custom digit tile offset.
fix_svalue(x, y, value, pal)	Display a numeric value using the default ASCII digit offset.
display_digit(x, y, value, pal, offset)	Display a uint32_t value without floating-point helpers.
displayCreditP1() / displayCreditP2()	Show credit counters using BIOS memory.
mess_outtest()	Demo helper for the BIOS message output path.
setfix()	Enable the S-FIX border behavior used by the SDK demo.
setBIOSMESSBusy() / setBIOSMESSReady()	Gate access to the BIOS message output path.

Timing and Housekeeping Helpers

Function	Purpose
clearRAM()	Clear the SDK work RAM area.
clearSprs()	Clear the visible sprite list.

Function	Purpose
<code>clearFix()</code>	Clear the FIX layer. Restores BRDFIX (game S ROM) after the BIOS clear call.
<code>waitVbl()</code>	Wait for the next VBlank.
<code>cycle10ms()</code>	Rough 10 ms delay helper.
<code>cycle1s()</code>	Rough 1 second delay helper.
<code>cyclexms1(ticks_10ms)</code>	Delay in coarse 10 ms units.
<code>cyclexs(seconds)</code>	Delay in coarse second units.
<code>cyclexms(millisecond s)</code>	Delay in millisecond-style SDK units.
<code>kickWatchdog()</code>	Service the hardware watchdog register.
<code>sleep1FFF()</code>	Short busy-wait helper used by low-level routines.

Input and Status Helpers

Function	Purpose
<code>poll_joystick()</code>	Read the BIOS-managed joystick state.
<code>read_plcredit()</code>	Read the player 1 credit counter.
<code>read_joy1()</code>	Read the raw joystick hardware ports directly.

Sound API

The full command map, ranges, track indices, and driver behavior live in:

- `sound/SOUND_DRIVER_GUIDE.txt`
- `sound/driver/readme`

High-level sound control categories:

Category	Functions
Core control	<code>soundInit</code> , <code>soundReset</code> , <code>soundStopAll</code> , <code>soundStopMusic</code> , <code>soundCancelFade</code> , <code>soundSceneReset</code>
Raw command path	<code>playSoundtest</code> , <code>soundCommand</code> , <code>isZ80Ready</code>
Music and samples	<code>playMusic</code> , <code>playSFX</code> , <code>playSFXB</code>
FM	<code>playFMDebug</code> , <code>playFMTrack</code> , <code>soundSetFMVolume</code>
SSG	<code>playSSGTrack</code> , <code>soundSetSSGPreset</code> , <code>playInsertCoinSSG</code>
Voice cues	<code>playVoiceCue</code> , <code>playGetReadyVoice</code> , <code>playAttackVoice</code> , <code>playCoinThenReady</code>
Fade / mix	<code>soundFadeOut</code> , <code>soundFadeIn</code> , <code>soundFadeOutSpeed</code> , <code>soundFadeInSpeed</code> , <code>soundSetTempo</code> , <code>soundSetADPCMAVolume</code> , <code>soundSetADPCMBVolume</code> , <code>soundSetSSGVolume</code> , <code>soundApplyMix</code>

Category	Functions
Scene helpers	soundPlayDemoFM, soundPlayTitleMusic, soundPlayGameLoop

Current shipped helper behavior:

- `soundPlayTitleMusic(track)` resets the scene, plays the title gong, then starts the ADPCM-B title theme
- `soundPlayGameLoop(track)` resets the scene and maps the requested loop to a direct ADPCM-B stage or ending TRACK
- `playMusic(track)` remains the explicit path for MML/SSG playback when you want the music engine directly

2D Game Engine Layer

The repository also includes a reusable game engine layer under `sdk/2d_engine/ng_*`.

Primary entry header:

```
#include "sdk/2d_engine/ng_game_engine.h"
```

Core startup:

```
game_engine_init();
```

Per-frame entry:

```
waitVbl();
game_engine_frame();
```

Main runtime modules:

Module	Purpose
<code>ng_actions</code>	const action scripts with commands like FRAME, WAIT, MOVE, SFX, MUSIC, EVENT, LOOP
<code>ng_chars</code>	fixed-size character pool with fixed-point motion and sprite binding
<code>ng_game_events</code>	small queued event system
<code>ng_timers</code>	frame timers
<code>ng_progress</code>	progress counters
<code>ng_status</code>	boolean status flags
<code>ng_properties</code>	live shared values grouped by system
<code>ng_level</code>	level state, world bounds, camera follow, and joystick camera scroll
<code>ng_fix</code>	cached FIX-layer text and numeric output
<code>ng_npcs</code>	fixed-size NPC slots bound to characters and think callbacks
<code>ng_physics</code>	fixed-point gravity, drag, world bounds, and solid-rectangle resolution
<code>ng_border_constraints</code>	invisible trigger rectangles that emit events
<code>ng_sprite_group</code>	grouped Neo Geo sprite upload helpers

Important current build note:

- the linker scripts reserve game_engine_bss for out/ng_*.o
- Makefile and MakefileWin32.mak compile and link the sdk/2d_engine/ng_* modules by default
- use docs/MAKEFILE_INTEGRATION.md to inspect the exact build and linker wiring

Camera helpers:

```
level_set_world_bounds(0, 0, 639, 447);
level_camera_follow(player->x, player->y, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_X, 320, 224); /* horizontal scroll */
level_camera_joystick(2, NG_CAMERA_AXIS_Y, 320, 224); /* vertical scroll */
level_camera_joystick(2, NG_CAMERA_AXIS_BOTH, 320, 224); /* free camera */
```

Characters keep world-space x/y values for physics and collision. The engine subtracts the current level scroll when drawing sprites.

Action-state pattern:

```
static const NGAActionCmd player_run[] = {
    SPEED(0, 0),
    FRAME(tile_run_0, 3),
    FRAME(tile_run_1, 3),
    LOOP()
};

actions_register(ACT_PLAYER_RUN, player_run);
char_action(player, ACT_PLAYER_RUN);
```

Use a per-character callback to map joystick combinations to actions. The demo uses the priority attack, jump, run, idle so combinations produce deterministic results.

NPC and opponent pattern:

```
NGNpc *opponent = npc_spawn(OPPONENT_KIND, CHAR_OPPONENT, 240, 180);
npc_set_think(opponent, opponent_think, 1);

if (ng_rect_hit(char_body_rect(player), char_body_rect(npc_char(opponent)))) {
    char_damage(player, 1);
}
```

Artbox asset categories:

Category	Typical PNG names	Use
background	background_*.png, zz_npc_forest_alley.png, legacy numbered pages	full-screen scene/background pages
main_character	sprite_*.png	player/main-character animation frames
opponent	opponent_*.png	enemy/opponent sprite frames
npc	z_npc_*.png	non-player character sprite frames

The category is written into artbox/assets_manifest.json and generated into artbox/sprite_meta.h as NG_ASSET_CATEGORY_*.

See docs/ARTBOX_PIPELINE.md for the full rule and category reference.

Typical Call Patterns

One-time startup

```
soundInit();
clearSprs();
clearFix();
```

Draw a sprite and move it

```
uint16_t scb2 = setSCB2(0x0F, 0xAF);
uint16_t scb3 = setSCB3(496 - y, 1, 9);
uint16_t scb4 = setSCB4(x);

vram_sprite(vram_start, 1, vram_offset, tiles, attrs, tile_count, scb2, scb3, scb4);
vram_sprite_mv(x, vram_offset, setSCB4(x + 8));
```

FIX text and number output

```
fixtext_out(10, 10, "INSERT COIN", 0);
display_digit(20, 15, score, 0, 48);
```

Direct title or stage theme

```
soundPlayTitleMusic(0);
soundPlayGameLoop(SOUND_MUSIC_A);
```

Manual layered sound scene

```
soundSceneReset();
soundSetADPCMAVolume(0x34);
soundSetADPCMBVolume(0xB8);
soundSetSSGVolume(0x08);
soundSetFMVolume(0x0C);

playSFX(SOUND_SFX_1);
playMusic(SOUND_MUSIC_A);
playFMTrack(SOUND_FM_B);
```

Notes

- sdk/neogeo.h is the intended public call surface for the 68000 side.
- The playable sound driver is the assembler implementation in sound/driver/driver.asm.
- make m1rom / make m1rom-asm builds the authoritative ASM sound runtime.
- make m1rom-c builds the experimental C-linked sound runtime for comparison only.
- The current 68000 build no longer links the legacy softfloat/ runtime by default.
- clearFix() calls the BIOS SYS_FIX_CLEAR routine which resets BRDFIX to 0. The SDK now restores BRDFIX (BSET.B #0,REG_BRDFIX) immediately after that call so the game S ROM is always selected. Never call clearFix() without following up with text draws — the game S ROM is restored automatically.
- soundCommand() no longer calls isZ80Ready() after writing the command byte. The trailing poll created a race condition with the Z80 NMI handler that caused a permanent 68k deadlock. Callers

that need to verify Z80 readiness before sending a second command should call `isZ80Ready()` explicitly between commands.

PART 14

2D Engine Layer Guide

The engine layer end to end: headers, startup, the per-frame call, characters, action scripts, levels, NPCs, physics, and events.

`docs/GAME_ENGINE_LAYER.md`

v1.7.0 — engine occlusion direction *The boxed rule at the top of*

*sdk/2d_engine/ng_sprite_pool.h (and the sdk/2d_engine_plus/ng_sprite_pool.hpp copy) is authoritative: **LOWER hardware slot = drawn BEHIND higher-numbered sprites** when they overlap. The Neo Geo LSPC walks the sprite control blocks sequentially from slot 0 upward, so a sprite at slot 1 is written first and any later slot covers it. Backgrounds belong at low slots 1–32 (NG_SPR_BG0_FIRST=1, NG_SPR_BG1_FIRST=17); characters at 96–223 (NG_SPR_CHAR_FIRST..NG_SPR_CHAR_LAST); FX, particles and temporary front effects at higher slots so they draw on top. See sdk/2d_engine/ng_sprite_pool.h for the canonical layout. Both sdk/2d_engine/ (C, gnu99) and sdk/2d_engine_plus/ (C++14, -fno-exceptions -fno-rtti -fno-threadsafe-statics) ship with the same public API surface. Select with USE_2D_PLUS=0 or USE_2D_PLUS=1 on the make line.*

This repository carries a reusable 2D game engine layer under sdk/2d_engine/ng_*.

The layer is plain C. No float, no malloc during gameplay, no division in the frame loop. It is not a C++ object system and not an entity-component framework.

Module overview

Core engine modules

Module	Header	Purpose
Definitions	ng_defs.h	Shared types, constants, pool sizes
Characters	ng_chars.h	Fixed-size character pool, per-kind callbacks
Actions	ng_actions.h	Action script execution (FRAME, WAIT, MOVE, SFX, ...)
Level	ng_level.h	Level bounds, camera scroll, joystick camera helpers
Background	ng_bg.h	Sprite-based background layer with parallax ratios
FIX cache	ng_fix.h	Cached FIX tile writes — only rewrites changed cells
NPCs	ng_npcs.h	NPC pool with patrol, think hooks, home/bounds
Physics	ng_physics.h	Fixed-point velocity, gravity, collision solids
Game events	ng_game_events.h	Fixed ring-buffer event queue
Border constraints	ng_border_constraints.h	Trigger rects that emit events on enter
Status	ng_status.h	Bit-flag status set
Timers	ng_timers.h	Fixed-size timer pool
Progress	ng_progress.h	Named progress counters
Properties	ng_properties.h	Runtime shared-value matrix
Game time	ng_game_time.h	Frame counter
Interrupt	ng_game_interrupt.h	Per-frame hook dispatch

Deluxe 2D engine modules (v1.7.0+)

Module	Header	Purpose
Sprite groups	ng_sprite_group.h	Dirty-flag sticky-bit sprite chains
Render queue	ng_render_queue.h	128-slot VBlank-safe deferred VRAM/palette writes
Fixed-point	ng_fixed.h	16.16 fixed-point math, sin/cos/shrink lookup tables

Module	Header	Purpose
Camera	ng_camera.h	Smooth follow, dead zone, shake, cinematic pan
Palette FX	ng_palette_fx.h	Fade, flash, pulse, color cycle — queue-safe
Particles	ng_particles.h	32-slot pool, 8 types, priority eviction
Feedback	ng_feedback.h	Hitstop + shake + flash + sound hook in one call
Depth FX	ng_depthfx.h	NGVec3 perspective projection, Z→shrink/fog
Debug HUD	ng_debug.h	Fix-layer perf overlay (NG_DEBUG_PERF=1)

VRAM and CRAM write contract

The Neo Geo LSPC and the 68000 share the VRAM and CRAM buses. Writes that land during active video can collide with the LSPC's per-scanline reads and show up as torn sprites, partial palettes, or "snow" on the display.

Current (v1.3.x) contract

Subsystem	Where it writes	Status
ng_bg_draw()	Direct SCB1/SCB2/SCB3/SCB4 writes	Direct, must run inside the vblank window after waitVbl()
ng_chars_draw()	Direct SCB1/SCB2/SCB3/SCB4 writes	Direct, must run inside the vblank window after waitVbl()
ng_particles_draw()	Direct SCB writes	Direct, must run inside the vblank window
ng_render_queue_flush()	Drains queued SCB / palette commands	Vblank-safe by design
ng_palette_fx_*	Queues via ng_render_queue	Vblank-safe
ng_feedback_*	State only; rendering rides ng_render_queue	Vblank-safe
demo_draw_sprite_screen{,_flip}	Queues into the demo's sprite queue	Vblank-safe (drained at uframe top)

What this means in practice:

- ****CRAM writes (palette RAM at 0x400000) MUST go through the render queue.**** Use `ng_rq_palette_upload()` or `ng_palette_fx_*`. Writing the palette directly from game logic during active video produces rolling "snow" pixels on every line currently being scanned out.
- ****The big draws (`ng_bg_draw`, `ng_chars_draw`, `ng_particles_draw`) are still direct VRAM writes**, but the engine drives them only from inside the per-frame pump immediately after `waitVbl()` returns, so the writes complete before the LSPC starts scanning the next frame.**
- A future v1.4.0 may move those big draws fully through the queue; until then, keep custom rendering code aligned with the pump order (`waitVbl()` -> draws -> `ng_render_queue_flush()` -> updates).

Headers

Use the aggregate include:

```
#include "sdk/2d_engine/ng_engine.h"
```

Or include only the modules you need:

- sdk/2d_engine/ng_defs.h
- sdk/2d_engine/ng_actions.h
- sdk/2d_engine/ng_chars.h
- sdk/2d_engine/ng_game_events.h
- sdk/2d_engine/ng_game_interrupt.h
- sdk/2d_engine/ng_game_time.h
- sdk/2d_engine/ng_progress.h
- sdk/2d_engine/ng_properties.h
- sdk/2d_engine/ng_level.h
- sdk/2d_engine/ng_fix.h
- sdk/2d_engine/ng_npcs.h
- sdk/2d_engine/ng_physics.h
- sdk/2d_engine/ng_status.h
- sdk/2d_engine/ng_timers.h
- sdk/2d_engine/ng_border_constraints.h
- sdk/2d_engine/ng_sprite_group.h
- sdk/2d_engine/ng_render_queue.h
- sdk/2d_engine/ng_fixed.h
- sdk/2d_engine/ng_camera.h
- sdk/2d_engine/ng_palette_fx.h
- sdk/2d_engine/ng_particles.h
- sdk/2d_engine/ng_feedback.h
- sdk/2d_engine/ng_depthfx.h
- sdk/2d_engine/ng_debug.h

Startup

Initialize the game engine once during game startup:

```
game_engine_init();
```

That resets:

- game time
- timers
- progress slots
- status flags
- event queue
- level and camera state
- cached FIX text state
- border constraints
- action table
- character pool
- NPC pool

- physics bodies and solids

Per-frame call

Call the game engine once per frame after `waitVbl()`:

```
waitVbl();
game_engine_frame();
```

`game_engine_frame()` runs this sequence:

1. `game_time_tick()`
2. optional `before_logic` hook
3. `timers_update()`
4. `level_update()`
5. `npcs_update()`
6. `physics_update_pre()`
7. `border_constraints_update()`
8. `chars_update()`
9. `physics_resolve()`
10. optional `collision_logic` hook
11. `game_events_update()`
12. optional `after_events` hook
13. `progress_update()`
14. optional `before_draw` hook
15. `chars_draw()`
16. optional `after_draw` hook

Optional frame hooks

You can inject high-level callbacks around the engine frame pass:

```
game_engine_set_hooks(
    before_logic,
    collision_logic,
    after_events,
    before_draw,
    after_draw
);
```

Characters

Characters are stored in a fixed pool of `NG_MAX_CHARS`.

Core calls:

```
NGCharacter *c = chars_add(kind, x, y);
chars_remove(c);
chars_clear_kind(kind);
chars_find(kind);
chars_at(index);
chars_count();
```

Each NGCharacter holds:

- active/visible state
- kind, state, facing
- integer and fixed-point position
- velocity
- current action-script position
- sprite binding and palette
- body rectangle
- hit rectangle
- HP
- general-purpose scratch fields data0...data2

Character logic callback

Register one per-kind callback:

```
chars_set_game_interrupt(DEMO_CHAR_PLAYER, player_tick);
```

That callback is executed once per frame for each active character of that kind.

Sprite binding

Attach a Neo Geo sprite strip group to a character:

For generated multi-palette artwork, also bind the asset's tile stride and tile_palettes map using `ng_char_set_tile_stride()` followed by `ng_char_set_palette_map()`. The latter accepts NULL for single-bank art. `ng_char_bind_asset()` validates an entire NGSpriteAssetView atomically. See Artbox Graphics Pipeline for lifetime, reset and C++ API details. Recompile game objects after updating these structs.

```
char_set_sprite(c, 32, 6, 16, tile_base, NG_PAL_PLAYER_BASE);  
char_set_body(c, -8, -24, 16, 32);  
char_set_pos(c, 120, 180);  
char_set_speed(c, 1, 0);
```

Useful helpers:

- `char_set_sprite`
- `char_set_body`
- `char_set_pos`
- `char_set_speed`
- `char_damage`
- `char_heal`
- `char_body_rect`
- `char_hit_rect`

Sprite-group level helpers live in `sdk/2d_engine/ng_sprite_group.h`.

Characters store world-space x/y coordinates. `chars_draw()` subtracts the current level scroll before writing sprite transforms, so physics, collisions, and AI stay in world coordinates while rendering happens in screen space.

Action scripts

Action scripts are const arrays of `NGActionCmd`.

Register them:

```
actions_register(ACT_PLAYER_WALK, player_walk_script);
char_action(player, ACT_PLAYER_WALK);
```

Useful commands:

- `FRAME(tile, ticks)`
- `WAIT(ticks)`
- `MOVE(dx, dy)`
- `SPEED(vx, vy)`
- `SCALE(sx, sy)`
- `PALETTE(bank)`
- `FLIP(hflip, vflip)`
- `SFX(id)`
- `MUSIC(id)`
- `FX(kind, dx, dy)`
- `HITBOX(x, y, w, h)`
- `CLEAR_HITBOX()`
- `EVENT(id, a, b)`
- `GOTO(action_id)`
- `LOOP()`
- `END()`

Sound hooks are externalized:

```
actions_set_sound_hooks(sfx_hook, music_hook);
actions_set_fx_hook(fx_hook);
```

That keeps the game engine layer independent from a specific game sound policy.

Input-driven action state

Use the per-kind character callback to translate input into action scripts. Keep the priority explicit when several buttons are pressed at once:

```
static void player_tick(NGCharacter *player)
{
    uint16_t input = poll_joystick();

    if (input & (1u << CNT_A)) {
        char_action(player, ACT_PLAYER_ATTACK);
    } else if (input & (1u << CNT_UP)) {
        char_action(player, ACT_PLAYER_JUMP);
    } else if (input & ((1u << CNT_LEFT) | (1u << CNT_RIGHT))) {
        char_action(player, ACT_PLAYER_RUN);
    } else {
        char_action(player, ACT_PLAYER_IDLE);
    }
}
```

The demo follows this finite-state-machine style for idle, run, jump, hit, and attack. One active action owns the current animation, hitbox, speed changes, and sound cues.

Level and Camera

ng_level owns level metadata, scroll values, camera properties, and world bounds:

```
level_set_world_bounds(0, 0, 639, 447);
level_set_camera(0, 0, 320, 224);
level_move_camera(2, 0, 320, 224);
level_camera_follow(player->x, player->y, 320, 224);
```

Joystick camera helpers:

```
level_camera_joystick(2, NG_CAMERA_AXIS_X, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_Y, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_BOTH, 320, 224);
```

level_set_camera() clamps to the current world bounds and writes both NG_PROP_GROUP_LEVEL scroll values and NG_PROP_GROUP_CAMERA values.

NPCs

ng_npcs binds small NPC records to normal NGCharacter slots.

```
NGNpc *npc = npc_spawn(NPC_KIND, CHAR_KIND, 160, 180);
npc_set_home(npc, 160, 180);
npc_set_patrol_bounds(npc, 80, 240, 180, 180);
npc_set_think(npc, npc_think_patrol, 1);
```

Use npc_char(npc) to access the backing character for body rectangles, sprites, HP, and actions.

Physics

ng_physics supplies optional fixed-point movement support:

```
physics_attach(player, NG_PHYSICS_GRAVITY | NG_PHYSICS_WORLD | NG_PHYSICS_SOLIDS);
physics_set_gravity(player, NG_FP_FROM_FRAC(1, 4), NG_TO_FP(4));
physics_add_solid(0, 208, 320, 16, 0);
```

The frame pass calls physics_update_pre() before character updates and physics_resolve() after character updates.

FIX Cache

ng_fix wraps FIX text writes with a small cache so repeated HUD/debug strings do not rewrite unchanged cells every frame:

```
ng_fix_puts(2, 1, "READY", 0);
ng_fix_put_u16(10, 1, lives, 0, 48);
```

Game events

Events are queued in a fixed ring buffer:

```
game_events_set_handler(my_handler);
game_events_send(id, a, b, c);
game_events_read(&event);
game_events_count();
```

Use events for:

- trigger volumes
- combat notifications
- script progression
- UI or stage transitions

Timers, progress, status, properties

Timers:

```
timer_start(id, frames);
timer_stop(id);
timer_active(id);
timer_done(id);
timer_value(id);
timer_percent_left(id);
```

Progress:

```
progress_start(id, max);
progress_set(id, value);
progress_add(id, amount);
progress_value(id);
progress_percent(id);
progress_done(id);
```

Status:

```
status_set(id);
status_clear(id);
status_has(id);
status_toggle(id);
```

Properties matrix:

```
prop_set(group, id, value);
prop_get(group, id);
prop_add(group, id, amount);
prop_clear_group(group);
```

The properties matrix is the runtime's shared live-value table. It is a good fit for player position, stage scroll, level mode, boss phase, timers, and debug values that many systems need to read.

Border constraints

Border constraints are invisible rectangles that emit game events when the player rectangle enters them.

```
border_constraints_load(list, count);
border_constraints_update();
border_constraints_reset_used();
```

Each `NGBorderConstraint` carries:

- `x, y, w, h`
- `event_id`
- payload `a, b`
- `once`
- `used`

Use them for:

- boss-room start triggers
- camera locks
- cutscene triggers
- tutorial or message gates

Runtime limits

See `sdk/2d_engine/ng_defs.h` for the fixed capacities:

- `NG_MAX_CHARS`
- `NG_MAX_ACTIONS`
- `NG_MAX_GAME_EVENTS`
- `NG_MAX_TIMERS`
- `NG_MAX_PROGRESS`
- `NG_MAX_STATUS`
- `NG_MAX_BORDER_CONSTRAINTS`
- `NG_MAX_NPCS`
- `NG_MAX_SOLIDS`
- `NG_FIX_WIDTH`
- `NG_FIX_HEIGHT`

Important build note

The engine source files are present in the repository, the linker scripts reserve `game_engine_bss` for `out/ng_*.o`, and the default makefiles now compile and link the `sdk/2d_engine/ng_*.c` modules automatically.

For the current build wiring, follow:

- `docs/MAKEFILE_INTEGRATION.md`

Compatibility aliases

The old names are still accepted for existing code:

- `#include "sdk/2d_engine/ng_runtime.h"`
- `game_runtime_init()`
- `game_interrupt_set_hooks()`
- `game_interrupt()`

PART 15

The FIX SDK Module

The standalone low-level FIX-layer helper module, and how it differs from the engine's cached FIX text.

`docs/NG_FIX_SDK.md`

sdk/ng_fix is the low-level FIX-layer helper module for the NeoGeo SDK.

The FIX layer is the 8x8 tile plane sourced from the S1 ROM. It is best used for HUD text, scores, captions, debug overlays, menu frames, bars, simple UI icons, and scene labels. It is not a scrolling background layer.

Files

```
#include "sdk/ng_fix/ng_fix.h"
```

The module is intentionally separate from sdk/2d_engine. The 2D engine may use it, but the FIX SDK can also be used directly by title screens, sound tests, the BIOS/demo flow, and diagnostic tools.

Core ideas

- Screen size: 40x32 FIX cells.
- Each cell is one 8x8 S1 tile.
- Palette is limited to 0..15.
- Blank cells use tile \$00FF by default, matching the BIOS clear pattern.
- Cached writes avoid rewriting unchanged FIX cells every frame.

Basic text

```
ngfix_init();  
ngfix_clear();  
ngfix_puts(2, 2, "NEO GEO SDK", 0);  
ngfix_center(4, "FIX LAYER TEST", 1);
```

Raw tile output

Use raw tile writes when the S1 ROM contains icons, Japanese glyphs, custom borders, meters, or font tiles that are not ASCII text.

```
ngfix_put_tile(10, 8, 0x0120, 2); /* tile $120, palette 2 */  
ngfix_write_tile(11, 8, 0x0121, 2); /* forced hardware write */
```

Custom font mapping

The default mapping uses the character byte as the tile number, preserving the existing SDK behavior. For a custom S1 font, set `ascii_base`.

```
NGFixFont font = { 0x0200, 0x00FF, '?', 1 };  
ngfix_set_font(&font);  
ngfix_puts(2, 2, "CUSTOM FONT", 0);
```

Numbers, boxes, bars

```
ngfix_put_u16(2, 5, score, 0, 5);  
ngfix_put_hex16(2, 6, 0x7777, 1);  
ngfix_box(1, 1, 38, 6, 0);  
ngfix_bar_u8(4, 8, 16, hp, max_hp, 2, '#', '-');
```

Compatibility

sdk/2d_engine/ng_fix.c now delegates to this module, so existing calls such as `ng_fix_puts()` continue to work while the standalone FIX SDK becomes the common implementation.

PART 16

Scene and Video Ownership

The model that keeps generated screens, sprite chains, and raw effects from fighting over the same hardware state.

`docs/NEOUNIVERSAL_VIDEO_MODEL.md`

This note documents the scene/video ownership model added for the NeoGeoSDK demo and engine layers.

Purpose

Neo Geo software is visually powerful, but many systems can write to the same hardware state:

- generated showScreenN() functions
- engine character sprite chains
- raw pseudo-3D floor effects
- software raycaster columns
- MVS eyecatcher code
- FIX text/HUD output
- palette loading

When those systems are called directly, sprites can remain partially enabled, old SCB3 sticky-chain data can survive between scenes, and the next scene may show only one vertical strip or become visually occluded.

The new model separates ownership:

```
sdk/ng_video/    scene-level video ownership, clears, captions, sprite ranges
sdk/ng_show/     safe wrapper for generated showScreenN() art
sdk/ng_scene/    scene sequencing and manual advance flow
sdk/ng_audio/    sound-scene helpers and labeled playback
sdk/ng_fix/      FIX/S1 8x8 tile and text layer
sdk/2d_engine/ng_depthfx.* optical pseudo-3D depth for 2D sprites
```

Rules

1. Demo code should use ngvideo_begin_scene() before a visual scene.
2. Generated showScreenN() calls should go through ngshow_draw() or ngshow_draw_simple().
3. Captions and HUD text should use ngfix_*, not raw fixtext_out().
4. 2D sprite depth illusions should use ng_depthfx_apply_y() rather than local scale code.
5. A scene should end with ngvideo_end_scene() or ngvideo_hard_clear().

Result

The demo becomes a stable compositor instead of unrelated routines fighting for SCB state. This is still true NeoGeo-style 2D: no polygon renderer is implied. The depth effects are optical illusions made from sprite shrink, Y sorting, priority bands, FIX overlays and careful scene timing.

PART 17

2D Engine Reference — C

Every public call in the plain-C 2D engine, grouped by module.

`docs/API_2D_ENGINE_C.md`

****Eagle Software · Neo Geo SDK v1.7.0 · sdk/2d_engine/****

Every public call in the plain-C 2D engine, grouped by module. This file is generated from the headers by `tools/gen_api_reference.py` — the prototypes here are the prototypes that exist.

The C++ build of the same engine is documented in `API_2D_ENGINE_CPP.md`; it exposes these same extern "C" entry points plus method wrappers, so the two are link-compatible.

```
#include "sdk/2d_engine/ng_engine.h"  /* everything */
```

Every function carries the `NEOGEO_USER` linkage tag, which places it in the cartridge's user code section. It is omitted from the signatures below for readability.

Contents

Core — `ng_defs` · `ng_engine` · `ng_game_events` · `ng_game_interrupt` · `ng_game_time` · `ng_progress` · `ng_properties` · `ng_runtime` · `ng_scene` · `ng_status` · `ng_timers`

Math — `ng_fixed`

Video — `ng_art_asset` · `ng_bg` · `ng_depthfx` · `ng_fix` · `ng_palette_assets` · `ng_palette_fx` · `ng_render_queue` · `ng_sprite_group` · `ng_sprite_pool` · `ng_sprite_window` · `ng_vram`

Characters — `ng_actions` · `ng_border_constraints` · `ng_chars` · `ng_joystick` · `ng_npcs` · `ng_physics`

World — `ng_camera` · `ng_level`

Effects — `ng_debug` · `ng_demo_advanced` · `ng_feedback` · `ng_particles`

Core

ng_defs

`sdk/2d_engine/ng_defs.h` — Sizes, engine constants, and the `NEOGEO_USER` linkage tag.

```
uint16_t ng_abs16(int16_t v);
```

Constant	Value
<code>NG_TRUE</code>	1
<code>NG_FALSE</code>	0
<code>NG_FRAME_RATE</code>	60
<code>NG_FRAMES_PER_SEC</code>	<code>NG_FRAME_RATE</code>
<code>NG_SCREEN_WIDTH</code>	320
<code>NG_SCREEN_HEIGHT</code>	224
<code>NG_SPRITE_CULL_LEFT</code>	-64
<code>NG_SPRITE_CULL_RIGHT</code>	384
<code>NG_SPRITE_CULL_TOP</code>	-128
<code>NG_SPRITE_CULL_BOTTOM</code>	640

Constant	Value
NG_RENDER_BAND_BACK	0
NG_RENDER_BAND_NPC	1
NG_RENDER_BAND_ENEMY	2
NG_RENDER_BAND_PLAYER	3
NG_RENDER_BAND_FX	4
NG_RENDER_BAND_FRONT	5
NG_FP_SHIFT	8
NG_FP_ONE	(1 << NG_FP_SHIFT)
NG_MAX_CHARS	64
NG_MAX_CHAR_KINDS	128
NG_MAX_ACTIONS	256
NG_MAX_GAME_EVENTS	32
NG_MAX_TIMERS	64
NG_MAX_PROGRESS	64
NG_MAX_STATUS	256
NG_PROP_GROUPS	16
NG_PROP_COUNT	32
NG_MAX_BORDER_CONSTRAINTS	128
NG_MAX_NPCS	32
NG_MAX_SOLIDS	64
NG_FIX_WIDTH	40
NG_FIX_HEIGHT	32
NG_SPRITE_MAX_STRIPS	32
NG_SPRITE_MAX_HEIGHT_TILES	32
NG_SPRITE_FULL_XSCALE	0xff
NG_SPRITE_FULL_YSCALE	0xff
NG_PAL_FIX_BASE	0x00
NG_PAL_UI_BASE	0x04
NG_PAL_STAGE_BASE	0x10
NG_PAL_PLAYER_BASE	0x20
NG_PAL_ENEMY_BASE	0x30
NG_PAL_FX_BASE	0x40
NG_PAL_BOSS_BASE	0x50
NG_PAL_CUTSCENE_BASE	0x60

ng_engine

sdk/2d_engine/ng_engine.h — Umbrella include. One #include pulls in every public module.

Types and declarations only — see the header.

ng_game_events

sdk/2d_engine/ng_game_events.h — A small event queue with an optional handler callback.

```
void ng_game_events_set_handler(NGGameEventHandler handler);
uint8_t ng_game_events_send(uint16_t id, uint16_t a, uint16_t b, uint16_t c);
uint8_t ng_game_events_read(NGGameEvent *out);
void ng_game_events_update(void);
uint8_t ng_game_events_count(void);
```

ng_game_interrupt

sdk/2d_engine/ng_game_interrupt.h — The frame tick and the five hooks you attach your game to.

ng_game_engine_frame() runs the whole engine in a fixed order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — and flushes the render queue during VBlank. Register your own code at the five hook points rather than reimplementing the tick.

```
void ng_game_runtime_init(void);
void ng_game_engine_set_hooks( NGInterruptHook before_logic, NGInterruptHook collision_logic, NGInterruptHook after_events, NGInterruptHook before_draw, NGInterruptHook after_draw );
void ng_game_interrupt_set_hooks( NGInterruptHook before_logic, NGInterruptHook collision_logic, NGInterruptHook after_events, NGInterruptHook before_draw, NGInterruptHook after_draw );
void ng_game_engine_frame(void);
void ng_game_interrupt(void);
```

ng_game_time

sdk/2d_engine/ng_game_time.h — Frame and stage counters at 60 Hz.

```
void ng_game_time_tick(void);
void ng_game_time_reset_stage(void);
uint32_t ng_game_time_frame(void);
uint16_t ng_game_time_second(void);
uint32_t ng_game_time_stage_frame(void);
```

ng_progress

sdk/2d_engine/ng_progress.h — Bounded 0..max counters — charge meters, load bars, HP ramps.

```
void ng_progress_start(uint8_t id, uint16_t max);
void ng_progress_set(uint8_t id, uint16_t value);
void ng_progress_add(uint8_t id, uint16_t amount);
uint16_t ng_progress_value(uint8_t id);
uint8_t ng_progress_percent(uint8_t id);
uint8_t ng_progress_done(uint8_t id);
void ng_progress_update(void);
```

ng_properties

sdk/2d_engine/ng_properties.h — A grouped int32 property matrix for game-defined values.

```
void ng_prop_set(uint8_t group, uint8_t id, int32_t value);
int32_t ng_prop_get(uint8_t group, uint8_t id);
void ng_prop_add(uint8_t group, uint8_t id, int32_t amount);
void ng_prop_clear_group(uint8_t group);
```

ng_runtime

sdk/2d_engine/ng_runtime.h — Thin alias header — includes the whole engine.

Types and declarations only — see the header.

ng_scene

sdk/2d_engine/ng_scene.h — Scene entry/teardown helpers.

ng_scene_begin() is the "start from a known state" call: it clears hardware and engine state so nothing leaks in from the previous scene.

```
void ng_scene_clean_default(void);
void ng_scene_begin(uint8_t flags, uint8_t wait_vblank);
```

Constant	Value
NG_SCENE_CLEAN_FIX	0x01u
NG_SCENE_CLEAN_SPRITES	0x02u
NG_SCENE_CLEAN_CHARS	0x04u
NG_SCENE_CLEAN_PHYSICS	0x08u
NG_SCENE_CLEAN_EVENTS	0x10u
NG_SCENE_CLEAN_TIMERS	0x20u
NG_SCENE_CLEAN_PROGRESS	0x40u
NG_SCENE_CLEAN_PALETTEFX	0x80u
NG_SCENE_CLEAN_DEFAULT	(\

ng_status

sdk/2d_engine/ng_status.h — A bitset of "has this happened yet" flags.

```
void ng_status_set(uint16_t status_id);
void ng_status_clear(uint16_t status_id);
uint8_t ng_status_has(uint16_t status_id);
void ng_status_toggle(uint16_t status_id);
```

ng_timers

sdk/2d_engine/ng_timers.h — One-shot countdown timers, addressed by id.

```
void ng_timer_start(uint8_t id, uint16_t frames);
void ng_timer_stop(uint8_t id);
void ng_timers_update(void);
uint8_t ng_timer_active(uint8_t id);
uint8_t ng_timer_done(uint8_t id);
uint16_t ng_timer_value(uint8_t id);
uint8_t ng_timer_percent_left(uint8_t id);
```

Math

ng_fixed

sdk/2d_engine/ng_fixed.h — 16.16 fixed-point helpers, sin/cos and shrink lookup tables.

There is no floating point on this target. Everything that would have been a float is 16.16 fixed-point, and every trig or scale lookup is a table.

Types and declarations only — see the header.

Video

ng_art_asset

sdk/2d_engine/ng_art_asset.h — Type queries on generated artbox asset records.

```
uint8_t ng_art_asset_is_type(const NGArtAsset *asset, uint8_t type);
```

Constant	Value
NG_ART_TYPE_SCREEN	0u
NG_ART_TYPE_SPRITE	1u
NG_ART_TYPE_BACKGROUND	2u
NG_ART_TYPE_FIX	3u
NG_ART_TYPE_EFFECT	4u

ng_bg

sdk/2d_engine/ng_bg.h — Two scrolling sprite background layers with parallax factors.

```
void ng_bg_hide_all(void);
void ng_bg_mark_dirty(uint8_t layer);
```

Constant	Value
NG_BG_LAYERS	2

Constant	Value
NG_BG_LAYER0	0
NG_BG_LAYER1	1
NG_SCREEN_TABLE_MAX	200

ng_depthfx

sdk/2d_engine/ng_depthfx.h — Perspective projection, Z-to-shrink, fog palettes, starfields.

The Neo Geo can only shrink sprites, so depth is expressed as reduction plus a palette swap for fog banding.

Constant	Value
NG_DEPTH_CX	160
NG_DEPTH_CY	112
NG_DEPTH_PAL_NEAR	0
NG_DEPTH_PAL_MID	1
NG_DEPTH_PAL_FAR	2
NG_DEPTH_PAL_VERY_FAR	3

ng_fix

sdk/2d_engine/ng_fix.h — The FIX text layer, with a dirty-cell cache.

The layer is 40 x 32 cells of which 28 rows are visible; these helpers apply the row offset for you. Use `ng_fix_blank_cell()` to empty a cell — the BIOS font draws its space glyph as an opaque colour-2 plate.

```
void ng_fix_invalidate_all(void);
void ng_fix_clear(void);
void ng_fix_clear_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t pal);
void ng_fix_putc(uint8_t x, uint8_t y, char ch, uint8_t pal);
void ng_fix_puts(uint8_t x, uint8_t y, const char *text, uint8_t pal);
void ng_fix_put_u16(uint8_t x, uint8_t y, uint16_t value, uint8_t pal, uint16_t tile_offset);
void ng_fix_put_u32(uint8_t x, uint8_t y, uint32_t value, uint8_t pal, uint16_t tile_offset);
```

Constant	Value
NG_FIX_SAFE_X	1
NG_FIX_SAFE_Y	2
NG_FIX_SAFE_WIDTH	38
NG_FIX_SAFE_HEIGHT	28

ng_palette_assets

sdk/2d_engine/ng_palette_assets.h — Generated palette asset table lookup.

Types and declarations only — see the header.

ng_palette_fx

sdk/2d_engine/ng_palette_fx.h — Fade, flash, pulse and colour cycle on a palette bank.

Every effect is queue-safe: the palette upload goes through the render queue, so an effect started in game logic still lands inside VBlank.

```
void ng_palette_fx_update(void);
void ng_palette_load_bank(uint8_t palette_slot, const uint16_t *pal);
uint8_t ng_palette_load_asset(const NGPaletteAsset *assets, uint16_t count, uint16_t asset_id);
```

Constant	Value
NG_PALFX_MAX_SLOTS	8
NG_PALFX_NONE	0
NG_PALFX_FADE_IN	1
NG_PALFX_FADE_OUT	2
NG_PALFX_FLASH_WHITE	3
NG_PALFX_FLASH_RED	4
NG_PALFX_FLASH_BLUE	5
NG_PALFX_PULSE	6
NG_PALFX_CYCLE	7

ng_render_queue

sdk/2d_engine/ng_render_queue.h — A 128-slot deferred write queue, flushed inside VBlank.

The engine never writes VRAM from the game logic path. Modules post commands here and `ng_render_queue_flush()` drains them during the ~3 ms VBlank window. `ng_rq_free()` tells you how much room is left.

Constant	Value
NG_RQ_MAX_CMDS	128
NG_RQ_MAX_PAL_UPLOADS	8
NG_RQ_SPRITE_POS	0
NG_RQ_SPRITE_TILE	1
NG_RQ_SPRITE_PAL	2
NG_RQ_SPRITE_SHRINK	3
NG_RQ_SPRITE_HIDE	4
NG_RQ_SCB234	5

ng_sprite_group

sdk/2d_engine/ng_sprite_group.h — Dirty-flag sprite chains — write only what changed.

A group is a strip-0 anchor plus N chained strips welded together by the SCB3 sticky bit. Setting a property marks a dirty flag; the flush writes only the words that actually changed.

```

void ng_sprite_group_set_tile_base(NGSpriteGroup *g, uint16_t tileBase);
void ng_sprite_group_set_tile_stride(NGSpriteGroup *g, uint16_t tileStride);
void ng_sprite_group_set_palette(NGSpriteGroup *g, uint8_t palette);
void ng_sprite_group_set_active_rows(NGSpriteGroup *g, uint8_t activeRows);
void ng_sprite_group_set_pos(NGSpriteGroup *g, int16_t x, int16_t y);
void ng_sprite_group_move(NGSpriteGroup *g, int16_t dx, int16_t dy);
void ng_sprite_group_set_scale(NGSpriteGroup *g, uint8_t xScale, uint8_t yScale);
void ng_sprite_group_set_flip(NGSpriteGroup *g, uint8_t hflip, uint8_t vflip);
void ng_sprite_group_set_auto_anim(NGSpriteGroup *g, uint8_t autoAnim4, uint8_t autoAnim8);
void ng_sprite_group_set_visible(NGSpriteGroup *g, uint8_t visible);
void ng_sprite_group_upload(NGSpriteGroup *g);
void ng_sprite_group_update_transform(NGSpriteGroup *g);
void ng_sprite_group_hide(NGSpriteGroup *g);
void ng_engine_init_hardware(uint16_t transparentTile);
void ng_sprite_hide_vram_base(uint16_t spriteBase, uint16_t count);
void ng_sprite_hide_all(void);
void ng_sprite_disable_hw_range(uint16_t first, uint16_t count);
void ng_sprite_park_off(uint16_t spr);
void ng_sprite_park_off_range(uint16_t first, uint16_t count);

```

Scale bytes: the two axes are not the same width

`ng_sprite_group_set_scale()` and `NGCharacter.scale_x / .scale_y` take a byte per axis, but the hardware reads them differently:

axis	field	drawn fraction
X	top nibble only	$((\text{value} \gg 4) + 1) / 16$
Y	whole byte	$(\text{value} + 1) / 256$

X has sixteen steps, Y has 256. The two only land on the same fraction when the low nibble is F. Passing the same byte to both axes — the obvious thing to do — therefore does **not** give a square scale unless you picked the value carefully:

```

ng_sprite_group_set_scale(&g, 0x80u, 0x80u); /* 56.25% wide, 50.4% tall */
ng_sprite_group_set_scale(&g, 0x8Fu, 0x8Fu); /* 56.25% both – correct */

```

A byte written as `0xN0` leaves the sprite up to 12% shorter than it is wide. Use `NG_SCALE(n)` from `ng_defs.h`, which takes the size in sixteenths and builds a byte whose axes match:

```

ng_sprite_group_set_scale(&g, NG_SCALE(8), NG_SCALE(8)); /* half size */
c->scale_x = c->scale_y = NG_SCALE(5); /* 5/16 */

```

Code that positions artwork by scaling an offset ($\text{offset} * \text{scale} / 256$) is assuming the Y rule, so it is only correct for `0xNF` values as well.

Shrinking a sprite changes how many characters it is

SCB3 carries the number of **active characters**, and that field is the sprite's height on screen: the hardware covers exactly $\text{rows} * 16$ scanlines with it and never consults the shrink register to decide otherwise. Vertical shrink only chooses which source row of the SCB1 map each of those scanlines reads.

So the count is not a property of the artwork, it is a property of the artwork *at the size you are drawing it*. A sixteen-character image drawn at half height is eight active characters. Leave it at sixteen and the picture finishes half way down a window built for twice its height, and the rest of the window repeats it.

Both engines derive this for you — `ng_sprite_group_set_scale()` and the character scale fields feed it — so you never write the count yourself. What you do have to know is the consequence for the tilemap: because the row lookup can land past the last row of real art, **every map row it can reach must hold the transparent tile**, or the sprite grows a band of whatever the previous frame left in VRAM. `ng_sprite_group_upload()` and `ng_sprite_group_flush()` blank those rows.

That reach is bounded, which is what keeps the cost down. The lookup reads a source row inside the first sixteen, and reaches rows 16..31 only through the mirror it applies once a sprite spans more than 256 scanlines. Under seventeen active characters that mirror never engages, so the upload stops at sixteen rows — half the VRAM traffic of blanking all thirty-two.

One practical consequence for scrolling: a sprite is a whole number of characters tall, so a backdrop cropped to a window whose edge is not on a character boundary always overhangs it, by up to 15 px. The overhang is not optional; only its destination is. Either run the backdrop edge to edge so the overhang leaves the screen, or cover it with the FIX layer, which draws in front of every sprite.

Shrink is not a substitute for importing at the right size

`ng_sprite_group_set_scale()` and the character scale fields shrink in hardware, and the hardware shrink drops rows and columns rather than resampling them. What it drops is a dither the art pipeline chose for the pixels it expected to reach the screen, so a sprite drawn at half its imported size loses half of that pattern and one drawn at a quarter loses the shape with it.

Use the shrink for what it is good at — an effect that changes over time, depth, a boss growing as it approaches — and import static art at the size it is actually drawn. See "Import size" in the artbox pipeline reference.

Uploading versus flushing

`ng_sprite_group_upload()` rewrites everything, including the tilemap — one write per tile per strip. `ng_sprite_group_flush()` writes only what the dirty flags say changed, and a group that merely moved costs **two words in total**, not two per strip: the hardware reads no position from a chained strip, so a move touches the driver strip alone. A show has to write them all, because parking a slot clears its chain bit.

Upload once when a group is built; flush every frame after that. Calling `upload()` per frame for a scrolling backdrop is enough VRAM traffic to overrun vblank, and the sprite writes that follow it then land during active display, which shows up as tearing and flicker on everything drawn afterwards — not just on the background.

Constant	Value
NG_SGF_DIRTY_POS	0x01
NG_SGF_DIRTY_TILE	0x02
NG_SGF_DIRTY_PALETTE	0x04
NG_SGF_DIRTY_SHRINK	0x08
NG_SGF_DIRTY_VIS	0x10
NG_SGF_DIRTY_ALL	0x1F
NG_SPRITE_DISABLED_YREG	256u
NG_SPRITE_DISABLED_X	496u
NG_SPRITE_DISABLED_SCB3	((uint16_t)(NG_SPRITE_DISABLED_YREG << 7))
NG_SPRITE_BLANK_TILE	0xFFFFu

Constant	Value
NG_SPRITE_BLANK_ATTR	0x0000u

ng_sprite_pool

sdk/2d_engine/ng_sprite_pool.h — The hardware slot map. Constants only, no functions.

HIGHER slot number is drawn IN FRONT. This is the observed hardware direction and it is the opposite of what earlier revisions of this header claimed. Backgrounds belong at low slots, foreground effects at high ones. The legacy names NG_SPR_BG0_FIRST / NG_SPR_BG1_FIRST predate the correction and are kept only for existing code.

Constant	Value
NG_SPR_TOTAL	380
NG_SPR_LAST	379
NG_SPR_TITLE_FIRST	33
NG_SPR_TITLE_LAST	95
NG_SPR_CHAR_FIRST	96
NG_SPR_CHAR_LAST	223
NG_SPR_FX_FIRST	224
NG_SPR_FX_LAST	255
NG_SPR_PART_FIRST	256
NG_SPR_PART_LAST	287
NG_SPR_TEMP_FIRST	288
NG_SPR_TEMP_LAST	379
NG_SPR_BG0_FIRST	1
NG_SPR_BG1_FIRST	17
NG_SPR_BG_STRIP	16

ng_sprite_window

sdk/2d_engine/ng_sprite_window.h — Tracks the strips a variable-size sprite currently occupies.

When a sprite shrinks between frames the strips it no longer uses have to be actively hidden, or the old art stays on screen. That bookkeeping is what a window does.

```
void ng_sprite_window_reset(NGSpriteWindow *window);
void ng_sprite_window_set_current(NGSpriteWindow *window, uint8_t current_strips);
void ng_sprite_window_set_shape(NGSpriteWindow *window, uint8_t current_strips, uint8_t current_rows);
void ng_sprite_window_clear(NGSpriteWindow *window);
void ng_sprite_window_clear_tail(NGSpriteWindow *window);
void ng_sprite_window_hide(NGSpriteWindow *window);
```

ng_vram

sdk/2d_engine/ng_vram.h — Direct VRAM sprite-slot clearing helpers.

```
uint16_t ng_vram_scb1_to_sprite_slot(uint16_t scb1_base);
void ng_vram_clear_sprite_slot(uint16_t sprite_slot);
void ng_vram_clear_sprite_range(uint16_t first_sprite, uint16_t count);
void ng_vram_clear_sprite_vram_base(uint16_t sprite_base, uint16_t count);
void ng_vram_clear_all_sprites(void);
```

Characters

ng_actions

sdk/2d_engine/ng_actions.h — Table-driven action scripts with sound and FX hooks.

```
void ng_actions_register(uint16_t action_id, const NGAActionCmd *script);
void ng_actions_set_sound_hooks(NGActionSfxHook sfx_hook, NGAActionMusicHook music_hook);
void ng_actions_set_fx_hook(NGActionFxHook fx_hook);
void ng_char_action(struct NGCharacter *c, uint16_t action_id);
void ng_actions_update(struct NGCharacter *c);
```

ng_border_constraints

sdk/2d_engine/ng_border_constraints.h — Keep characters inside declared screen regions.

```
void ng_border_constraints_load(NGBorderConstraint *list, uint16_t count);
void ng_border_constraints_update(void);
void ng_border_constraints_reset_used(void);
```

ng_chars

sdk/2d_engine/ng_chars.h — The character pool: 64 objects, movement, animation, hitboxes, damage.

ng_chars_draw() hands out hardware strips from NG_SPR_CHAR_FIRST upward. Two things catch people: ng_chars_count() returns the number of ACTIVE objects, not a high-water index, so it is not a safe array bound — iterate NG_MAX_CHARS and skip inactive slots. And a character is removed with ng_chars_remove(); setting a "destroy pending" life state is not acted on by the engine.

```

NGCharacter* chars_add(uint8_t kind, int16_t x, int16_t y);
void ng_chars_remove(NGCharacter *c);
void ng_chars_clear_kind(uint8_t kind);
NGCharacter* chars_find(uint8_t kind);
NGCharacter* chars_at(uint8_t index);
uint8_t ng_chars_count(void);
uint8_t ng_chars_index(NGCharacter *c);
void ng_chars_set_game_interrupt(uint8_t kind, NGCharInterrupt fn);
void ng_chars_reset_slot(uint8_t index);
void ng_chars_begin_scene_arena(uint8_t arena_id);
void ng_chars_clear_arena(uint8_t arena_id);
void ng_chars_set_default_arena(uint8_t arena_id);
void ng_chars_set_fixed_step(uint8_t updates_per_frame);
void ng_chars_update_fixed(void);
void ng_chars_defrag_slots(void);
void ng_chars_update(void);
void ng_chars_draw(void);
void ng_char_set_asset_bounds(NGCharacter *c, uint16_t tileStart, uint16_t tileEnd);
uint8_t ng_char_bind_asset(NGCharacter *c, const NGSpriteAssetView *asset);
void ng_char_set_cull_margin(NGCharacter *c, int16_t l, int16_t r, int16_t t, int16_t b);
void ng_char_set_anim_clip(NGCharacter *c, const NGAAnimClip *clip);
void ng_char_anim_update(NGCharacter *c);
uint8_t ng_char_validate_asset_window(uint16_t tileBase, uint8_t strips, uint8_t rows, uint16_t stride,
    uint16_t tileStart, uint16_t tileEnd);
uint8_t ng_palette_claim(uint8_t palette_slot, uint8_t owner_kind);
void ng_palette_release(uint8_t palette_slot, uint8_t owner_kind);
void ng_char_set_tile_stride(NGCharacter *c, uint16_t stride);
void ng_char_set_body(NGCharacter *c, int16_t x, int16_t y, int16_t w, int16_t h);
void ng_char_set_pos(NGCharacter *c, int16_t x, int16_t y);
void ng_char_set_speed(NGCharacter *c, int16_t vx_px, int16_t vy_px);
void ng_char_set_speed_fp(NGCharacter *c, int32_t vx_fp, int32_t vy_fp);
void ng_char_add_speed_fp(NGCharacter *c, int32_t ax_fp, int32_t ay_fp);
void ng_char_set_priority(NGCharacter *c, uint8_t priority_band, int16_t depth_offset);
void ng_char_damage(NGCharacter *c, uint8_t amount);
void ng_char_heal(NGCharacter *c, uint8_t amount);
NGRect ng_char_body_rect(NGCharacter *c);
NGRect ng_char_hit_rect(NGCharacter *c);

```

Constant	Value
NG_CHAR_LIFE_FREE	0u
NG_CHAR_LIFE_ALLOCATED	1u
NG_CHAR_LIFE_VISIBLE	2u
NG_CHAR_LIFE_HIDDEN	3u
NG_CHAR_LIFE_DESTROY_PENDING	4u
NG_STRICT_RENDER_VALIDATE	1

ng_joystick

sdk/2d_engine/ng_joystick.h — Edge-detected input, repeat, motion specials, and character control.

Poll once per frame with `ng_joystick_update()`, then read edges with `ng_joy_pressed()`. Acting on the raw held state instead fires every frame the button is down.

```

void ng_joystick_update(void);
const NGJoystickState *ng_joystick_state(void);
uint16_t ng_joy_pressed(void);
uint16_t ng_joy_released(void);
uint16_t ng_joy_repeat(void);
uint8_t ng_joy_direction(void);
uint8_t ng_joy_held_frames(uint16_t mask);
uint8_t ng_joy_special_dp(uint8_t facing_left, uint16_t attack_button_mask);
void ng_joy_apply_hitbox(NGCharacter *c, uint8_t active_frames, int16_t x, int16_t y, int16_t w, int16_t h);
void ng_joy_set_event_handler(NGJoyEventHandler fn);
uint8_t ng_joy_pop_command(NGJoyCommand *out);

```

Constant	Value
NG_EVENT_JOY_JUMP	0x5001
NG_EVENT_JOY_FIRE_LIGHT	0x5002
NG_EVENT_JOY_FIRE_HEAVY	0x5003
NG_EVENT_JOY_HIT	0x5004
NG_EVENT_JOY_SPECIAL_QCF	0x5005
NG_EVENT_JOY_SPECIAL_DP	0x5006
NG_EVENT_JOY_COMMAND	0x5007

ng_npcs

sdk/2d_engine/ng_npcs.h — NPC wrappers: patrol bounds, home position, think callbacks.

```

void ng_npc_remove(NGNpc *npc);
uint8_t ng_npc_count(void);
void ng_npc_set_think(NGNpc *npc, NGNpcThink think, uint16_t interval_frames);
void ng_npc_set_home(NGNpc *npc, int16_t x, int16_t y);
void ng_npc_set_patrol_bounds(NGNpc *npc, int16_t min_x, int16_t max_x, int16_t min_y, int16_t max_y);
void ng_npcs_update(void);
void ng_npc_think_hover(NGNpc *npc, NGCharacter *c);

```

Constant	Value
NG_NPC_FLAG_PATROL_X	0x0001
NG_NPC_FLAG_PATROL_Y	0x0002
NG_NPC_FLAG_FACE_MOTION	0x0004

ng_physics

sdk/2d_engine/ng_physics.h — Gravity, drag, solid rectangles, and grounded tests.

```

void ng_physics_attach(NGCharacter *c, uint16_t flags);
void ng_physics_detach(NGCharacter *c);
void ng_physics_set_gravity(NGCharacter *c, int32_t gravity_fp, int32_t max_fall_fp);
void ng_physics_set_drag(NGCharacter *c, int32_t drag_x_fp, int32_t drag_y_fp);
void ng_physics_clear_solids(void);
uint8_t ng_physics_add_solid(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t flags);
void ng_physics_update_pre(void);
void ng_physics_resolve(void);
uint8_t ng_physics_is_grounded(NGCharacter *c);

```

Constant	Value
NG_PHYSICS_GRAVITY	0x0001
NG_PHYSICS_WORLD	0x0002
NG_PHYSICS_SOLIDS	0x0004
NG_PHYSICS_DRAG_X	0x0008
NG_PHYSICS_DRAG_Y	0x0010

World

ng_camera

sdk/2d_engine/ng_camera.h — Follow, dead zone, look-ahead, shake, cinematic pan, bounds clamp.

Constant	Value
NG_SCREEN_W	320
NG_CAM_FREE	0x00
NG_CAM_FOLLOW	0x01
NG_CAM_CINEMATIC	0x02

ng_level

sdk/2d_engine/ng_level.h — Level state, world bounds, scroll and camera helpers.

```
void ng_level_set_id(uint8_t level_id);
void ng_level_set_mode(uint8_t mode);
void ng_level_set_background(uint16_t screen_id);
void ng_level_set_overlay(uint16_t screen_id);
void ng_level_set_backdrop(uint16_t color);
void ng_level_set_fix_palette(uint8_t palette);
void ng_level_set_scroll(int16_t x, int16_t y);
void ng_level_move_scroll(int16_t dx, int16_t dy);
void ng_level_set_world_bounds(int16_t left, int16_t top, int16_t right, int16_t bottom);
void ng_level_set_camera(int16_t x, int16_t y, int16_t screen_w, int16_t screen_h);
void ng_level_move_camera(int16_t dx, int16_t dy, int16_t screen_w, int16_t screen_h);
void ng_level_camera_follow(int16_t target_x, int16_t target_y, int16_t screen_w, int16_t screen_h);
void ng_level_camera_input(uint16_t joystick, int16_t speed, uint8_t axes, int16_t screen_w, int16_t screen_h);
void ng_level_camera_joystick(int16_t speed, uint8_t axes, int16_t screen_w, int16_t screen_h);
void ng_level_update(void);
```

Constant	Value	
NG_CAMERA_AXIS_X	0x01	
NG_CAMERA_AXIS_Y	0x02	
NG_CAMERA_AXIS_BOTH	`(NG_CAMERA_AXIS_X	NG_CAMERA_AXIS_Y)`

Effects

ng_debug

sdk/2d_engine/ng_debug.h — A FIX-layer performance overlay.

Compile with -DNG_DEBUG_PERF=1 to enable.

```
void ng_debug_draw(void);
void ng_debug_clear(void);
```

Constant	Value
NG_DBG_X	22
NG_DBG_Y_START	2
NG_DBG_PAL	4
NG_DBG_SUPPRESS_UNUSED	\

ng_demo_advanced

sdk/2d_engine/ng_demo_advanced.h — Prebuilt showcase routines used by the demo ROM.

```
void demo_smooth_camera(void);
void demo_palette_flash_cycle(void);
void demo_hitstop_and_shake(void);
void demo_particle_stress(void);
void demo_depth_starfield(void);
void demo_boss_depth_shrink(void);
void demo_perf_hud(void);
```

ng_feedback

sdk/2d_engine/ng_feedback.h — Hitstop, shake, palette flash and a sound hook, in one call.

ng_impact_event() is the whole game-feel package for a hit. Prefer it over assembling the four effects by hand so intensity stays consistent.

```
void ng_feedback_flash_red(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
uint8_t ng_feedback_is_slow_motion(void);
```

Constant	Value
NG_FEEDBACK_MAX_EVENTS	4
NG_IMPACT_LIGHT	0
NG_IMPACT_MEDIUM	1
NG_IMPACT_HEAVY	2
NG_IMPACT_BOSS	3

ng_particles

sdk/2d_engine/ng_particles.h — A 32-slot fixed pool with typed spawns and priority eviction.

The pool never grows and never allocates. When it is full a new spawn evicts the lowest-priority live particle, so a flood of dust cannot push out an explosion.

```
NGParticle * ng_spawn_dust(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_slash_trail(int16_t x, int16_t y, int8_t dir, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_explosion(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_smoke(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_magic_spark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
uint8_t ng_particles_count_priority(uint8_t priority);
```

Constant	Value
NG_PART_MAX_PARTICLES	32
NG_PART_PRI_CRITICAL	0
NG_PART_PRI_NORMAL	1
NG_PART_PRI_OPTIONAL	2
NG_PART_DUST	0
NG_PART_HIT_SPARK	1
NG_PART_SLASH_TRAIL	2
NG_PART_EXPLOSION	3
NG_PART_MAGIC_SPARK	4
NG_PART_SMOKE	5
NG_PART_WATER_SPLASH	6
NG_PART_METAL_HIT	7
NG_PART_ANIM_FRAMES	4

PART 18

2D Engine Reference — C++

The C++14 build of the same engine: classes, methods, and how it relates to the C API.

`docs/API_2D_ENGINE_CPP.md`

****Eagle Software · Neo Geo SDK v1.7.0 · sdk/2d_engine_plus/****

The C++14 build of the 2D engine. This is not a different engine — it is the same design with the same public ABI, so a C game can link against this build and a C++ game can link against the C one.

```
make p1 USE_2D_PLUS=1      # link this engine
make p1                    # link the C engine
```

```
#include "sdk/2d_engine_plus/ng_engine_plus.hpp"
```

What this build adds

Every function in `API_2D_ENGINE_C.md` exists here unchanged, declared extern "C". On top of them this build adds member methods and singletons, so the same work reads as `cam.follow(...)` and `CharManager::instance()` instead of a free function plus an explicit pointer. The wrappers are inline and compile to the same hardware writes — there is no runtime cost and no vtable.

Compiled under `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics -ffreestanding -march=68000`. No standard library, no malloc, no floating point, and no global constructors that depend on initialisation order.

Choosing between the builds

Build	Choose it when
<code>sdk/2d_engine</code>	You are writing C. Smallest code-size footprint and the simplest call chain.
<code>sdk/2d_engine_plus</code>	You are writing C++. You want member methods, singletons, and the scene-stack helpers.

One engine per ROM. `games/demo_plus` (id 778) is the smoke-test target that links only against this build.

Classes and methods

Extracted from the headers by `tools/gen_api_reference.py`. Types with no methods are plain data structures shared with the C build.

ng_actions.hpp

Table-driven action scripts with sound and FX hooks.

****ActionRegistry****

```
static ActionRegistry& instance();
void NEOGEO_USER init();
void NEOGEO_USER registerScript(uint16_t action_id, const NGAActionCmd *script);
const NGAActionCmd* NEOGEO_USER get(uint16_t action_id);
void NEOGEO_USER setSoundHooks(NGActionSfxHook sfx, NGAActionMusicHook music);
void NEOGEO_USER setFxHook(NGActionFxHook fx);
void NEOGEO_USER update(struct NGCharacter *c);
NGActionSfxHook sfxHook();
NGActionMusicHook musicHook();
NGActionFxHook fxHook();
```

ng_camera.hpp

Follow, dead zone, look-ahead, shake, cinematic pan, bounds clamp.

****NGCamera****

```
void NEOGEO_USER init();
void NEOGEO_USER setBounds(int16_t left, int16_t top, int16_t right, int16_t bottom);
void NEOGEO_USER setFollowSpeed(uint8_t speed);
void NEOGEO_USER setDeadZone(uint8_t half_w, uint8_t half_h);
void NEOGEO_USER setLookAhead(int16_t max_x, int16_t max_y, uint8_t rate);
void NEOGEO_USER snap(int16_t x, int16_t y);
void NEOGEO_USER shake(uint8_t amp, uint8_t frames);
void NEOGEO_USER panTo(int16_t dest_x, int16_t dest_y, uint8_t speed);
void NEOGEO_USER update(int16_t target_x, int16_t target_y, int16_t target_vx);
void NEOGEO_USER apply(int16_t target_x, int16_t target_y, int16_t target_vx);
int16_t NEOGEO_USER worldToScreenX(int16_t world_x);
int16_t NEOGEO_USER worldToScreenY(int16_t world_y);
```

ng_chars.hpp

The character pool: 64 objects, movement, animation, hitboxes, damage.

****NGCharacter****

```
void NEOGEO_USER setPos(int16_t x, int16_t y);
void NEOGEO_USER setSpeed(int16_t vx_px, int16_t vy_px);
void NEOGEO_USER setSpeedFp(int32_t vx_fp, int32_t vy_fp);
void NEOGEO_USER addSpeedFp(int32_t ax_fp, int32_t ay_fp);
void NEOGEO_USER setSprite(uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uint16_t tileBase, uint8_t palette);
void NEOGEO_USER setAssetBounds(uint16_t tileStart, uint16_t tileEnd);
uint8_t NEOGEO_USER bindAsset(const NGSpriteAssetView *asset);
void NEOGEO_USER setCullMargin(int16_t l, int16_t r, int16_t t, int16_t b);
void NEOGEO_USER setAnimClip(const NGAnimClip *clip);
void NEOGEO_USER animUpdate();
void NEOGEO_USER setTileStride(uint16_t stride);
void NEOGEO_USER setBody(int16_t x, int16_t y, int16_t w, int16_t h);
void NEOGEO_USER setPriority(uint8_t band, int16_t offset);
void NEOGEO_USER damage(uint8_t amount);
void NEOGEO_USER heal(uint8_t amount);
NGRect NEOGEO_USER bodyRect();
NGRect NEOGEO_USER hitRect();
```

****CharManager****

```
static CharManager& instance();
void NEOGEO_USER init();
NGCharacter* NEOGEO_USER add(uint8_t kind, int16_t x, int16_t y);
void NEOGEO_USER remove(NGCharacter *c);
void NEOGEO_USER clearKind(uint8_t kind);
void NEOGEO_USER resetSlot(uint8_t index);
void NEOGEO_USER clearArena(uint8_t arena_id);
void NEOGEO_USER setDefaultArena(uint8_t arena_id);
void NEOGEO_USER setFixedStep(uint8_t updates_per_frame);
void NEOGEO_USER updateFixed();
void NEOGEO_USER defragSlots();
NGCharacter* NEOGEO_USER find(uint8_t kind);
NGCharacter* NEOGEO_USER at(uint8_t index);
uint8_t NEOGEO_USER count();
uint8_t NEOGEO_USER indexOf(const NGCharacter *c);
void NEOGEO_USER setInterrupt(uint8_t kind, NGCharInterrupt fn);
void NEOGEO_USER update();
void NEOGEO_USER draw();
void NEOGEO_USER clearUploadSlot(uint8_t index);
```

ng_feedback.hpp

Hitstop, shake, palette flash and a sound hook, in one call.

****FeedbackSystem****

```
static FeedbackSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER setSfxHook(NGFeedbackSfxHook hook);
void NEOGEO_USER update();
void NEOGEO_USER hitstop(uint8_t frames);
uint8_t NEOGEO_USER isHitstop();
uint8_t NEOGEO_USER hitstopRemaining();
void NEOGEO_USER shake(NGCamera *cam, uint8_t amp, uint8_t frames);
void NEOGEO_USER flashWhite(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void NEOGEO_USER flashRed(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void NEOGEO_USER slowMotion(uint8_t frames);
uint8_t NEOGEO_USER isSlowMotion();
uint8_t NEOGEO_USER slowMotionRemaining();
void NEOGEO_USER impactEvent(uint8_t impact_kind, uint8_t palette_slot, const uint16_t *base_pal, NGCamera *cam, uint16_t sfx_id, int16_t spark_x, int16_t spark_y, uint16_t spark_tile, uint8_t spark_pal);
```

ng_npcs.hpp

NPC wrappers: patrol bounds, home position, think callbacks.

****NpcManager****

```
static NpcManager& instance();
void NEOGEO_USER init();
NGNpc* NEOGEO_USER spawn(uint8_t npc_kind, uint8_t char_kind, int16_t x, int16_t y);
void NEOGEO_USER remove(NGNpc *npc);
NGNpc* NEOGEO_USER at(uint8_t index);
NGNpc* NEOGEO_USER find(uint8_t npc_kind);
NGCharacter* NEOGEO_USER charOf(NGNpc *npc);
uint8_t NEOGEO_USER count();
void NEOGEO_USER setThink(NGNpc *npc, NGNpcThink think, uint16_t interval_frames);
void NEOGEO_USER setHome(NGNpc *npc, int16_t x, int16_t y);
void NEOGEO_USER setPatrolBounds(NGNpc *npc, int16_t min_x, int16_t max_x, int16_t min_y, int16_t max_y);
void NEOGEO_USER update();
static void NEOGEO_USER thinkPatrol(NGNpc *npc, NGCharacter *c);
static void NEOGEO_USER thinkHover(NGNpc *npc, NGCharacter *c);
```

ng_palette_fx.hpp

Fade, flash, pulse and colour cycle on a palette bank.

****PaletteFxSystem****

```
static PaletteFxSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER update();
void NEOGEO_USER uploadBase(uint8_t palette_slot, const uint16_t *pal);
void NEOGEO_USER fadeIn(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER fadeOut(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashWhite(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashRed(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashBlue(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER pulse(uint8_t palette_slot, const uint16_t *base_pal, uint8_t period);
void NEOGEO_USER cycle(uint8_t palette_slot, const uint16_t *base_pal, uint8_t start, uint8_t end);
void NEOGEO_USER stop(uint8_t palette_slot);
uint8_t NEOGEO_USER active(uint8_t palette_slot);
```

ng_particles.hpp

A 32-slot fixed pool with typed spawns and priority eviction.

****ParticleSystem****

```
static ParticleSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER update();
uint16_t NEOGEO_USER draw(uint16_t first_slot, uint16_t sprite_budget_used);
NGParticle* NEOGEO_USER spawn(uint8_t type, uint8_t priority, int16_t x, int16_t y, int32_t vx_fp, int32_t vy_fp, uint8_t lifetime, uint16_t tile_base, uint8_t palette, uint8_t frame_count, uint8_t anim_period);
NGParticle* NEOGEO_USER spawnHitSpark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnDust(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnSlashTrail(int16_t x, int16_t y, int8_t dir, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnExplosion(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnSmoke(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnMagicSpark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
uint8_t NEOGEO_USER count();
uint8_t NEOGEO_USER countPriority(uint8_t priority);
```

ng_physics.hpp

Gravity, drag, solid rectangles, and grounded tests.

****PhysicsWorld****

```
static PhysicsWorld& instance();
void NEOGEO_USER init();
void NEOGEO_USER attach(NGCharacter *c, uint16_t flags);
void NEOGEO_USER detach(NGCharacter *c);
NGPhysicsBody* NEOGEO_USER body(NGCharacter *c);
void NEOGEO_USER setGravity(NGCharacter *c, int32_t gravity_fp, int32_t max_fall_fp);
void NEOGEO_USER setDrag(NGCharacter *c, int32_t drag_x_fp, int32_t drag_y_fp);
void NEOGEO_USER clearSolids();
uint8_t NEOGEO_USER addSolid(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t flags);
void NEOGEO_USER updatePre();
void NEOGEO_USER resolve();
uint8_t NEOGEO_USER isGrounded(NGCharacter *c);
```

ng_sprite_group.hpp

Dirty-flag sprite chains — write only what changed.

****NGSpriteGroup****

```

void NEOGEO_USER init(uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uint16_t tileBase, uint8_t palette);
void NEOGEO_USER markDirty(uint8_t flags);
void NEOGEO_USER flush();
void NEOGEO_USER setTileBase(uint16_t tileBase);
void NEOGEO_USER setTileStride(uint16_t tileStride);
void NEOGEO_USER setPalette(uint8_t palette);
void NEOGEO_USER setActiveRows(uint8_t rows);
void NEOGEO_USER setPos(int16_t x, int16_t y);
void NEOGEO_USER move(int16_t dx, int16_t dy);
void NEOGEO_USER setScale(uint8_t xScale, uint8_t yScale);
void NEOGEO_USER setFlip(uint8_t hflip, uint8_t vflip);
void NEOGEO_USER setAutoAnim(uint8_t aa4, uint8_t aa8);
void NEOGEO_USER setVisible(uint8_t v);
void NEOGEO_USER upload();
void NEOGEO_USER updateTransform();
void NEOGEO_USER hide();
static void NEOGEO_USER hideRange(uint16_t firstSprite, uint16_t count);
static void NEOGEO_USER hideVramBase(uint16_t spriteBase, uint16_t count);
static void NEOGEO_USER hideAll();
static void NEOGEO_USER initHardware(uint16_t transparentTile);

```

ng_sprite_window.hpp

Tracks the strips a variable-size sprite currently occupies.

****NGSpriteWindow****

```

void NEOGEO_USER init(uint8_t owner, uint16_t first_slot, uint8_t max_strips);
void NEOGEO_USER reset();
void NEOGEO_USER setCurrent(uint8_t current_strips);
void NEOGEO_USER setShape(uint8_t current_strips, uint8_t current_rows);
void NEOGEO_USER clear();
void NEOGEO_USER clearTail();
void NEOGEO_USER hide();

```

Free functions

The extern "C" surface is identical to the C build. See `API_2D_ENGINE_C.md` for the full list; the module layout, constants, and semantics all carry over unchanged.

PART 19

Deluxe 2D Module Guides

Camera, depth FX, palette FX, particles, the render queue, sprite groups, and the performance rules that govern all of them.

`docs/DELUXE_MODULES.md`

Seven short guides, one per deluxe engine module added in v1.7.0, each with the problem it solves, its key parameters, a worked example, and the mistakes people actually make with it. The API reference sections list every call; these guides explain why the call exists.

Camera System

What It Does

NGCamera (Stage 5) provides smooth-follow, look-ahead, dead zone, camera shake, and cinematic pan on top of the existing NGLevelState scroll.

Why It Matters on NeoGeo

The NeoGeo has no hardware scroll register for all sprites. "Scrolling" is implemented by adjusting SCB4 (X position) and SCB3 (Y position) for every sprite group and background layer. Abrupt per-frame jumps look jarring. The camera applies sub-pixel interpolation so motion is perceptually smooth at 60 fps even when the player is accelerating.

Hardware Limitation It Solves

- Sub-pixel position storage (16.16 fixed-point) prevents the "1-pixel snap" stutter that integer-only camera causes at slow scroll speeds.
- Shake pattern uses a pre-baked 8-entry array — no random calls, no division.

Key Parameters

Parameter	Type	Description
follow_speed	uint8_t	1=very slow, 255=instant snap, 64=smooth arcade feel
dead_zone_x/y	uint8_t	pixels target can move before camera follows
look_ahead_x	int16_t	horizontal look-ahead in pixels
shake_amp	uint8_t	max shake amplitude in pixels (2-4 recommended)

Example

```

NGCamera cam;
ng_camera_init(&cam);
ng_camera_set_bounds(&cam, 0, 0, 2047, 224);
ng_camera_set_follow_speed(&cam, 0x30);
ng_camera_set_dead_zone(&cam, 32, 16);

// Per frame:
ng_camera_apply(&cam, player->x, player->y, player->vx_fp >> NG_FP_SHIFT);

```

Common Mistakes

- Forgetting to call `ng_camera_set_bounds()` after `ng_level_set_world_bounds()`: the camera will not clamp to world edges.
- Setting `follow_speed = 255` (instant): looks like no camera system at all. Use 32..80 for arcade smoothness.
- Shake amplitude > 6 pixels: creates nausea on a CRT. Keep it 2..4.

Performance Advice

- One NGCamera per scene. No need for multiple camera instances.
- `ng_camera_apply()` calls `ng_level_set_scroll()` internally — no extra work.
- Camera shake costs zero at rest (just a conditional on `shake_frames`).

Depth FX

What It Does

`ng_depthfx` (Stage 10) projects 3D (x, y, z) world positions onto the NeoGeo screen with perspective-correct scaling using the SCB2 shrink register. Z distance selects both the sprite's apparent size and its fog palette.

Why It Matters on NeoGeo

NeoGeo sprites have a hardware shrink register (SCB2) that scales sprites down from full size (0xFF vertical, 0x0F horizontal nibble) to tiny. By driving this register with a Z-distance lookup table, you get convincing depth scaling for flying objects, bosses approaching, background elements at varying depth.

Hardware Limitation It Solves

- Avoids division for projection: uses `ng_shrink_tab[]` lookup table.
- SCB2 horizontal shrink is a 4-bit nibble (0..15); SCB2 vertical is 8-bit (0..255). The projection maps both from a single Z value.
- Fog palette selection: 4 depth bands map to palette slot offsets.

Coordinate System

```
X: left-right,  positive = right
Y: up-down,    positive = down (screen space)
Z: depth,      0 = camera plane, 127 = far clip
```

Vanishing point default: screen centre (160, 112).

Example

```
// At scene init:
ng_depthfx_init();
ng_depthfx_set_fog_palettes(0, 0, 1, 2); // near=+0, far=+2 palette offset

// Per frame:
NGVec3 boss_pos = { 160, 112, 40 }; // Z=40 = mid-distance
ng_depthfx_draw_group(&boss_group, boss_pos, PAL_BOSS);
```

Shrink Table

`ng_shrink_tab[z]` maps `Z=0..127` to shrink value `0xFF..0x20`.

- `Z=0`: 0xFF (full size)
- `Z=64`: ~0x7F (half size)
- `Z=127`: 0x20 (minimum visible)

Common Mistakes

- Using `Z > 127`: clamps to 127 automatically but consider culling farther objects.

- Forgetting fog palettes: at default, all Z depths use the same palette. Call `ng_depthfx_set_fog_palettes()` to enable distance fog.

Performance Advice

- Two multiplies per projected object (`screen_x`, `screen_y`) = fast on 68000.
- The shrink table lookup is a single array read — $O(1)$.
- Cull objects with `proj.visible == 0` before calling `ng_sprite_group_update_transform()`.

Palette FX

What It Does

`ng_palette_fx` (Stage 7) manages animated palette effects: fade in/out, white flash, red/blue tint flash, brightness pulse, and colour cycle.

All effects modify a 16-colour work palette in RAM and upload it via the render queue — no direct palette RAM writes during game logic.

Why It Matters on NeoGeo

NeoGeo palette RAM at 0x400000 is writable any time, but writing mid-frame can cause visible colour tears on sprites being rendered on that scanline. The queue defers all palette writes to VBlank for tear-free colour changes.

Hardware Limitation It Solves

- Safe palette upload timing: all writes happen via `ng_render_queue_flush()`.
- Multiple simultaneous effects on up to `NG_PALFX_MAX_SLOTS = 8` palette slots.
- No blending hardware — effects are CPU-computed once per frame.

NeoGeo Colour Format

```
Bit 15: dark flag (halves all channels when set)
Bits 14-10: red (0..31)
Bits 9-5:   green (0..31)
Bits 4-0:   blue (0..31)
Value 0x8000 = transparent (colour index 0)
```

Example

```
// At scene load:
ng_palfx_upload_base(PAL_PLAYER, player_palette);

// On hit:
ng_palfx_flash_white(PAL_PLAYER, player_palette, 6);

// Per frame:
ng_palette_fx_update();           // computes work palette, queues upload
ng_render_queue_flush();          // applies at VBlank
```

Available Effects

Function	Description
<code>ng_palfx_fade_in</code>	Black → base palette over N frames

Function	Description
ng_palfx_fade_out	Base palette → black over N frames
ng_palfx_flash_white	Brief white tint, decays to normal
ng_palfx_flash_red	Red damage tint
ng_palfx_flash_blue	Blue magic tint
ng_palfx_pulse	Looping brightness oscillation
ng_palfx_cycle	Rotate palette entries [start..end]

Common Mistakes

- Writing to palette RAM directly: `*(uint16_t*)0x400000 = colour;` This bypasses the queue and can cause mid-frame tears. Always use `ng_rq_palette_upload()` or the palfx helpers.
- Forgetting to call `ng_palette_fx_update()` per frame: effects will not advance.

Performance Advice

- Each active slot blends 15 colours: 15×3 multiply+shift = ~45 operations. With 8 slots active simultaneously = 360 operations — acceptable in VBlank.
- For scene-wide fade (all palettes), loop over all slots and call `ng_palfx_fade_out()` on each. Budget accordingly.

Particle System

What It Does

`ng_particles` (Stage 8) manages a fixed pool of short-lived sprite effects: hit sparks, dust, slash trails, explosions, smoke, magic sparks.

Each particle is one 16 px wide sprite strip, animated via tile cycling.

Why It Matters on NeoGeo

Particles add enormous visual richness with minimal artwork — a few tile frames shared across dozens of simultaneous particles. The pool avoids malloc entirely.

Hardware Limitation It Solves

- Fixed pool (32 slots): no memory allocation, deterministic worst-case cost.
- Priority drop: when `sprite_budget_used >= 200`, optional-priority particles are skipped in the draw call but remain alive. Critical particles always draw.
- Gravity is table-free: simple integer add per frame, no lookup needed.

Priority Levels

Level	Behaviour
NG_PART_PRI_CRITICAL	Always drawn regardless of sprite budget
NG_PART_PRI_NORMAL	Drawn unless extreme budget pressure
NG_PART_PRI_OPTIONAL	Dropped when <code>sprite_budget_used >= threshold</code>

Example

```
// At scene init:
ng_particles_init();

// On sword hit:
ng_spawn_hit_spark(hit_x, hit_y, TILE_SPARK_BASE, PAL_FX);

// On landing:
ng_spawn_dust(foot_x, foot_y, TILE_DUST_BASE, PAL_FX);

// Per frame:
ng_particles_update();
next_slot = ng_particles_draw(first_char_slot + chars_drawn, sprites_used);
```

Velocity Format

Velocities are in 8.8 fixed-point (matching `NG_FP_SHIFT = 8`):

- $1 \ll 8 = 1.0$ pixel per frame
- $1 \ll 7 = 0.5$ pixel per frame

Common Mistakes

- Calling `ng_particles_draw()` before `ng_particles_update()`: animation frames will be one frame behind.
- Not reserving sprite slots: particles need slots from the character range (0..299). Pass the first available slot after characters are drawn.

Performance Advice

- 32 particles at 1 strip each = 32 SCB1+SCB2+SCB3+SCB4 writes = fast.
- Limit long-lived optional particles (smoke) to avoid pool saturation.
- Use `ng_particles_count()` to feed the debug HUD sprite counter.

Recent Fixes

- **Sprite budget reporting** (29c5787): the budget counter now correctly reflects the actual number of sprite slots consumed by active particles. Previously the counter was miscounted, causing the debug HUD to show incorrect occupancy.
- **Stale slot cleanup** (3fac09d): when a particle expires, its sprite slot is now explicitly cleared (SCB3 height set to 0). Previously expired particles could leave visible ghost sprites until another particle reused the same slot.
- **Valid slot bounds** (97e14e0): the demo sprite cleanup loop is clamped to valid slot indices, preventing out-of-bounds VRAM writes during scene transitions.

Render Queue

What It Does

`ng_render_queue` (Stage 3) is a deferred VRAM / palette write buffer. Game logic posts `NGRenderCmd` entries during the active frame; at VBlank the queue is drained in a single burst via `ng_render_queue_flush()`.

Why It Matters on NeoGeo

Direct VRAM writes during active display can corrupt the currently-displayed scanline. The safe window is the 16-line VBlank period. The queue separates the "what to draw" decision (game logic time) from the "when to write hardware" moment (VBlank time).

Hardware Limitation It Solves

- Race condition between CPU writes and the LSPC sprite engine reading VRAM.
- Palette flickering caused by mid-frame colour updates.
- Ensures all sprite moves for a given frame are applied atomically.

Queue Capacities

Queue	Slots	Memory
Sprite cmds	128	768 bytes
Palette uploads	8	4 bytes + pointer per slot

Example

```
// Game logic (safe to call any time):
ng_rq_sprite_pos(slot, scb3, scb4);
ng_rq_sprite_shrink(slot, scb2);
ng_rq_palette_upload(pal_slot, my_palette);

// VBlank handler / after waitVbl():
ng_render_queue_flush();
```

Common Mistakes

- Calling `ng_render_queue_flush()` inside the game logic loop (before VBlank): This defeats the purpose. Always flush at or after `waitVbl()`.
- Overflowing the queue: if `ng_rq_free() == 0`, new commands are silently dropped. Increase `NG_RQ_MAX_CMDS` if needed.

Performance Advice

- The flush loop is a tight SCB234 write burst; 128 commands costs approximately 256 VRAM_ADDR + VRAM_RW pairs, well within the VBlank budget.
- For palette uploads, batch palette changes and avoid uploading the same slot twice in one frame.

Sprite Groups

What It Does

NGSpriteGroup wraps one or more adjacent NeoGeo hardware sprite slots into a single logical object. A "group" is a multi-strip character (e.g. 4 strips × 16 px wide = 64 px wide character). The group tracks position, scale, palette, flip flags, and animation tile base in one struct.

Why It Matters on NeoGeo

NeoGeo sprites are 16 px wide strips. A 64 px wide character requires 4 adjacent strips. Strips 1..N must set the "sticky" (chain) bit in SCB3 so hardware glues them to the driver strip. Without correct sticky chaining, strips split apart when the character moves.

Hardware Limitation It Solves

- Prevents strip-split on move by always writing driver SCB3 first, then sticky SCB3 for chained strips.
- Dirty-flag system avoids redundant VRAM writes: only changed attributes are uploaded each frame.
- `ng_sprite_group_flush()` checks `g->dirty` and skips unchanged SCB regions.

Dirty Flags

```

#define NG_SGF_DIRTY_POS      0x01 // position changed
#define NG_SGF_DIRTY_TILE    0x02 // tile base / stride changed
#define NG_SGF_DIRTY_PALETTE 0x04 // palette slot changed
#define NG_SGF_DIRTY_SHRINK   0x08 // scale changed
#define NG_SGF_DIRTY_VIS      0x10 // visibility changed

```

All setters (`ng_sprite_group_set_pos`, `ng_sprite_group_set_palette`, etc.) automatically set the appropriate dirty bit.

Example

```

NGSpriteGroup player;

// Init: 4-strip, 8-tile-high, tile base 0x100, palette 2
ng_sprite_group_init(&player, 0, 4, 8, 0x100, 2);

// Game loop:
ng_sprite_group_set_pos(&player, x, y); // sets DIRTY_POS automatically
ng_sprite_group_flush(&player);          // writes only SCB3/4; zero wasted writes

```

Common Mistakes

- Calling `ng_sprite_group_upload()` every frame: this re-writes all SCB1 data every frame even when the tile hasn't changed. Use `ng_sprite_group_flush()` after init and only call `ng_sprite_group_upload()` when tile data changes.
- Forgetting to set `tileStride`: if the sprite sheet is laid out in rows, stride must equal the number of strips. Call `ng_sprite_group_set_tile_stride()`.

Performance Advice

- For static backgrounds: call `ng_sprite_group_upload()` once at scene load. Never call it again unless the tile set changes.
- For moving characters: use `ng_sprite_group_set_pos()` + `ng_sprite_group_flush()` each frame (writes SCB3+SCB4 only = 2×2 words = 8 bytes per strip).
- Limit groups to ≤ 380 total hardware slots across all active groups.

Performance Rules

NeoGeo 68000 Hardware Budget

Resource	Limit
CPU clock	~12 MHz (68000)
VBlank window	~16 scanlines $\approx$ 1.1 ms

Resource	Limit
Hardware sprites	380 total (slots 0-379)
Palette slots	16 per bank × 2 banks
FIX layer cells	40 × 32

Rule 1: No malloc/free During Gameplay

Use fixed-size pools for everything: particles, sprite groups, game objects. `ng_particles`, `ng_chars`, `ng_npcs` all use static arrays. Dynamic allocation fragments RAM and causes non-deterministic pause spikes.

Rule 2: No float/double

The 68000 has no FPU. Any float triggers a software emulation routine costing hundreds of cycles. Use `NGFixed` (16.16) or 8.8 fixed-point from `ng_defs.h`.

Rule 3: Avoid Division in the Frame Loop

Integer division on the 68000 (`DIVS/DIVU`) is 76-158 cycles per call. In a 60 Hz frame with ~200,000 available cycles, 10 divisions = 1,580 cycles wasted. Use:

- Pre-computed lookup tables (`ng_shrink_tab`, `ng_sin_tab`)
- Right-shift instead of divide-by-power-of-2
- Fixed-point multiply (`NGFX_MUL`) instead of divide

Rule 4: Use Lookup Tables

`ng_fixed.c` provides:

- `ng_sin_tab[256]` / `ng_cos_tab[256]`: sine/cosine without trig
- `ng_shrink_tab[128]`: depth → SCB2 shrink without perspective divide

Rule 5: Fixed-Size Pools

System	Pool size	Adjustable
Characters	64	<code>NG_MAX_CHARS</code>
NPCs	32	<code>NG_MAX_NPCS</code>
Particles	32	<code>NG_PART_MAX_PARTICLES</code>
Palette FX	8	<code>NG_PALFX_MAX_SLOTS</code>
Render queue	128	<code>NG_RQ_MAX_CMDS</code>

Rule 6: Use Dirty Flags

`NGSpriteGroup.dirty` tracks which SCB regions need updating. Only set bits get written to VRAM. A static sprite costs zero VRAM writes per frame after initial upload.

Rule 7: VBlank-Safe Queues

All VRAM and palette writes go through `ng_render_queue`. Game logic posts commands; `ng_render_queue_flush()` applies them at VBlank. Never write directly to `VRAM_ADDR/VRAM_RW` in game logic.

Rule 8: Do Not Update Unchanged Sprites

The dirty-flag system in `NGSpriteGroup` and the render queue enforce this. The Y-depth sorter in `ng_chars.c` re-assigns slots only when sort order changes.

Rule 9: Prefer Pre-Baked ROM Assets

Animation tile sequences, palette data, and audio samples should be in ROM. Do not decompress at runtime; do not compute palette at runtime.

Rule 10: Keep Particles Droppable by Priority

`NG_PART_PRI_OPTIONAL` particles are silently skipped when sprite budget > 200. Use `NG_PART_PRI_CRITICAL` only for essential effects (hit sparks, death burst).

Rule 11: Keep Large Objects Sticky-Bit Chained

Multi-strip sprites MUST use the chain bit (SCB3 bit 6) for strips 1..N. `ng_sprite_group_upload()` and `ng_sprite_group_flush()` handle this correctly. Never write sticky-strip SCB3 without bit 6 set.

Rule 12: Keep Demos at Stable 60 FPS

Monitor `VertBlank` at `0x100000`: if the flag is not set when your game logic finishes, VBlank was missed. Set `ng_dbg_vblank_overflow = 1` in the debug HUD. Target: game logic + VRAM writes < 60% of the inter-VBlank window.

Sprite Budget Allocation Guide

Use	Slots	Notes
Player character	0-7	up to 8 strips
Enemy characters	8-63	2-8 strips each
Particles	64-95	up to 32 strips
NPCs / objects	96-199	varied
Background layer 0	300-315	16 strips
Background layer 1	316-331	16 strips
Reserved/free	332-379	

Total active hardware sprites must not exceed 380.

PART 20

Sound Driver Reference

The Z80 driver, the YM2610, the command protocol, and the audio content pipelines.

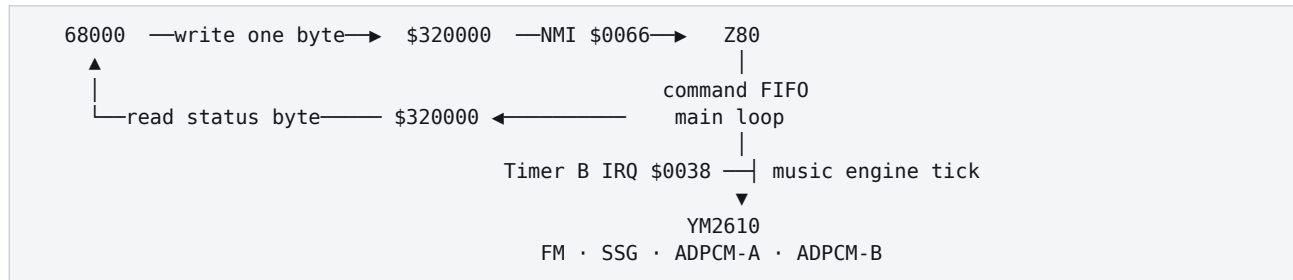
`docs/SOUND_DRIVER.md`

Eagle Software · Neo Geo SDK v1.7.0

Everything the Neo Geo makes noise with runs on a second CPU. This document explains the architecture, the 68000-side API you actually call, the raw command protocol underneath it, the content pipelines that feed it, and the practices that keep audio stable across scene changes.

The exhaustive per-function argument reference lives in `../sound/SOUND_DRIVER_GUIDE.txt`; this document is the map. For the composing tools see `SOUND_STUDIO_GUIDE.md`.

1. Architecture



1. The 68000 sends one-byte commands through `REG_SOUND`.
2. The Z80 receives each command by NMI at vector `$0066`.
3. The driver pushes commands into a FIFO in Z80 RAM.
4. The main Z80 loop drains the FIFO and dispatches.
5. Timer B IRQ at vector `$0038` advances the music engine.

Nothing about this is synchronous. `playSFX()` returns immediately; it has posted a byte, not played a sound. Design around that.

The four sound areas

Area	Channels	Use
ADPCM-A	6 hardware channels, one-shot	Short sound effects, voice samples
ADPCM-B	1 stream at a time	Backing TRACKs, ambience, long cues
SSG	3 square waves	Melody from the MML engine, plus a standalone SSG sequencer
FM	Compiled sequencer, 16-patch voice bank	Music, stings, formant effects

Where the code lives

File	Role
<code>sound/driver/driver.asm</code>	The Z80 driver. Command decoder, FIFO, music engine, patch loader.
<code>sound/driver/driver.c</code>	Experimental C rewrite of the same driver (make <code>m1rom-c</code>).
<code>sound/m1/m1.asm</code>	M1 ROM wrapper the driver is assembled into.
<code>sdk/neogeo.h</code> , <code>sdk/neogelib.c</code>	The 68000-side API — every function below.
<code>sdk/sound_ids.h</code>	Named track / SFX / FM / SSG ids.

File	Role
sdk/sound_voice_ids.h	Generated voice-sample ids (SOUND_VOICE_*).
sound/tools/*.py	MML, FM, SSG, patch, sample and V-ROM compilers.

2. The 68000-side API

Include `sdk/neogeo.h` and `sdk/sound_ids.h`. Call `soundInit()` once at boot before anything else.

Core control

Call	Effect
<code>soundInit()</code>	Boot the Z80 driver and YM2610. Required once.
<code>soundReset()</code>	Driver soft reset.
<code>soundStopAll()</code>	Stop every channel, every area.
<code>soundStopMusic()</code>	Stop MML, SSG and FM sequences; leaves ADPCM playback alone.
<code>soundCancelFade()</code>	Abort a fade and restore the stored base volumes.
<code>soundSceneReset()</code>	The one to call on a scene change — see §5.

Playback

Call	Plays
<code>playMusic(track)</code>	MML music track (SOUND_MUSIC_A..._H, 8 tracks)
<code>playSFX(n)</code>	ADPCM-A one-shot (SOUND_SFX_1..._12, plus the voice bank)
<code>playSFXB(n)</code>	ADPCM-B sample (SOUND_TRACK_*, 9 tracks; index 4 / SOUND_TRACK_E / 5.wav is reserved for the eyecatcher)
<code>playFMTrack(n)</code>	Compiled FM track (SOUND_FM_A..._H, 8 tracks)
<code>playFMDebug()</code>	FM track 0 as a bring-up tone
<code>playSSGTrack(n)</code>	Standalone SSG sequence (SOUND_SSG_A..._I, 9 tracks)
<code>soundSetSSGPrereset(n)</code>	Select one of 16 SSG timbre presets
<code>playInsertCoinSSG()</code>	The coin jingle

Scene helpers

`soundPlayDemoFM(fm_track)`, `soundPlayTitleMusic(music_track)` and `soundPlayGameLoop(music_track)` wrap the common "stop what's playing, start this, at the right volume" sequences. Prefer them over hand-rolled stop-then-start pairs.

Voice

The driver carries a generated speech bank — the letters a–z, digits, punctuation, and whole words — encoded as ADPCM-A samples.

Call	Effect
<code>speakText("READY!")</code>	Prefers a whole-word sample, then falls back to letters, digits and punctuation
<code>playVoiceWord(SOUND_VOICE_WORD_GAME_OVER)</code>	One whole-word sample
<code>playVoiceLetter(SOUND_VOICE_LETTER_A)</code>	One letter
<code>playVoiceNumber(n)</code>	A direct number sample when one exists, otherwise digit-by-digit
<code>playGetReadyVoice()</code> , <code>playAttackVoice()</code> , <code>playCoinThenReady()</code>	Prebuilt cues

`speakText()` calls `soundStopMusic()` first, because speech and SSG melody fight for the same channels.

Volume, tempo, fades

```

soundSetADPCMAVolume(v);  soundSetADPCMBVolume(v);
soundSetSSGVVolume(v);    soundSetFMVolume(v);      /* 0..15 */
soundSetTempo(t);          soundFMSetTempo(period); /* 1..8 */
soundFMSetBPM(120);        soundSSGSetBPM(120);      /* 1..255 BPM */
soundFadeOut();            soundFadeIn();
soundFadeOutSpeed(speed);  soundFadeInSpeed(speed);

```

Chip-level controls

Call	YM2610 register
<code>soundSetADPCMBPan(pan)</code>	\$11 L/R pan — \$C0 both, \$80 left, \$40 right, \$00 mute
<code>soundSetADPCMBLoop(enable)</code>	\$10 repeat on subsequent B starts; 0 one-shot, nonzero repeat
<code>soundFMSetLFO(rateEn)</code>	\$22 LFO — bit 3 enables, bits 0-2 set the rate
<code>soundSetSSGNoise(period)</code>	\$06 noise period, 5 bits

The LFO register is global to the chip. Call `soundFMSetLFO()` after `playFMTrack()`. The live setting survives volume changes, patch reloads and track loops; starting another track or resetting releases the override. PMS/AMS must be nonzero in the patch to make modulation audible.

Background loops and fades

Enable repeat before starting a B sample. Playback then loops at the sample's hardware end address, independently of scene frame rate. It does not require a frame-count watchdog or an early software restart:

```

soundSceneReset();
soundSetADPCMBVolume(0xB8);
soundSetTempo(1);          /* legacy fade divider, not the sample's pitch */
soundSetADPCMBLoop(1);
playSFXB(SOUND_TRACK_B);

```

The setting affects subsequent starts, not an already playing sample. `soundStopAll()` and reset restore one-shot mode; stopping only B retains the preference. The demo's chapter beds opt in explicitly. The existing `soundPlayGameLoop()` helper still starts a one-shot bed despite its name. Seamless musical phrasing also requires a source WAV with loopable endpoints.

Fade steps occur every $\max(1, 255 - \text{speed}) * t / 14.4076$ seconds, where t is the legacy `soundSetTempo()` divider (default 3). Each step changes the stored A/B/SSG mixer levels by 16. With divider 1, speed `0xFE` fades a B level of `0xB8` to zero in about 0.84 seconds. Wait for that interval before stopping the sample. The old no-argument fade default (`0x20`) is deliberately slow; use an explicit speed for short transitions. FM volume is controlled separately.

Precise tempo and sample conversion

The default ASM driver uses a 124.008 Hz Timer-B tick. FM, standalone SSG and MML each maintain a fractional clock with 12 steps per quarter note. T120 means 120 BPM, not a rounded selection among a handful of slow tick divisors. The live BPM functions accept 1..255, clamp zero to one, and persist across MML loop headers until the next track starts. `soundFMSetTempo(1..8)` remains available as the legacy raw-period API, approximately 69.4 ms per period unit. MML duration expiry shares the FM/SSG tick rule, so sequences with equal BPM and phrase lengths stay aligned. Inline `@a / @b / @f / @s` cues keep both the music cursor and the number they name; a cue that once started the wrong sample, or nothing, now starts what it says.

The tracks shipped in the tree are paced between 9 and 72 BPM. That is the pace they were tuned at by ear - an earlier driver stepped every track on a slow fixed clock whatever T said - and the numbers were rewritten to say so when T became a real tempo, so nothing changed in what you hear. Write new tracks at whatever tempo you mean.

The SSG voice sits one octave above its MIDI number: an 04 note in an SSG track sounds at what a piano calls octave 5. That is where a 4-bit square wave carries over the FM and ADPCM beds, and every SSG track in the tree was written against it.

New WAV conversions use a band-limited polyphase filter (plain numpy, no extra package). ADPCM-A is encoded from the hardware reset predictor and its 256-byte alignment tail is encoded toward silence. Voices use their own sample end addresses rather than a shared short timeout. ADPCM-A samples never cross a 1 MiB page.

ADPCM-B conversions default to 32 kHz. A `<filename>.json` sidecar records `sample_rate` for each raw/encoded file; keep it with the asset. VROM compilation generates per-sample Delta-N values for registers `$19/$1A`. Legacy encoded files without sidecars are treated as 16 kHz. Set `ADPCMB_RATE=16000` in the environment before conversion for a smaller bank. Original WAV files are not modified. Do not mix a newly packed V1 with an older M1 sample table.

```
make GAME=demo sound
make GAME=demo pl hash
python3 -m unittest discover -s tests -p test_sound_tools.py
python3 tools/sound_capture.py --output ../showcase-qa/audio
python3 tools/sound_capture_report.py ../showcase-qa/audio
```

On Windows use `make -f MakefileWin32.mak` and `py` in place of `python3`. The report checks recorded audio as well as register traffic. Sustained-note pitch measurements distinguish LFO-off from the enabled rates; writing `$22` alone is not considered proof of audible modulation. Rate sidecars use LF line endings on both platforms so a rebuild does not churn their contents.

`m1rom` packs matching V1/sample tables before assembly. Data tables are placed contiguously below `$8000`; overlapping writes and fixed-bank overflow fail the build. These timing and rate extensions currently target `driver.asm`; the separately linked C driver remains experimental.

CSM — Composite Sine Mode

CSM is an advanced existing path on the channel with register base `$32` (`$A2/$A6` frequency). It needs its own patch; the normal sequencer writes the other port-A channel and does not initialize the CSM voice.

```

soundFMCSMLoadVoice(5);          /* driver cmd $1D, patch index 0..15 */
soundFMCSMBegin(period_hi);      /* driver cmd $1B — reg $24/$25/$27 */
soundFMCSMSweep(from, to, ms);   /* slide the formant, then end CSM */
soundFMCSMEnd();                 /* driver cmd $1C, keeps Timer B alive */

```

Begin writes \$27 = \$CB; end writes \$1A, preserving Timer B and silencing the CSM channel. `soundStopAll()` also clears the mode. CSM synthesis was not changed or acoustically qualified in this pass and is not part of the sound chapter's LFO demonstration.

3. The raw command protocol

Use the SDK functions first. Reach for raw bytes only for a feature with no wrapper yet — `playSoundtest(index)` in `sdk/neogeolib.c` sends one directly.

Byte	Meaning
\$01	Driver init
\$02	BIOS eyecatcher / boot music (mapped to music track 1)
\$03	Driver soft reset
\$04	Stop all playback
\$05 <i>n</i>	ADPCM-A volume
\$06 <i>n</i>	ADPCM-B volume
\$07 <i>n</i>	SSG music volume
\$0A <i>n</i>	Fade-out speed
\$0C	Stop ADPCM-A only
\$0D	Stop ADPCM-B only
\$0E <i>n</i>	Live tempo divisor
\$0F	Stop MML, SSG and FM sequences
\$11	Cancel fade, restore base volumes
\$12 <i>n</i>	Fade-in speed
\$13 <i>n</i>	FM volume (low nibble, 0-15)
\$14 <i>n</i>	SSG preset index (low nibble, 0-15)
\$15 <i>n</i>	ADPCM-B pan (\$C0/\$80/\$40/\$00)
\$16 <i>n</i>	ADPCM-A sample by full index — used for sample ids ≥ 64
\$17 <i>n</i>	FM LFO enable/rate (low nibble)
\$18 <i>n</i>	ADPCM-B repeat for subsequent starts (0 off, 1 on)
\$19 <i>n</i>	SSG noise period (low 5 bits)
\$1A <i>n</i>	Legacy FM step interval, 1..8
\$1B <i>n</i>	CSM begin, Timer-A high byte
\$1C	CSM end
\$1D <i>n</i>	CSM voice patch, 0..15
\$1E <i>n</i>	Live FM BPM, 1..255
\$1F <i>n</i>	Live standalone SSG BPM, 1..255

Byte	Meaning
\$20+n	Start music track <i>n</i>
\$28, \$29	Direct trigger, ADPCM-B samples 0 and 1
\$30	FM debug tone (FM track 0)
\$31 <i>n</i>	FM track select
\$32 <i>n</i>	SSG track select
\$40+n	ADPCM-A sample <i>n</i> , compact range 0-63
\$80+n	ADPCM-B sample <i>n</i>

How a byte is acknowledged

Every byte is one NMI on the Z80. The handler reads the latch, drops the reply port (\$320000 read from the 68000) to 0, files the byte in a 32-byte queue, and raises the reply to 1 again. The driver's main loop executes queued commands afterwards, so the acknowledgement means *accepted*, not *done*.

At rest the reply reads 1, and it has to: the BIOS reads that port before it sends its own \$03 and re-initialises the driver if it finds anything else there. That is also why 1 on its own cannot prove a byte arrived - it is what the port held before the write. `soundCommand()` therefore waits for the reply to drop to 0 after writing, and only then for the 1 that follows. Writing the next byte on the stale 1 overwrites the latch before the Z80 has read it, or fires a second NMI inside the handler, which files both bytes into the same queue slot; either way one byte is lost, and for a prefixed command that means the next command is swallowed as the missing parameter. The demo's beds failed to start exactly that way until the wait was added.

All SDK wrappers go through `soundCommand()`, so a prefix byte and its parameter can be sent back to back. Between *commands* the demo still leaves a `waitVbl()` so the driver has finished the previous one before the next arrives - a stop-all and a start in the same frame race the driver's own busy-waits:

```
soundSetSSGPreset(3);
waitVbl();
playSSGTrack(SOUND_SSG_B);
```

Isolated and live audio checks are different tests

`tools/sound_capture.py` parks the 68000 and feeds the driver one byte per frame from Lua: it proves the driver, the tables and the samples, and it would have passed while every chapter of the demo was silent. Run `tools/demo_audio_capture.py` as well - it boots the real ROM set, lets the 68000 send its own commands, records the mix and every latch byte, and `tools/demo_audio_report.py --strict` fails if any chapter goes quiet:

```
python3 tools/demo_audio_capture.py --output ../showcase-qa/live --press-every 9
python3 tools/demo_audio_report.py ../showcase-qa/live --strict
```

- `--press-every` walks the reel with A the way a player does; - `--coin-at` drops a coin mid-reel to check the cue lands over the music; - `--trace-z80` adds the driver's queue traffic to `events.tsv` when a byte goes missing.

4. Content pipelines

make sound runs all of these. Each has its own target if you only changed one kind of asset.

Source	Compiler	Target	Lands in
games/<game>/sound/mml/*.mml	mml_compile.py	mml	music_data.inc
games/<game>/sound/fm/*.mml	fm_compile.py	fm	fm_data.inc
games/<game>/sound/fm/patches.fm	fm_patch_compile.py	fmpatches	fm_patch_table.inc
games/<game>/sound/ssg/*.mml	ssg_compile.py	ssg	ssg_data.inc
games/<game>/sound/ssg/config.ssg	ssg_config_compile.py	ssgconfig	ssg_config.inc
games/<game>/sound/samples/in_wav_a/*.wav	WAV converter + adpcm_enc.py	samples	ADPCM-A + rate sidecars
games/<game>/sound/samples/in_wav_b/*.wav	WAV converter + adpcm_enc.py	samples	ADPCM-B + rate sidecars
games/<game>/sound/samples/in_wav_a_voice/*.wav	WAV converter + adpcm_enc.py	samples	ADPCM-A voice bank
encoded sample banks	vrom.py	vrom	V-ROM, sample_table.inc, sdk/sound_voice_ids.h
all of the above	wla-z80 + wlalink	mlrom	<id>-m1.m1

Sample numbering is positional: playSFX(0) is 1.wav, playSFX(SOUND_SFX_7) is 7.wav. Renaming or inserting a file rennumbers everything after it.

The Python sample path is the default; S0X=/path/to/sox switches to the SoX conversion path only if you explicitly want it.

Two driver builds

```
make mlrom-asm      # assemble sound/driver/driver.asm (the shipping driver)
make mlrom-c        # build the experimental C driver
make compare-driver # build both and diff the ROM images
```

driver.asm is the driver that ships. The C version exists to be diffed against it, and compare-driver is how you check a change to one against the other.

5. Practices that keep audio stable

Initialise once. soundInit() in game_boot(), never per scene.

Reset on scene change, do not stack stops. soundSceneReset() exists because the failure mode of hand-rolled teardown is a fade left running, a CSM mode left set, or a volume left at zero, three scenes later.

Space multi-byte commands. See §3.

Do not poll for completion. There is no reliable "sample finished" handshake for general use. Drive timing from your own frame counter.

Fade, then stop. A hard soundStopAll() under a playing ADPCM-B stream is audible as a click. soundFadeOut() then stop reads as intentional.

Watch the ADPCM-A channel count. Six channels. A seventh trigger steals one of the six, and the one it steals is whichever the driver picks — not necessarily the least important. Budget effects the way you budget sprites.

6. What the demo shows

Chapter 23 of the demo ROM (SOUND) previews ADPCM-B beds, pan, short ADPCM-A effects, FM 4/6, three SSG melodies, persistent LFO rates, live BPM changes, recorded voice, formant effects and fades. The full mix uses a B bed with sparse A cues, avoiding unrelated melodies stacked over each other. The separate MML example pairs the warm FM bell with a quiet G-minor SSG bass and one drum cue over a matching seven-bar phrase at 112 BPM. Chapter 24 (SKY LANCE) drives a pure-SSG mix under gameplay, with every Z80 command spaced by `waitVbl()` so multi-step setup applies cleanly. See `DEMO_CHAPTERS.md`.

PART 21

Sound Studio Guide

The desktop audio-authoring tool: track, mix and ROM tabs, the live waveform view, and the MML designer.

`docs/SOUND_STUDIO_GUIDE.md`

v1.7.0 — new tabs Sound Studio gained SEVEN new tabs in v1.7.0: - **Live Waveform** — generate a clip (C major arpeggio / step pattern / MML phrase) and watch the WAVEFORM + FFT SPECTRUM animate as the clip plays through QAudioSink. Spectrum is colour-graded (green low-freq → yellow mid → red high) so the harmonic balance is read at a glance. - **MML Designer** — visual piano-roll MML editor. Click cells in a 2-octave × N-step grid to place notes; the corresponding `t### l# o# c d e ...` text is generated live and can be saved to `sound/mml/`. Step count, tempo, and step length are adjustable. Plus the five tabs from the previous v1.7.0 commits: - **Pipeline** — runs every audio build step (samples / vrom / fmpatches / mml / ssg / ssgconfig / fm / m1rom) with a status pill per step and a shared live log. "Run Full Pipeline" chains them sequentially and stops on the first failure. - **Track Browser** — tree of every audio asset in `sound/` (MML scripts, SSG presets, WAV samples). Click a file to preview its content (text files inline, binaries reported by size). - **Audio Mix** — live mixer for `soundApplyMix(adpcma, adpcmb, ssg, fm)`. Four sliders, five preset buttons (Default game / FM showcase / Pure SSG / Stage mix / Mute) and an auto-updated C call you can copy into your game init. - **ROM Inspector** — table of every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size + mtime + present/absent colour-coded. - **Identifiers** — read-only side-by-side view of `sdk/sound_ids.h` and `sound/driver/driver_defs.h` so you don't have to grep when wiring code or MML. The unified demo's chapter 03 (SOUND) replaces the noisy raw FM-patch demo with a clean MML music intro (SOUND_MUSIC_E). When auditioning patches through Sound Studio, prefer the MML/composition path over `playFMTrack(SOUND_FM_H)` alone — the patch table doesn't carry sequencing. Sound Studio's panels map 1-to-1 to the on-ROM channels: FM (4 ch), SSG (3 ch), ADPCM-A (6 ch sampled), ADPCM-B (1 ch streamed). The identifier headers `sdk/sound_ids.h` and `sound/driver/driver_defs.h` remain the source of truth for what each command does on the Z80 side.

`sound/sound_studio.py` is a PyQt6 desktop tool for composing and previewing NeoGeo YM2610 audio without leaving your dev environment.

```
python3 sound/sound_studio.py
```

Requirements: Python 3, PyQt6, numpy, scipy.

```
pip install PyQt6 numpy scipy
```

Tabs

FM Patches

Edit the FM patch bank (`sound/fm/patches.fm`).

- **Patch list** — select any of the 16 patches; click + **New Patch** to add one.
- **Name / ALG / FB / Stereo / LFO** — top-level patch controls.
 - ALG (0-7): operator routing algorithm. The diagram on the right updates live.
 - FB (0-7): Op1 self-feedback amount.
 - Stereo (hex): C0 = both channels, 80 = left, 40 = right.
 - LFO (hex): global LFO enable and rate.
- **Operator grid** (OP1-OP4) — each operator has seven parameters in hex:
 - DT/MUL: upper nibble = detune, lower nibble = frequency multiplier.
 - TL: total level (attenuation). 00 = loudest, 7F = silent.
 - AR: attack rate (0-31). Higher = faster attack.

- DR: first decay rate.
- SR: sustain/second decay rate.
- SL/RR: upper nibble = sustain level, lower nibble = release rate.
- ENV: SSG-EG envelope type (00 = off).
- **Piano keyboard** — click any key to synthesize and play a preview note through your speakers. The waveform display shows the simulated output.
- **Save patches.fm** — writes changes back to sound/fm/patches.fm.

After saving, rebuild the patch table:

```
make fmpatches          # Linux
make -f MakefileWin32.mak fmpatches  # Windows
```

MML Composer

Write and preview MML melody tracks for FM and SSG channels.

- **Mode** selector — switch between FM tracks (sound/fm/*.mml) and SSG tracks (sound/ssg/*.mml).
- **Track list** — lists all .mml files in the selected directory.
- **Editor** — syntax-highlighted MML text editor. Comments, note commands, and tempo/volume directives are colour-coded.
- **Piano roll** — updates live as you type. Each note appears as a coloured bar at its pitch and position in time. Different instrument indices use different colours.
- **+ New Track** — creates a new .mml file with a starter template.
- **Save Track** — writes the editor content back to disk.
- **Compile All** — runs fm_compile.py or ssg_compile.py over all tracks and writes the output .inc file into sound/driver/.
- **Piano keyboard** (right panel) — audition individual notes using the currently simulated patch.

MML syntax reference:

Command	Meaning
T<n>	Tempo in BPM (default 120)
V<n>	Volume 0-15
I<n>	Instrument / patch index (0-15 for FM, 0-N for SSG)
O<n>	Octave 0-8
L<n>	Default note length (1=whole, 4=quarter, 16=sixteenth)
> / <	Octave up / down
C-B	Note name
C+ / C-	Sharp / flat
R	Rest
C4.	Dotted note (1.5× length)
;comment	Line comment

After compiling, rebuild the M1 ROM:

```
make mlrom          # Linux
make -f MakefileWin32.mak mlrom  # Windows
```

SSG Presets

Edit the SSG preset bank (sound/ssg/config.ssg).

- **Preset list** — all named presets; click + **New Preset** to add one.
- **Name** — free-text preset name.
- **Tone Mask** (hex) — mixer register. 0x38 = all three SSG channels output tone, noise off.
- **Vol A / B / C** (hex, 0-F) — per-channel volume. 0 = silent, F = maximum.
- **Noise Freq** (hex) — noise generator period. 00 = highest pitch noise.
- **Piano keyboard** — click a key to hear the preset as a square wave at that pitch.
- **Save config.ssg** — writes the preset bank back to disk.

After saving, rebuild the SSG config and M1 ROM:

```
make ssgconfig mlrom
```

ADPCM Samples

Browse and preview the raw WAV source files used to build the VROM.

- **Channel selector** — switch between ADPCM-A (sound/samples/in_wav_a/) for sound effects and ADPCM-B (sound/samples/in_wav_b/) for streaming music/ambient.
- **Sample list** — all .wav files in the selected input directory.
- **Waveform display** — shows the waveform of the selected file.
- **Info panel** — sample rate, channels, duration, frame count.
- **► Play** — plays the selected WAV through your speakers.
- **Add WAV Sample...** — file picker to copy a new WAV into the input directory.

After adding or replacing samples, re-encode and rebuild the VROM:

```
make samples vrom      # Linux
make -f MakefileWin32.mak samples vrom  # Windows
```

Channel limits:

Channel	Simultaneous	Use case
ADPCM-A	6	Short SFX, punches, voices
ADPCM-B	1	Long loops, ambient beds, ending themes

YM2610 Simulator

Preview all channels together to hear how they blend before building a ROM.

- **Channel grid** — 10 rows: FM 1-4, SSG A-C, ADPCM-A 1-2, ADPCM-B.
 - **Note** spinbox — MIDI note number with note name suffix (e.g. 60 C4).
 - **Instrument** spinbox — patch index for FM, preset index for SSG.
 - **Volume** spinbox — 0-15 per channel.

- **▶ button** — play that single channel in isolation.
- **▶ Play All FM + SSG Channels** — synthesizes all FM and SSG channels simultaneously, mixes them, and plays the combined result. The waveform display shows the mix.
- **Waveform display** — shows the most recently synthesized output.

The simulator uses numpy-based FM and square-wave synthesis at 44100 Hz and plays through QAudioSink. It approximates YM2610 timbre — use it to tune patches and balance volumes before committing a build.

Typical Workflow

New FM instrument:

1. Open **FM Patches** → **+ New Patch**
2. Set ALG=7 (all carriers additive) to start
3. Lower TL on operators you want audible (00-30)
4. Set AR=1F for instant attack; adjust DR, SL/RR for the shape
5. Click piano keys to audition — iterate until it sounds right
6. **Save patches.fm** → make `fmpatches m1rom`

New melody track:

1. Open **MML Composer**, Mode = FM (or SSG)
2. **+ New Track** → write the melody in the editor
3. Watch the piano roll update as you type
4. **Compile All** → make `m1rom`

New sound effect:

1. Drop a .wav file into `sound/samples/in_wav_a/`
2. Open **ADPCM Samples** → confirm waveform and duration
3. make `samples vrom` to re-encode and pack

Balance a scene:

1. Open **YM2610 Simulator**
2. Set each channel's note and instrument to match your scene
3. **Play All** → adjust volume spinboxes until the mix feels right
4. Apply those volume values to your `soundSetFMVolume()` / `soundSetSSGVolume()` calls in `main.c`

PART 22

Graphics Pipeline Reference

PNG to C-ROM: the quantiser, asset ordering, generated outputs, and Artbox Studio.

`docs/ARTBOX_PIPELINE.md`

Eagle Software · Neo Geo SDK v1.7.0

Artbox converts PNG sources into Neo Geo graphics data and the generated metadata your game code binds against. It is the stage that decides how your art actually looks on the machine, so it is worth understanding rather than treating as a black box.

The colour target

A Neo Geo colour word is not RGB444. Bit 15 is the common "dark" bit, bits 14-12 carry the low bit of each five-bit channel, and bits 11-0 carry their upper four bits. The common dark bit produces two interleaved intensity grids; it is not an independent sixth bit for each channel. The converter fits on this hardware lattice and verifies the packed words.

Each sprite or tile can use **15 colours plus transparent** — 4 bits of index, where index 0 is transparent.

Default quantiser

`artbox/romdbimgimport.py` routes normal builds through `artbox/palette_banks.py`. Both makefiles use this path. Older experimental converters remain available, but their contrast enhancement, CLAHE and per-tile diffusion are not part of the default import.

1. Fit the first 15-colour palette to the whole image's visible content, weighted by source pixel frequency. Refine in CIE-Lab, snapping to the hardware grid during refinement rather than only after fitting.
2. Preserve that whole-image palette as a fallback. For each extra bank, fit the worst remaining tile, then consider refitting the whole region that benefits. Accept a bank only when it reduces nearest-colour error.
3. Select one bank per 16x16 tile. Keep the indices and that bank map together through C-ROM encoding, generated metadata and runtime binding.
4. Apply no dither to normal sprite art. Backgrounds may use low-amplitude, image-aligned ordered dither; no diffusion wave crosses sprite contours.

Solid core pixels train sprite palettes where sufficient core coverage exists. Tiny antialiased craft fall back to visible pixels so their palette is not fitted to a few highlights alone. Alpha below 128 becomes index zero. Opaque black remains a nonzero index. The default path does not force extra contrast or saturation onto the artist's colours.

`palette_banks = 1..16` is an upper limit, not a promise to consume every bank. Static sprites now honour it as screens do. Animation families retain one shared master palette; raising their budget alone does not enable per-frame extra banks, which could introduce colour changes during motion.

Resampling

Screen fitting uses `fit_screen_for_display()` and accounts for the chosen vertical hardware shrink. Sprite fitting uses `fit_sprite_rgba()`: preserve aspect ratio, reduce only when necessary, then pad transparently at the requested anchor. The current sprite fitter rounds canvas dimensions down to multiples of 16, so choose explicit multiples for predictable output. It uses Pillow's RGBA Lanczos thumbnail path, not the optional linear-light helper available in `img2neo.py`.

`prepare_source_sprite()` applies optional border-connected halo cleanup and alpha-edge colour bleeding after fitting. Original PNG files are not rewritten. Avoid global halo stripping on luminous effects: bright pixels can be intentional art, not a matte defect.

Palette banks

`allocate_extra_palettes()` assigns absolute hardware banks after fitting. Byte-identical ordered colour words share one slot, including extra banks. This step does not approximate or reorder colours: pixel indices are already encoded, so two palettes containing the same colours in a different order are not interchangeable. The verifier compares the actual words, not only the manifest's `palette_key` hash.

The generated `NGArtAsset.tile_palettes` pointer starts at the artwork's first active tile. It uses the same row stride as the tile data. Bind it with `ng_sprite_group_set_palette_map()` or `ng_char_set_palette_map()`; loading the banks without binding this map renders the right pixels in the wrong colours. Neither setter loads palettes or writes VRAM immediately.

Import size

Import an asset at the size it is drawn. This matters more than any single setting in the quantiser.

The sprite chip can shrink, and it is tempting to import large and let the hardware fit the art to the scene. But the hardware does not resample when it shrinks — it drops whole rows and columns, and what it drops is a dither pattern the quantiser chose for pixels it expected to survive. Halve a sprite in hardware and half of that pattern is gone; quarter it and the shape goes with it. The source can be flawless and the result still arrives as a smear of colour.

The pipeline fits and quantises against the pixels it actually produces. The rule is: set each rule's `target_width/target_height` to the largest size that asset is ever drawn, and draw it at full scale.

Smaller assets need fewer hardware sprite strips. This ROM layout still reserves 256 tiles per asset, so smaller imports currently leave transparent C-ROM padding rather than reducing the reserved allocation.

`cat_sky_planes`, `cat_sky_opponents`, `cat_sky_bosses` and `cat_big_enemy` exist for exactly this — per-asset ceilings for sprites whose scene size is known.

One caveat before lowering a ceiling on art that is already in use: changing it moves that asset's strip count and tile stride, and `ng_char_set_sprite()` silently drops a bind whose asset window fails validation. A character bound through that path then keeps whatever it had before, which shows up as stale or missing art rather than as an error. The demo's own `cat_characters` ceiling is still 256 for that reason. Check what a ceiling change does to `NG_ASSET_STRIPS_*` and to any slot map built around it before assuming it is free.

Synthesised backdrops

`artbox/gen_starfield.py` generates a tiling deep-space page — a multi-hue nebula from wrapping fractal noise, dust lanes, and three layers of stars. The noise lattice wraps, so the page joins itself exactly rather than approximately, and a backdrop scrolled as two copies shows no line at the wrap.

```
python3 artbox/gen_starfield.py left.png right.png --seed 7
```

Given several outputs it generates one page at the combined width and cuts it into columns, so pages laid side by side join without a seam down the middle. Everything is synthesised, so the result carries no licence.

Tile stride

Every asset carries a `tile_stride` in its metadata: the width of the canvas it was imported onto, in tiles. Row n of the artwork begins that many tiles after row $n-1$.

Anything binding an asset has to use it — `ng_char_set_tile_stride()`, or `ng_sprite_group_set_tile_stride()`. **Do not assume 16**. Sixteen is the stride of a 256 px canvas and nothing more; an asset imported onto a narrower one has a narrower stride, and binding it with 16 reads every row after the first from further along the C ROM than the artwork is. The symptom is the right palette over the wrong art — a recognisable shape in plausible colours that is not the asset you asked for, which is easy to mistake for a quantiser problem.

Both engines validate `ng_char_bind_asset()` against the incoming asset's stride and bounds before modifying the character. It returns zero on failure and leaves the previous frame intact; check its return value. The lower-level `ng_char_set_sprite()` returns void and retains its existing bounds checks. Bind the map after setting sprite geometry and stride:

```
ng_char_set_sprite(character, first_slot, asset->strips, asset->active_rows,
                  asset->tile_base, asset->palette_bank);
ng_char_set_tile_stride(character, asset->tile_stride);
ng_char_set_palette_map(character, asset->tile_palettes);
```

For an atomic bind, zero-initialize `NGSpriteAssetView`, fill its geometry, range, offsets and `tile_palettes`, then call `ng_char_bind_asset()`. The map contains absolute bank numbers and must remain alive while bound; generated ROM tables satisfy that lifetime. NULL selects the single bank in `character->palette`. `ng_char_set_sprite()` resets the map, so set it again after changing the asset. The C++ method is `NGCharacter::setPaletteMap()`; the same C entry point is available in both engines. Rebuild all game objects when updating the SDK because `NGCharacter` and `NGSpriteAssetView` gained a field.

Palette RAM budget

The SDK allocates 239 asset banks: 16..254. It reserves 0..15 for FIX and 255 for backdrop/HUD use. Each asset bank has 15 visible entries plus transparent index zero, so the asset budget is 3585 visible colour entries. That is an allocation ceiling, not a guarantee of that many distinct colours.

That budget is global to a game — an asset owns its banks for the life of the ROM, so every asset's banks have to coexist. Two things keep it from being wasted:

Assets holding the same palette share a bank. Base banks used to be positional, one per asset, so an animation whose frames all render against one shared palette occupied a bank per frame holding the same colours. Base and extra banks now share only when all sixteen ordered words match. `palette_key` is a stable diagnostic fingerprint; the verifier reads the actual base words from `neopal.bin` and extra words from the manifest. Changing a shared bank for a palette effect changes every object using it. Copy to a separately reserved effect bank before an independent colour effect.

Extra banks go to what covers the screen. A background occupies the whole frame and a sprite occupies a fraction of it, so the budget is weighted accordingly. Budgets live in each game's `artbox/assets.cfg`, not only in the shared generator defaults. Existing configs are preserved. Sky Lance has enough capacity for 16-bank bosses, 4-bank opponents and 3-bank player craft. The demo retains conservative single-bank animation families. Do not copy one game's budgets into another without rebuilding and checking total occupancy. Additional banks are used only when beneficial.

What extra banks cannot fix

A tile reads one palette. Extra banks help where the variety is *between* tiles and not at all where it is *within* one, which is why they transform a background more than a small character. An earlier

isolated fitting study measured these mean dE values; they are examples, not current build totals:

	1 bank	8 banks	16 banks	one per tile
background	3.93	3.18	2.99	2.29
boss sprite	11.78	9.30	8.23	5.85
character	11.39	10.16	9.77	8.06

A bank per tile is the floor this hardware allows, and it is out of reach for a game with 86 character frames — that alone would want more palette RAM than exists.

The second hardware palette bank

The hardware has two complete palette sets, selected by a write to REG_PALBANK0 / REG_PALBANK1, so 8192 entries can be resident and 4096 displayed. The allocator currently uses one set. The second set may be useful for prepared whole-scene palette switches, but it does not remove the 15-visible-colours-per-tile limit.

Palette anchors

Index zero is always transparent, independently of its RGB value. The default quantiser preserves all distinct hardware colours when they fit within fifteen entries. More complex sources use frequency-weighted fitting; there is no automatic UI detector or unconditional black/white reservation. The optional older converters have separate anchor heuristics.

HD conditioning

The older `img2neo_tile.py` and optional HD tools expose bilateral filtering, CLAHE and sharpening. Those controls do not run in the default source-faithful path. Compare any optional converter against the source and actual C-ROMs before adopting it; stronger contrast or more dither is not automatically more accurate. No converter can recover detail removed by resizing or make the hardware display unrestricted RGB per pixel.

Alternative pipelines

Route	Script	When
Default	<code>palette_banks.py</code> via <code>romdbimgimport.py</code>	Whole-image fallback, error-driven extra banks, stable animation masters.
CRT	<code>img2neo_crt.py</code>	<code>make_art_crt</code> or <code>ARTBOX_CRT=1</code> . Lab k-means, horizontal-biased dither, gamma 1.20 / contrast 1.10 pre-boost.
HD	<code>img2neo_hd.py</code>	Opt-in conditioning; measure against the source before adopting it.
Legacy	<code>ARTBOX_LEGACY=1</code>	The original nearest-neighbour path, kept for diffing.
FIX HD	<code>fixtiles_hd.py</code>	Per-tile palette FIX conversion; <code>--sharp-text</code> binarises glyph sources.

`img2neo.py` also exposes `alpha_bleed()`, which stops anti-aliased contours baking the source PNG's hidden transparent-pixel RGB (usually near-white) into the indexed sprite. That is the fix for the pale halo around sprite edges, and `resize_rgba_linear()` runs it before any resize so the halo never gets a chance to become opaque.

A sprite whose background is opaque instead of transparent is a different problem and no filter can fix it: index 0 is transparency on this hardware, so an opaque matte is not a colour that happens to be wrong, it is a solid block the sprite carries around with it. Repair the source with `artbox/fix_sprite_alpha.py` PATH, which flood-fills only background connected to the image border and leaves interior colour alone.

Asset ordering and ids

Categories are processed in a fixed order:

```
backgrounds, characters, effects, eyecatcher, npc, screens, titles
```

then any other directory alphabetically. Within a category, files are processed in filename order.

An asset id is its 1-based position in that sequence. Inserting a file in the middle of a category rennumbers everything after it, and your game code refers to assets by id. The shipped games pad filenames to control placement, and you should too.

Animated sprites share a derived 15-colour master palette per animation family (`build_master_sprite_palettes()`). The demo keeps the warrior, selection portraits, eagle, and each remaining character row in separate families. NPC filenames are grouped by their stable name prefix. This keeps colors stable within an animation without forcing unrelated artwork into one 15-color budget. A sprite that looks wrong after being renamed or moved between categories may have entered the wrong palette family.

Rule matching in `assets.cfg` has three priorities; the lowest is a rule with an empty `match_category` plus a filename pattern, which is how the `legacy_screens` rule (`[0-9].png, mode = screen`) catches numbered files in any directory.

Generated outputs

File	Contents
<code><id>-c1.c1, <id>-c2.c2</code>	Sprite tile data, the C-ROM pair
<code><id>-s1.s1</code>	FIX tile ROM
<code>screens.c</code>	One <code>showScreenN()</code> entry point per asset
<code>sprite_meta.h</code>	Per-asset geometry — tile base, strips, rows, padding
<code>assets_manifest.json</code>	The manifest the rest of the pipeline reads
<code>neo.pal, std.pal, neopal.bin</code>	Palette data
<code>infix_palettes.h</code>	FIX palettes for INFIX photo pages

`main.c` in each game holds the generated `ng_screen_table[]` and `ng_screen_count`, which `ng_bg_set_by_id()` resolves screen ids through. A game with no screen art links against a weak fallback in `ng_bg.c` instead.

Verifying a build

```
python3 tools/verify_artbox_palettes.py --root . --game demo
python3 tools/verify_sfix_output.py
python3 artbox/validate_assets.py
```

`make art` runs the palette check itself and fails the build on a mismatch.

Host regressions and captured runtime checks:

```
make unit-tests
python3 -m unittest discover -s tests -p 'test_artbox*.py' -v
python3 tools/game_capture.py --game skylance --output /tmp/sky-capture --seconds 180
python3 tools/artbox_runtime_report.py --game skylance --capture /tmp/sky-capture --strict
```

The capture stores screenshots, SCB VRAM and palette RAM. The runtime report compares populated tile entries with generated bank assignments and colour words. Palette-effect scenes intentionally change colours and are not suitable for an unconditional strict comparison. Captures mute audio and cannot verify sound quality. Rebuild P1 after rebuilding art so runtime tables match C-ROMs.

For source fidelity, retain `assets_manifest.json` and `neorom.db` from the previous build in a separate folder, then run:

```
python3 tools/artbox_quality_report.py --game skylance --before /tmp/sky-before --output /tmp/sky-quality
```

This decodes both C-ROMs, checks them against the importer indices and writes source/before/after comparisons plus per-asset Lab error. Matching asset lists are required. It accepts manifests produced on either Windows or WSL.

Artbox Studio

`artbox/artbox_studio.py` is the PyQt6 desktop front end for everything above. It does not replace the make targets — it drives the same scripts — but it makes the feedback loop visual.

Tab	What it does
Pipeline	Runs every step (<code>img2neo</code> , <code>genscreens</code> , <code>gen_sprite_meta</code> , <code>fixtiles</code> , <code>romdbimgimport</code> , <code>romtiles</code> , <code>romdbfiximport</code> , <code>createromdb</code>) with a status pill per step and a live log. "Run Full Pipeline" chains the standard set.
Asset Browser	Tree of every PNG under <code>artbox/in/*</code> and <code>artbox/infix/</code> , with dimensions, file size, and a native-scale preview.
Hex Sprite Inspector	Pick a tile index and see its decoded 16×16 grid, the raw C1/C2 ROM bytes, and a live palette-swap dropdown. The tool for debugging palette assignment and tile order.
Movement Designer	Build a frame sequence (tile + duration), loop-preview it at a chosen FPS, and export <code>anim_frame_tile[] / anim_frame_dur[]</code> C arrays in vblank units.
Level Designer	Paint a 20×14 tilemap with tile thumbnails; export a <code>level_tile[LEVEL_H][LEVEL_W]</code> C array.
HD Compare	Run one PNG through both the standard and HD pipelines and see the indexed results side by side. Output lands in <code>artbox/out/_hd_cmp/</code> ; originals are untouched.
ROM Inventory	Every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size, mtime, and present/absent colour coding.
Asset Rules	Editor for <code>artbox/assets.cfg</code> , with save and reload.

The current pipeline is rule-driven through `artbox/assets.cfg`.

Inputs

- artbox/in/ contains PNG artwork for sprite pages and screen/background pages.
- artbox/infix/ contains FIX-layer source artwork.

Rule Fields

Each [rule:name] section in artbox/assets.cfg can define:

- pattern: filename glob, such as sprite_*.png
- mode: screen or sprite
- category: gameplay category written into generated metadata
- fit: crop/contain for screens, pad for sprites, or native for pre-tiled screens
- anchor: placement anchor such as center or bottom-center
- target_width / target_height: output canvas size
- palette_banks: maximum 1..16 banks for static art; shared animation masters remain single-bank
- dither: use none for sprites or ordered for gentle background dithering
- halo_strip / halo_luma_threshold: optional sprite border cleanup
- display_shrink_y: the screen fitting ratio expected at runtime
- older image-processing settings such as contrast and kmeans_iters affect optional converters, not the default fitter

fit = native bypasses screen aspect compensation and requires the source dimensions to equal the target canvas. Use it for pixel art that already has a tile-aligned repeat, such as the demo's 144-pixel Sky Lance sky loop. Applying contain to that art adds transparent bands and changes the repeat distance. Other screens retain their existing fitting and colour algorithms.

Categories

Generated metadata uses these category constants:

```
#define NG_ASSET_CATEGORY_BACKGROUND 0
#define NG_ASSET_CATEGORY_MAIN_CHARACTER 1
#define NG_ASSET_CATEGORY_OPPONENT 2
#define NG_ASSET_CATEGORY_NPC 3
```

Current naming rules:

PNG pattern	Mode	Category
[0-9].png	screen	background
background_*.png	screen	background
zz_npc_forest_alley.png	screen	background
sprite_*.png	sprite	main_character
opponent_*.png	sprite	opponent
z_npc_*.png	sprite	npc

Generated Files

The pipeline writes:

- artbox/assets_manifest.json
- artbox/out.srt
- artbox/sprite_meta.h
- artbox/screens.c
- Neo Geo graphics ROM data consumed by the main build

sprite_meta.h exposes NGSpriteAssetMeta records with tile base, palette bank, sprite strips, active rows, content bounds, mode, and category.

Build

Run the full artbox pipeline:

```
cd artbox
./makeartbox.sh
```

Windows:

```
cd artbox
makeartbox.bat
```

If makeartbox.bat fails with PIL/img2neo not available, install packages through the same interpreter used by py:

```
py -0p
py -m pip --version
py -m pip install --upgrade pip
py -m pip install numpy pillow pypng
cd artbox
py -c "import sys; print(sys.executable); import PIL, numpy, png, img2neo; print('OK')"
```

Then rebuild the ROM:

```
make
```

Runtime Use

The 68000 demo includes artbox/sprite_meta.h and uses the generated metadata to place sprite frames without hardcoding each frame's tile offsets, palette bank, active rows, and visible strips.

Gameplay code can use meta->category to distinguish backgrounds, main character frames, opponents, and NPC frames while keeping the graphics ROM layout stable.

Artbox Studio UI

artbox/artbox_studio.py is a PyQt6 desktop tool for inspecting and editing the graphics ROM without running the full pipeline.

```
python3 artbox/artbox_studio.py
```

Tabs

Tab	Function
Tile Grid	Scrollable viewer of all tiles from the combined C-ROM. Zoom 1-6x, per-asset palette, click to select, jump to any asset.
Sprite Designer	Configure a multi-strip sprite: tile base, strips, active rows, stride, palette bank. Exports a ready-to-paste C <code>char_set_sprite</code> call.
Hitbox Editor	Place draggable body and hit rectangles over the sprite preview. Exports a C hitbox struct definition.
Pixel Paint	16-color pixel art editor for patching individual tiles directly in the C-ROM buffer. Saves back to 777-c1.c1 / 777-c2.c2.

Tile decoding follows the exact layout written by `romtiles.py`:

- Each C-ROM stores **64 bytes per 16×16 tile** (4 blocks × 8 rows × 2 bytes).
- Block order: top-right, bottom-right, top-left, bottom-left.
- 777-c1.c1 is byte-flipped (`romts.sh /f`): row bytes are `[plane_a, plane_b]`.
- 777-c2.c2 similarly: `[plane_c, plane_d]`.
- Color index per pixel: `d<<3 | c<<2 | b<<1 | a` (4-bit, maps into `palette[0..15]`).

PART 23

Tools and Utilities

Every helper script that ships with the SDK - art conversion, sound encoding, verification, launchers, installers and the GIMP plug-ins.

`docs/TOOLS.md`

Eagle Software · Neo Geo SDK v1.7.0

Every helper script that ships with the SDK, what it does, and when you would run it by hand. Most of these are invoked for you by a make target — the reason to know them individually is debugging, one-off conversions, and building your own pipeline steps.

Anything under `artbox/` reads and writes the *current game's* `artbox` directory, which the makefiles select through `ARTBOX_DATA_DIR`. Run them through `make art` unless you specifically want to drive one stage on its own.

1. Graphics — `artbox/`

1.1 Conversion cores

These are the scripts that turn pixels into Neo Geo data. One of them is selected per build; the others are alternatives you opt into.

Script	Role			
<code>img2neo_tile.py</code>	The default. Tile-local quantiser: per-tile Lab k-means++, per-tile dither, hardware-lattice palette refit, void-and-cluster blue-noise dithering, greedy MAE bank dedup, Lab-nearest remap. Both makefiles export <code>ARTBOX_TILE=1</code> , so this is the route every ROM takes.			
<code>img2neo_crt.py</code>	CRT-preprocessing route: CIE-Lab k-means, horizontal-biased Floyd-Steinberg, gamma 1.20 / contrast 1.10 pre-boost. <code>make art-crt</code> or <code>ARTBOX_CRT=1</code> .			
<code>img2neo_hd.py</code>	High-quality photo route: bilateral filter, CLAHE on the Lab L channel, unsharp mask, blue-noise dither. CLI flags <code>--no-bilateral</code> , <code>--no-clahe</code> , <code>--no-unsharp</code> , <code>--dither blue</code>	<code>fs\</code>	<code>ordered\</code>	<code>none`</code>
<code>img2neo.py</code>	The original nearest-neighbour path, kept for diffing (<code>ARTBOX_LEGACY=1</code>). Also home to <code>alpha_bleed()</code> and <code>fit_sprite_rgba()</code> , which the other routes use.			
<code>fixtiles.py</code>	FIX-layer (S-ROM) tile conversion.			
<code>fixtiles_hd.py</code>	Per-tile-palette FIX conversion. <code>--sharp-text</code> binarises glyph and HUD sources; writes a sidecar <code>*.pal.json</code> with the per-tile palette table.			

1.2 Pipeline stages

Run in this order by `makeartbox.sh` / `makeartbox.bat`:

Script	What it does
<code>creatoromdb.py</code>	Creates the ROM database (<code>neorom.db</code>) the later stages import into.
<code>romdbimgimport.py</code>	Imports converted sprite and screen images into the ROM database. Owns the screen fitting modes — <code>crop</code> , <code>contain</code> , <code>pad</code> , <code>letterbox</code> — and records the real content box per asset.

Script	What it does
romdbfiximport.py	Imports FIX-layer tiles into the ROM database.
romtiles.py	Packs the database's tile data into the C-ROM pair (<id>-c1.c1, <id>-c2.c2).
genscreens.py	Emits screens.c — one showScreenN() entry point per asset, plus the geometry each needs.
gen_sprite_meta.py	Emits sprite_meta.h from assets_manifest.json — per-asset tile base, strips, rows and padding.
gen_eyecatcher.py	Emits eyecatcher.c from the manifest entries with category == "eyecatcher", as an animation.
gen_assets_cfg.py	Regenerates assets.cfg with the default per-category rules on each make art.
genmapdb.py, genmapfile.py	Build the tile map database and the map file the ROM tooling consumes.
asset_rules.py	The rule engine behind assets.cfg: category matching, filename patterns, fit and anchor resolution. Imported by the stages above rather than run directly.

1.3 Utilities

Script	Use
romtool.py	A portable Python 3 port of ROMWak. Splits and interleaves ROM images — /b splits alternating bytes, and the byte-swap and pad operations the P-ROM build relies on.
validate_assets.py	Checks generated sprite_meta.h for tile-window safety — that every asset's declared window is inside its allocated tile range. python3 artbox/validate_assets.py games/demo/artbox/sprite_meta.h
count_assets.py	Prints the number of assets the current in/ tree will produce. Useful before an id-sensitive change, because an asset id is its position in the build order.
fix_sprite_alpha.py	Repairs sprite transparency before C1/C2 generation, for sources whose alpha channel does not survive an editor round-trip. Run from the repository root.
process_eyecatcher_ng.py	Pre-quantises eyecatcher PNGs onto the Neo Geo colour grid before the main conversion, so the k-means stage starts from colours the hardware can actually hold.
sync_main_screens.py	Syncs the generated showScreenN() declarations and ng_screen_table[] into a game's main.c.
artbox_studio.py	The PyQt6 desktop front end — see §5.

1.4 Shell wrappers

Script	Platform	What it runs
makeartbox.sh / makeartbox.bat	Linux / Windows	The full art build for one game. GAME_ID=777 ./artbox/makeartbox.sh demo
artbox.sh / artbox.bat	both	The bare stage sequence, without the per-game path setup.

Script	Platform	What it runs
romts.sh / romts.bat	both	Sprite-tile ROM packing.
romfx.sh / romfx.bat	both	FIX ROM packing.
makeclean.sh / makeclean.bat, clean.sh / clean.bat	both	Remove generated artbox intermediates.

1.5 GIMP plug-ins — [tools/gimp-plugins/](#)

Manual pre-processing for source art, before it reaches `img2neo*.py`. Two sets of the same three tools: `gimp2.10/` (Python-Fu, `gimpfu`) and `gimp3.2/` (GObject-Introspection, GEGL buffers).

Plug-in	What it does
smart_background_remove	Removes a flat or near-flat background while keeping anti-aliased contours intact.
remove_white_edge_noise	Strips the pale fringe that survives a background removal — the same class of artefact <code>alpha_bleed()</code> handles in the pipeline, but applied at source.
neogeo_hd_pixel_enhance	Sharpens and cleans up-scaled or photographic source so the quantiser has crisper edges to cluster.

2. Sound — [sound/tools/](#)

2.1 Compilers

Script	Input	Output
mml_compile.py	*.mml music tracks	music_data.inc. Handles the SSG/music MML directives, including @aN to fire ADPCM-A sample <i>N</i> from a track.
fm_compiler.py	*.fm FM tracks	fm_data.inc
fm_patch_compiler.py	sound/fm/patches.fm	fm_patch_table.inc — 31-byte patches over four operators
ssg_compile.py	SSG tracks	ssg_data.inc
ssg_config_compile.py	SSG preset config	ssg_config.inc — 6-byte presets
vrom.py	encoded ADPCM banks	the V-ROM, sample_table.inc, and the generated <code>sdk/sound_voice_ids.h</code> . Bundles the voice bank immediately after the SFX bank and exports <code>ADPCMA_VOICE_BASE / ADPCMA_VOICE_COUNT</code> .

2.2 Sample encoding

Script	What it does
adpcm_enc.py	The YM2610 ADPCM encoder, matching real hardware decoding exactly. ADPCM-A is OKI-style 4-bit at a fixed 18.5 kHz with a 12-bit signed accumulator; ADPCM-B is the streamed format.
wav_to_raw_pcm.py	WAV to raw PCM, the stage before ADPCM encoding. Handles 8-, 16-, 24- and 32-bit sources.
enc_wave16le_a.sh / .bat	WAV to 16-bit little-endian PCM for the ADPCM-A bank.
enc_wave16le_b.sh / .bat	The same for the ADPCM-B bank.
enc_wave16le_a_voice.sh / .bat	The same for the voice bank. Skips cleanly when in_wav_a_voice/ is absent.
adpcm_enc_a.sh / .bat	PCM to ADPCM-A.
adpcm_enc_b.sh / .bat	PCM to ADPCM-B.
adpcm_enc_a_voice.sh / .bat	PCM to ADPCM-A for the voice bank.
adpcm_enc_process.sh / .bat	Chains all three encode stages in order: SFX, voice, then ADPCM-B.

The Python path is the default on both platforms. Set `S0X=/path/to/sox` only when you specifically want the SoX conversion route.

2.3 Driver build and comparison

Script	What it does
m1rom.sh / m1rom.bat	Assembles and links the M1 (Z80 driver) ROM with <code>wla-z80 + wlalink</code> .
vrom.sh / vrom.bat	Shell wrappers around <code>vrom.py</code> .
combine_split_driver.py	Merges the fixed Z80 prelude (<code>driver_prelude.asm</code>) with compiler-generated driver assembly. Used only by the experimental C driver path.
compare_m1.py	Compares two generated M1 ROM images and reports the first mismatch, with SHA256 of each. This is what <code>compare-driver</code> uses to check the assembly and C drivers against each other.

2.4 Sound Studio

`sound/sound_studio.py` — the PyQt6 desktop front end. See §5.

3. Verification — tools/

Run by the build, and worth running by hand when something looks wrong.

Script	Checks
<code>check_game_cfg.py</code>	That <code>GAME=</code> matches <code>CURRENT_GAME</code> in the selected <code>game.cfg</code> . Every <code>make</code> target depends on this, and it is what stops one game's assets being built into another game's ROM.
<code>verify_artbox_palettes.py</code>	Per-game artbox palette output: manifest count, palette bank range, and <code>neopal.bin</code> record integrity (136 bytes per record — an int32 index plus sixteen 64-bit palette words). <code>make art</code> runs it and fails the build on a mismatch.
<code>verify_sfix_output.py</code>	The generated FIX ROM: size, format, and tile layout.
<code>gen_api_reference.py</code>	Regenerates <code>docs/API_2D_ENGINE_C.md</code> and <code>docs/API_2D_ENGINE_CPP.md</code> from the engine headers, so the API reference cannot drift from the code. Run it after adding or changing a public engine call.
<code>make_manual_pdf.py</code>	Builds <code>docs/neogeosdk_v1.7.0_manual.pdf</code> — the programmer's manual, the SDK library reference, and both 2D engine references — from the markdown in <code>docs/</code> . Run <code>gen_api_reference.py</code> first if the engine changed.

```
python3 tools/verify_artbox_palettes.py --root . --game demo
python3 tools/verify_sfix_output.py --root .
python3 tools/gen_api_reference.py
python3 tools/make_manual_pdf.py
```

4. Launchers and environment

4.1 Repository root

Script	What it does
<code>neogeosdk.sh</code> / <code>neogeosdk.bat</code>	The interactive build menu — pick a game and a target without remembering the flags. Same as <code>make menu</code> .
<code>run.sh</code> / <code>run.bat</code>	Build and run the current game in MAME.
<code>run_neogeosdk.sh</code> / <code>.bat</code>	Run the built ROM in MAME with the correct rompath, hashpath and BIOS.
<code>run_neogeosdk_debug.sh</code> / <code>.bat</code>	The same with the MAME debugger attached.
<code>run_neogeosdk_lua.bat</code>	Run with a MAME Lua <code>-autoboot_script</code> , for automated inspection and input driving.
<code>run_neogeosdk_trace.bat</code>	Run with MAME tracing enabled.
<code>set_sdkhome.bat</code>	Sets <code>SDKHOME</code> for a Windows shell session.

4.2 dist/

make dist produces per-game run scripts alongside the packaged ROM: run_demo.sh / .bat, run_helloworld.sh / .bat, run_game.sh / .bat, and a _debug variant of each.

4.3 install/

One-shot toolchain installers:

Script	Host
install-linux.sh	Generic Linux
install-ubuntu.sh	Ubuntu, with the distro package names resolved
install-windows.bat	Native Windows
install-wsl.bat	WSL, including the Windows-side setup

See ../install/README.md.

4.4 win/

Prebuilt Windows helper binaries used by MakefileWin32.mak so the Windows build needs no extra installs: srec_cat.exe, srec_cmp.exe, srec_info.exe, xxd.exe.

5. Desktop applications

Both are PyQt6 and both drive the same scripts the makefiles do — they make the feedback loop visual, they do not replace the build.

5.1 Artbox Studio — artbox/artbox_studio.py

Tab	What it does
Pipeline	Runs every conversion step with a status pill per step and a live log. "Run Full Pipeline" chains the standard set; HD variants are opt-in.
Asset Browser	Tree of every PNG under artbox/in/* and artbox/infix/, with dimensions, file size, and a native-scale preview.
Hex Sprite Inspector	Pick a tile index and see its decoded 16 × 16 grid, the raw C1/C2 ROM bytes, and a live palette-swap dropdown. The tool for debugging palette assignment and tile order.
Movement Designer	Build a frame sequence (tile + duration), loop-preview it at a chosen FPS, and export anim_frame_tile[] / anim_frame_dur[] C arrays in vblank units.
Level Designer	Paint a 20 × 14 tilemap with tile thumbnails; export a level_tile[LEVEL_H][LEVEL_W] C array.
HD Compare	Run one PNG through both the standard and HD pipelines and compare the indexed results. Output lands in artbox/out/_hd_cmp/; originals are untouched.
ROM Inventory	Every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size, mtime, and present/absent colour coding.
Asset Rules	Editor for artbox/assets.cfg, with save and reload.

5.2 Sound Studio — `sound/sound_studio.py`

Track, Mix and ROM tabs, a live waveform view, and an MML designer for composing and auditioning music without a full `make_sound` cycle.

Full guide: `SOUND_STUDIO_GUIDE.md`.

6. Debug artefacts

`make debug-artifacts` writes these into `dump/`, using the cross-toolchain's own `binutils`:

File	From	Use
size report	<code>m68k-unknown-elf-size</code>	Section sizes — is the ROM about to overflow
symbol table	<code>m68k-unknown-elf-nm -n</code>	Symbol addresses, which is how you find a variable to watch from a MAME Lua script
ELF headers	<code>m68k-unknown-elf-readelf</code>	Segment layout
linker map	<code>ld -Map</code>	Where every object landed
disassembled by	<code>m68k-unknown-elf-objdump -d</code>	What the compiler actually emitted

`make gdb-trace` runs a batch GDB script and writes `dump/gdb_trace.txt`. See `GDB_GUIDE.md`.